

地效飞行器 RBF 神经网络自适应飞行控制

零基础详解 —— 从物理直觉到公式推导, 从方案选型到代码实现

配套工程: [matlab_simulink/](#)

面向: 机械设计制造及其自动化本科背景

本文档由 [matlab_simulink/tools/pdf/md2pdf_tut.py](#) 自动排版。排版引擎为纯 Python 标准库实现, 内嵌 TrueType 子集字体并写入 /ToUnicode 映射, 因此中文不会乱码, 文字可复制、可检索。

正文以 [docs/tutorial_src/*.md](#) 为唯一事实来源, 改内容只改 Markdown。

目 录

阅读指南	11
这份文档是什么	11
写给谁看	11
怎么读	11
一个重要的阅读提醒	11
全文地图	12
第1章 我们在解决什么问题	13
1.1 什么是地效飞行器	13
1.2 为什么"贴地飞"反而更难控制	13
难点1: 气动参数随高度剧烈变化	13
难点2: 高度静稳定性是负的	13
难点3: 连"浮沉模态"都会失稳	14
1.3 我们的具体对象	14
1.4 我们要做到什么	14
1.5 本章小结	14
1.6 自测题	15
第2章 基础补课	16
2.1 坐标系: 为什么需要两个	16
地面坐标系(惯性系)	16
机体坐标系	16
为什么要分开	16
2.2 三个角度: 迎角、俯仰角、航迹倾角	16
另外一个关键关系	17
2.3 四个力	17
升力 L	17
阻力 D	17
推力 T	18
重力 mg	18
2.4 牛顿第二定律在"运动坐标系"里怎么写	18
切向(沿航迹方向)	18
法向(垂直航迹方向)	18
旋转方程	18
2.5 微分方程与状态变量	19
2.6 线性化: 把非线性问题变成线性问题	19
泰勒展开复习	19
推广到多元	19
2.7 特征值与模态	20
特征值是什么	20
复特征值怎么读	20
机械类比	20
飞机的纵向模态	20
2.8 传递函数、带宽、相位裕度	21
传递函数	21
带宽	21
相位裕度	21

2.9 什么是"稳定"	21
2.10 本章小结	22
2.11 自测题	22
第3章 气动力怎么算	23
3.1 升力从哪来	23
一个常见的错误说法	23
量化: 升力系数	23
3.2 升力线斜率 a 是多少	23
3.3 有限翼展与诱导阻力	23
翼尖涡的形成	23
有效迎角变小	23
升力线斜率被"打折"	24
3.4 地面效应: 镜像涡模型	24
镜像原理	24
对下洗的影响	24
定义一个系数	25
3.5 地效因子 $\kappa(h)$ 长什么样	25
已知的两端	25
选择函数形式	25
本工程的参数	26
3.6 升力线斜率的地效修正(完整推导)	26
第1步: 写出自由空气的关系	26
第2步: 把诱导迎角乘上 κ	26
第3步: 代入	26
第4步: 整理	27
第5步: 得到结果	27
验算与物理检查	27
3.7 诱导阻力的地效修正	27
3.8 平尾: 下洗与操纵	28
平尾感受到的迎角	28
下洗角 ϵ 本身也受地效影响	28
平尾升力与升降舵	28
3.9 俯仰力矩: 一个统一的公式	29
力矩 = 位置矢量 $\times$ 力	29
符号含义的物理解释	29
本工程包含的力矩项	29
3.10 三个"看不见的力"	29
RAM 效应	30
升力非线性(失速软化)	30
参数失配	30
为什么要故意制造失配	30
3.11 本章小结	30
3.12 自测题	30
第4章 运动方程怎么来	32
4.1 状态量怎么选(一个重要的选择)	32
三种常见的选法	32
为什么最终选 C	32

还要加上两个执行器状态	32
4.2 沿航迹方向的方程	32
4.3 法向(垂直航迹) 的方程	33
曲线运动的加速度	33
法向方程	33
4.4 迎角方程	33
4.5 俯仰方程	34
4.6 高度方程	34
4.7 执行器方程(为什么必须建)	34
舵机(一阶惯性环节)	34
发动机	34
为什么必须建模(血的教训)	35
4.8 完整的七阶模型	35
数值积分: 为什么要用定步长 RK4	35
4.9 本章小结	35
4.10 自测题	36
第5章 这架飞机本身稳不稳	37
5.1 配平: 先找一个"什么都不动"的状态	37
什么是配平	37
配平方程	37
怎么解	37
5.2 配平结果说明了什么	37
5.3 线性化: 得到 A、B 矩阵	38
5.4 模态分析: 飞机怎么"晃"	38
5.5 发现一: 高度静稳定性是负的	39
什么是"高度静稳定性"	39
为什么? 四项分解	39
合计与自治性	40
这个分解随高度怎么变	41
这个结论有什么工程价值	41
5.6 发现二: 浮沉模态发散	41
先看"没有地效"的情况	41
用经典理论交叉验证	42
物理图像	42
5.7 一个重要的控制器设计推论	42
5.8 本章小结	43
5.9 自测题	43
第6章 控制方案选型	44
6.1 先把问题定义清楚	44
被控对象	44
控制目标	44
困难清单(按严重程度排)	44
关键判断: D1 是"参数不确定"还是"函数不确定"?	44
6.2 方案一: PID 控制	45
是什么	45
用到本工程会怎样	45
优点	45

缺点(逐条对照困难清单)	45
结论	45
6.3 方案二: 增益调度 PID	45
是什么	45
优点	46
缺点	46
结论	46
6.4 方案三: LQR / 极点配置	46
是什么	46
优点	46
缺点	46
结论	47
6.5 方案四: 动态逆(NDI, Nonlinear Dynamic Inversion)	47
是什么	47
优点	47
缺点(这是关键)	47
结论	47
6.6 方案五: 反步法(Backstepping)	48
是什么	48
优点	48
缺点	48
解决方案: 动态面控制(DSC)	48
结论	48
6.7 方案六: 滑模控制(SMC)	48
是什么	49
缺点	49
结论	49
6.8 方案七: 模型参考自适应控制(MRAC)	49
是什么	49
优点	49
缺点(关键)	50
为什么 MRAC 不够用	50
结论	50
6.9 方案八: L1 自适应控制	50
是什么	50
优点	50
缺点	50
结论	51
6.10 方案九: 神经网络自适应	51
是什么	51
为什么神经网络能解决"函数不确定"	51
关键设计选择: 用哪种网络	51
为什么 RBF 的两个特性这么重要	51
缺点	52
结论	52
6.11 方案组合: 最终架构	52
6.12 一个具体问题: 为什么必须有鲁棒项	53
一个重要的工程陷阱	53

6.13 为什么用三层串级结构	53
三层结构	53
理由 1: 时间尺度分离	53
理由 2: 物理意义清晰	53
理由 3: 可以和"人"比较	54
对比: 为什么不做单回路	54
6.14 完整对比表	54
6.15 本章小结	54
6.16 自测题	54
第 7 章 控制器从零推导	56
7.1 整体思路: 把大问题拆成四个小问题	56
7.2 第一步: 定义误差	56
7.3 第二步: 外环——从高度误差到航迹角指令	56
已知的运动学关系	56
想要什么	56
加上高度率阻尼	57
指令滤波	57
7.4 第三步: 中环——从航迹角到俯仰角(一个漂亮的恒等式)	57
关键恒等式	57
做法	57
为什么这是对的(很重要, 很多人以为是"凑"的)	57
这个结构的一个额外好处	57
加上限幅	58
7.5 第四步: 内环——DSC 动态面	58
目标	58
传统反步法的做法	58
DSC 的做法	58
得到角速度指令	58
再用一次 DSC	58
最终得到期望角加速度	58
7.6 第五步: 动态逆控制律	59
力矩方程	59
反解	59
加上补偿项	59
补偿后发生了什么(关键推导)	59
误差动态	60
7.7 第六步: RBF 神经网络	60
网络结构	60
基函数	60
输入选什么	60
归一化	60
中心和宽度怎么选	61
回归向量的量纲处理(关键细节)	61
7.8 第七步: 自适应律	61
从哪来	61
力矩通道的完整自适应律	62
速度通道的自适应律(符号相反!)	62
投影算子: 保证权值有界	63

7.9 第八步: 鲁棒项	63
7.10 第九步: 速度回路	63
7.11 第十步: Lyapunov 稳定性证明	64
构造 Lyapunov 函数	64
求导并代入	64
剩下的项	64
用不等式放缩	64
得到最终的微分不等式	65
解这个不等式	65
怎么读这个结论	65
级联系统的处理	65
7.12 第十一步: 离散化——一个必须面对的问题	65
问题	65
离散化的稳定性条件	66
代入本工程的数字	67
为什么两个通道的增益数字差这么多	67
7.13 本章小结	68
7.14 自测题	68
第 8 章 代码逐文件讲解	69
8.1 整体结构	69
8.2 为什么这么分文件	69
原则: 一个文件 = 一个物理概念	69
原则: 参数和逻辑分离	69
原则: 同名函数只做一件事	69
8.3 wig_params.m: 参数集怎么组织	70
分层组织	70
几个设计决定	70
8.4 wig_ge.m: 地效因子	70
为什么要有 `span` 参数	71
为什么要返回导数 `dkap`	71
边界处理	71
8.5 wig_aero.m: 气动力	71
整体流程	71
关键决定: `flag` 参数与 `is_truth` 字段	72
输出为什么有两个	72
力矩的统一写法	72
8.6 wig_dynamics.m: 运动方程	72
结构	72
关键细节	73
执行器的速率限制	73
8.7 wig_trim.m: 配平	73
双求解器策略	73
残差的无量纲化	74
阻尼线搜索	74
8.8 wig_rbf_init.m: 网络结构	74
中心为什么用矩阵存	74
宽度为什么取网格间距	75
8.9 wig_ctrl_update.m: 控制器核心(逐段讲解)	75

函数签名	75
第 0 段: 量测滤波	75
第 1 段: RBF 网络求值	76
第 2 段: 高度外环	76
第 3 段: 姿态内环	76
第 4 段: 动态逆	77
第 5 段: 鲁棒项与控制律	78
第 6 段: 速度回路自适应律	78
第 7 段: 自适应律	78
8.10 wig_simulate.m: 仿真引擎	79
核心循环结构	79
参考指令的二阶滤波器	80
8.11 几个 MATLAB 编程要点	80
要点 1: 预分配数组	80
要点 2: 向量化	80
要点 3: <code>`wig_sat`</code> 而不是 <code>`min(max(...))`</code>	80
要点 4: UTF-8 日志	81
要点 5: 异常也要保留输出	81
8.12 本章小结	81
8.13 自测题	81
第 9 章 调试踩过的坑	82
9.1 先说方法: 怎么定位“仿真发散”	82
第 1 步的实际结果	82
第 2 步的实际结果	82
9.2 坑一: 速度通道鲁棒项符号错误	82
症状	82
怎么发现的	83
根因	83
修复	84
为什么这个错误这么隐蔽	84
怎么避免	84
9.3 坑二: 自适应增益与回归向量量纲不匹配	84
症状	84
根因	84
修复	85
教训	85
为什么两个通道的增益数字差这么多	85
一个附加发现	85
9.4 坑三: 鲁棒项边界层过薄	86
症状	86
怎么发现的	86
根因	86
修复	87
教训	87
9.5 坑四: 推力重力前馈构成正反馈	87
症状	87
根因	87
修复	88

教训	88
9.6 坑五: 航迹指令滤波时间常数过大	88
症状	88
根因	88
修复	88
顺带发现的一个细节	89
9.7 整定流程总结	89
第 2 步为什么有效	89
第 3 步为什么要多点	89
第 4 步为什么必须做	89
整定前后的闭环极点对比('log_clp.txt')	90
9.8 五坑速查表	90
9.9 三条通用经验	90
经验 1: 先验证模型, 再怀疑控制器	90
经验 2: 用"结构性推理"缩小范围, 而不是盲目试参数	91
经验 3: 把每个坑写成"症状 -> 判据 -> 修复"的条目	91
9.10 自测题	91
第 10 章 结果解读	92
10.1 地效特性	92
10.2 配平特性	92
10.3 纵向模态	93
10.4 消融对比: 自适应到底有多大用	94
三个关键观察	95
10.5 参数摄动鲁棒性	97
10.6 大气扰动	98
关于 3.0 m/s 阵风的失败	99
10.7 极限低空与量测噪声	100
10.8 蒙特卡洛统计	101
怎么读这些数字	102
10.9 Simulink 与脚本一致性	102
10.10 全部结论	103
做到了什么	103
没做到什么	104
10.11 自测题	104
第 11 章 附录	105
11.1 符号表	105
状态量	105
气动量	105
几何与质量	105
控制器量	106
11.2 术语中英对照	106
11.3 数学工具速查	107
泰勒展开与线性化	107
中心差分	107
复特征值	107
一阶惯性环节	108
欧拉法离散的稳定条件	108

离散自适应律的稳定条件	108
双曲正切不等式	108
常用小角度近似($< 10^\circ$)	108
11.4 常见问题 FAQ	108
11.5 文件清单与运行说明	110
目录结构	110
运行方法	110
重新生成成本教程 PDF	110
11.6 延伸阅读建议	111
飞行力学入门	111
地效飞行器	111
非线性控制	111
自适应控制与神经网络	111
动态面控制	111
建议的阅读顺序	111

阅读指南

这份文档是什么

这是一份**从零讲起**的教程，讲清楚一件事：

一架小型地效飞行器，怎么给它设计一套“自己会适应”的飞行控制器，并且用 MATLAB 把它仿真验证出来。

配套的工程代码在 matlab_simulink/目录下，全部可以运行。本文档不是那个工程的“摘要”，而是**把工程里每一个决定的来龙去脉讲一遍**：为什么这么选、别的选法差在哪、公式是怎么一步步推出来的、代码为什么这么写。

写给谁看

写给**机械设计制造及其自动化**本科背景、毕业一段时间、飞行力学和控制理论基本忘光了的人。

具体来说，我假设你会这些（不会也没关系，第 2 章会补）：

- 微积分：会求导、知道导数的物理意义是“变化率”
- 理论力学： $F = ma$ 、力矩 = 力 $\times$ 力臂、静力平衡
- 线性代数：矩阵乘法、行列式、特征值（哪怕只记得名字）
- MATLAB：会跑脚本、知道函数和数组

我不假设你会这些（这些都在第 2 章和第 6 章从零讲）：

- 飞行力学：迎角、航迹倾角、升力、诱导阻力
- 状态空间、传递函数、根轨迹
- 什么是“自适应控制”、什么是“Lyapunov 稳定性”
- 什么是“神经网络”、什么是“RBF”

怎么读

全文 12 章，建议按顺序读。但如果你时间有限，可以这样跳：

你的目的	读哪几章
只想搞懂“为什么这么设计”	第 1 章 -> 第 6 章 -> 第 9 章
想看懂公式怎么来的	第 2 章 -> 第 3 章 -> 第 4 章 -> 第 7 章
想看懂代码	第 2 章 -> 第 8 章 -> 第 9 章
想直接看结论	第 1 章 -> 第 10 章

每一章末尾有「本章小结」，用几句话总结。每章中间有「为什么不用 xxx」的对比框，专门讲别的方案差在哪。

一个重要的阅读提醒

这份文档里出现的每一个公式，都不是“背下来的”，而是**从物理或者从数学定义一步步推出来的**。你读的时候如果卡住，先问自己一个问题：

这一步等号成立，是因为物理定律，还是因为数学定义？

- 因为**物理定律**（比如 $F = ma$ ）→ 这是模型的一部分，改不了
 - 因为**数学定义**（比如“升力系数定义为 $L / (q \cdot S)$ ”）→ 这是人为约定的，换个约定公式就变
- 分清这两类，公式推导就不难了。

全文地图

- 第1章 我们在解决什么问题 ← 物理直觉
- 第2章 基础补课 ← 数学与力学工具
- 第3章 气动力怎么算 ← 建模第一半
- 第4章 运动方程怎么来 ← 建模第二半
- 第5章 这架飞机本身稳不稳 ← 分析
- 第6章 控制方案选型 ← 为什么选这个方案(重点)
- 第7章 控制器从零推导 ← 公式推导(重点)
- 第8章 代码逐文件讲解 ← 代码为什么这么写(重点)
- 第9章 调试踩过的五个坑 ← 实战经验
- 第10章 结果解读 ← 仿真说明了什么
- 第11章 附录 ← 符号表、术语表、FAQ

第 1 章 我们在解决什么问题

1.1 什么是地效飞行器

地效飞行器（Wing-In-Ground effect vehicle，简称 WIG）是一种贴着地面或海面飞的飞行器。它飞得极低——低到机翼离地只有几十厘米，甚至十几厘米。

为什么要飞这么低？因为**飞得低，升力大，阻力小**。

这件事听起来有点反直觉。我们先做一个实验想象一下：

你在桌面上方 5 厘米处水平放一块硬纸板，然后快速往下压。
你会感觉到一股**明显的阻力**——空气被压在纸板和桌面之间，出不去，形成一个“气垫”。

这就是地面效应的直觉版本。机翼贴着地面飞的时候，机翼下方的空气被“困住”，压力升高，于是升力变大；同时，机翼后方的尾涡被地面“挡住”了一部分，诱导阻力变小。

在航空界，这个现象的量化说法是：

- 升力可以增大 **20% ~ 50%**
- 诱导阻力可以下降 **50% ~ 90%**
- 综合结果是**升阻比**（升力除以阻力，衡量“省不省油”的核心指标）可以从常规飞机的 15 左右提高到 **20 ~ 30**

所以地效飞行器的定位很清楚：**在特定航线上，用接近船舶的油耗，跑出接近飞机的速度。**

1.2 为什么“贴地飞”反而更难控制

飞得低，好处是气动效率高；坏处是——**你没有容错空间，而且气动特性随高度剧烈变化。**

具体有三个难点，后面的章节会一个个用数字证明：

难点 1：气动参数随高度剧烈变化

常规飞机飞在 1000 米和 1010 米，气动特性几乎没区别。但地效飞行器飞在 5 厘米和 15 厘米，气动特性差别巨大：

离地高度 h	h/b （除以翼展）	升力线斜率相对自由空气
0.05 m	0.021	+34%
0.20 m	0.083	+24%
0.50 m	0.208	+12%
1.20 m	0.500	+4%

（ 这些数字来自本工程的 run_analysis 计算结果，见第 10 章）

含义：控制器如果按“高度 20 厘米”调好了参数，飞到 5 厘米时这套参数就完全不对了。而且这个变化不是缓慢的——它是一个**非线性函数**。

难点 2：高度静稳定性是负的

这是最要命的一条。

常规飞机有个“自动回中”的特性：如果它意外下沉，机头会自动抬起，把它推回去。这叫**静稳定**。

地效飞行器**恰恰相反**。本工程算出来的结果是：

离地高度每减小 1 米，飞机会额外受到约 **10.8 牛·米** 的低头力矩。

低头意味着什么？机头朝下 -> 加速下沉 -> 离地更近 -> 低头力矩更大 -> **正反馈，越来越糟**。

这叫**地效高度不稳定**，是地效飞行器的固有问题，不是设计缺陷。第 5 章会详细拆解它由哪几部分组成。

难点 3: 连"浮沉模态"都会失稳

飞机受扰后会有几种固定的"晃动方式"，航空上叫**模态**。其中有一种叫**浮沉模态**(英文 phugoid)——飞机像海豚一样，一会儿爬升减速、一会儿俯冲加速，周期大约 10 秒。

- 在**常规飞机**上，浮沉模态是**临界稳定的**(几乎不衰减，但也不发散)，飞行员不太管它也没事
- 在**地效飞行器**上，本工程算出来它变成了**发散的**，发散时间常数最短只有约 2 秒

也就是说：**你不主动控制，它自己就会散架。**

1.3 我们的具体对象

本工程的对象是一架**小型地效无人机**:

参数	数值	说明
翼展 b	2.40 m	两个翼尖之间的距离
弦长 c	0.40 m	机翼前后方向的宽度
机翼面积 S	0.96 m ²	翼展 × 弦长
质量 m	12 kg	
巡航速度 v	18 m/s	约 65 km/h
飞行高度 h	0.05 ~ 0.50 m	这就是"地效区"
最大推力	35 N	推重比约 0.30

几个值得注意的点:

- **h/b 只有 0.02 ~ 0.21**，比常规飞机低两个数量级。所以地面效应是**主导效应**，不是修正项。
- **巡航高度只有 20 厘米**，比一张 A4 纸的短边还短。飞机稍微低个头就可能擦地。
- **这是无人机不是载人机**，所以可以用比较激进的控制方案。

1.4 我们要做到什么

目标很朴素:

让这架飞机在 0.05 ~ 0.50 米的高度带内，按指令稳定地升降、保持速度，即使我们对它的气动特性了解得并不准。

关键在最后半句：**"即使我们对它的气动特性了解得并不准"**。

这句话是整个课题的核心。因为:

- 工程上用的地效估算公式，精度通常只有 ± 30% 到 ± 50%
- 真实飞行器还有制造公差、结冰、地面粗糙度、波浪等各种扰动
- 如果控制器必须"精确知道"气动参数才能工作，那它在工程上就没有价值

所以我们要的不是"调到最优的控制器"，而是**"参数不准也能活下来的控制器"**。这就是"自适应控制"要解决的问题。

1.5 本章小结

- 地效飞行器靠贴近地面飞行获得高升阻比，代价是**极低的高度裕度和剧烈变化的气动特性**。
- 它有三个固有难题：气动随高度剧变、高度静稳定性为负、浮沉模态失稳。
- 本工程对象：翼展 2.4 m、质量 12 kg、飞行高度 0.05~0.50 m 的小型无人机。
- 核心目标不是"最优控制"，而是**"在模型不准的前提下仍然安全"**——这决定了后面所有的方案选择。

1.6 自测题

1. 为什么常规飞机不需要担心"高度"这个变量，地效飞行器却必须把它当成核心状态量？
2. "高度静稳定性为负"意味着什么？请用一句话描述它导致的恶性循环。
3. 如果地效估算公式的误差有 $\pm 40\%$ ，那么一个"精确依赖该公式"的控制器会有什么风险？

第2章 基础补课

这一章把后面要用到的概念全部补一遍。**已经会的可以跳过，但建议扫一眼**，因为我用的记号和约定可能和你当年学的的不完全一样。

2.1 坐标系：为什么需要两个

要描述一架飞机的运动，第一件事是**规定从哪儿看**。

地面坐标系(惯性系)

固定在地面上，x 轴指向前方，z 轴指向上方。飞机的高度 h 就是在这个坐标系里量的。

机体坐标系

固定在飞机上，跟着飞机一起转：

- x 轴：从机尾指向机头(叫"机体纵轴")
- y 轴：指向右翼
- z 轴：指向机腹(**注意：指向下**)

为什么 z 轴朝下？这是航空界的传统约定，原因很实际：飞机在平飞时重力沿 +z 方向，写方程时符号好记。

为什么要分开

因为**不同的物理量在不同的坐标系里最自然**：

物理量	在哪个系里最自然	原因
高度 h	地面系	地面不会动
升力 L 、阻力 D	气流方向	升力永远垂直于气流，阻力永远平行于气流
俯仰力矩 M	机体系	飞机绕自己的横轴转

关键的翻译关系：机体坐标系的 x 轴(机头方向) 和气流方向之间差一个角度，这个角度就是**迎角 α** 。

2.2 三个角度：迎角、俯仰角、航迹倾角

这是初学者最容易搞混的地方。用一张图来理解：



三者的关系是一个**恒等式**(不是近似，是定义)：

$$\theta = \gamma + \alpha$$

用大白话讲：

- **俯仰角 θ** ：飞机**姿态**(机头指哪儿)
- **航迹倾角 γ** ：飞机**实际运动方向**(往哪儿飞)
- **迎角 α** ：机翼**相对气流的迎风角度**(决定产生多少升力)

打个比方，这就像开车：

- θ 是车头朝向
- γ 是实际行驶方向
- α 是侧风相对于车头的角度

如果车在笔直前进($\gamma = 0$) 但方向盘打着($\theta = 5$ 度) , 那车身和运动方向就有夹角——飞机上对应的就是迎角 $\alpha = \theta - \gamma = 5$ 度。

为什么这个恒等式重要: 第7章会看到, 控制器里有一行 $\theta_c = \gamma_c + \alpha$, 它的正确性完全依赖这个恒等式。搞懂这一条, 那一行代码就懂了。

另外一个关键关系

高度变化率和航迹倾角的关系:

$$dh/dt = V \cdot \sin(\gamma)$$

意思很直白: **往上飞的速率 = 速度 × 爬升角的正弦**。这和"斜坡上汽车的高度变化率 = 车速 × $\sin(\text{坡度})$ "是同一件事。

2.3 四个力

飞机在纵向平面内受四个力。

升力 L

垂直于气流方向, 向上。来源是机翼上下表面的压力差。

工程上把它写成**无量纲系数**的形式(这是航空界的惯例, 因为这样可以跨尺寸对比) :

$$L = q \cdot S \cdot C_L$$

其中:

- $q = 0.5 \cdot \rho \cdot v^2$, 叫**动压**, 单位是帕斯卡(Pa) , 代表"气流冲击强度"。 ρ 是空气密度(海平面约 1.225 kg/m^3)
- S 是机翼面积(m^2)
- C_L 是**升力系数**, 无量纲

C_L 怎么来? 在小迎角范围内, 它和迎角成线性关系:

$$C_L = a \cdot (\alpha - \alpha_{L0})$$

- a 叫**升力线斜率**, 单位是 $1/\text{弧度}$ (每增加1 弧度迎角, 升力系数增加多少)
- α_{L0} 叫**零升迎角**, 是"升力为零时的迎角"。因为机翼通常有弯度, 所以这个角是负的(约 -3 度)

物理直觉: 迎角越大, 气流被偏折得越厉害, 升力越大。但迎角大到一定程度(约 15 度) 气流会分离, 升力突然掉下来——这就是**失速**。

阻力 D

平行于气流, 向后。分两部分:

$$D = q \cdot S \cdot (C_{D0} + C_{Di})$$

- C_{D0} : **废阻力系数**, 来自摩擦、形状、支架等, 和升力无关
- C_{Di} : **诱导阻力系数**, 和升力的平方成正比

诱导阻力是什么? 这是初学者最困惑的概念, 我们花点篇幅讲清楚。

机翼产生升力的原理是"把气流向下偏折"。但机翼是有限长的, 翼尖处的空气会从高压的下表面绕到低压的上表面, 形成**翼尖涡**。这两个涡在机翼后面拖出两条长长的尾迹, 它们在机翼处产生一个**向下的速度分量**(叫**下洗**)。

机翼本来感觉自己迎角是 α , 但因为有了下洗, 实际感受到的有效迎角变小了。要产生同样的升力, 就得把机头再抬高一点——而抬高机头就意味着升力的方向往后偏了一点, 产生一个额外的阻力分量。

这就是**诱导阻力**:

$$C_{Di} = C_L^2 / (\pi \cdot AR \cdot e)$$

- AR: **展弦比** = 翼展² / 机翼面积。翼展越长越细, 翼尖涡相对影响越小, 诱导阻力越小(这就是滑翔机翼展特别长的原因)
- e: **奥斯瓦尔德效率因子**, 约 0.8~0.9, 用来修正"机翼不是理想椭圆分布"带来的损失

记住一句话: 诱导阻力来自翼尖涡, 翼尖涡来自有限翼展。而地面效应**正好能削弱翼尖涡**——这就是地效飞行器省油的物理本质。

推力 T

由发动机产生, 沿推力线方向。本工程最大 35 N。

重力 mg

永远竖直向下, $12 \text{ kg} \times 9.8 \text{ m/s}^2 = 117.7 \text{ N}$ 。

2.4 牛顿第二定律在"运动坐标系"里怎么写

这是本章最重要的一节。

你本科学的是: $\mathbf{F} = m\mathbf{a}$ 。在静止坐标系里, $\mathbf{a} = d\mathbf{v}/dt$, 没什么花样。

但现在我们要在**跟着飞机转动的坐标系**里写方程, 这里有个关键区别: **物体沿曲线运动时, 加速度有两个分量**。

切向(沿航迹方向)

飞机沿航迹方向加速或减速, 加速度就是速度的变化率 dV/dt :

$$m \cdot dV/dt = T \cdot \cos(\alpha + i_T) - D - m \cdot g \cdot \sin(\gamma)$$

右边三项分别是:

- 推力的沿航迹分量(推力线可能有个安装角 i_T)
- 阻力(向后, 减速)
- 重力的沿航迹分量(爬升时重力拖后腿, 减速)

法向(垂直航迹方向)

这里是关键: 飞机沿曲线飞行时, 即使速度大小不变, 速度的**方向**在变, 所以有加速度。这个加速度叫**向心加速度**, 大小是:

$$a_{\text{向心}} = V \cdot (d\gamma/dt)$$

(你可能记得向心加速度 = V^2/R , 其中 R 是转弯半径。因为 $R = V/(d\gamma/dt)$, 代入就得到 $V \cdot d\gamma/dt$)

所以法向方程是:

$$m \cdot V \cdot d\gamma/dt = L + T \cdot \sin(\alpha + i_T) - m \cdot g \cdot \cos(\gamma)$$

机械类比: 这就是"汽车过拱桥"的方程。桥顶时重力向下、支持力向上, 两者之差提供向心力。飞机爬升时, "桥面"就是弯曲的航迹。

旋转方程

绕横轴的转动, 和你在理论力学里学的刚体转动完全一样:

$$I_{yy} \cdot dq/dt = M$$

- I_{yy} : 绕横轴的转动惯量($\text{kg} \cdot \text{m}^2$), 本对象是 1.15

- q : 俯仰角速度(rad/s) , $d\theta/dt = q$
- M : 俯仰力矩(N·m)

类比: 这就是 $J\ddot{\theta} = M$, 和转子动力学里的转动方程一模一样。

2.5 微分方程与状态变量

把上面几个方程合起来, 我们得到一组**联立一阶微分方程**:

$$\begin{aligned}dV/dt &= f_1(V, \gamma, \alpha, q, h, T, \delta e) \\d\gamma/dt &= f_2(\dots) \\d\alpha/dt &= f_3(\dots) \\dq/dt &= f_4(\dots) \\dh/dt &= f_5(\dots) \\dT/dt &= f_6(\dots) \\d\delta e/dt &= f_7(\dots)\end{aligned}$$

括号里那串 $(V, \gamma, \alpha, q, h, T, \delta e)$ 就是**状态变量**——它们是"描述系统当前情况所需的最小信息集合"。

为什么叫"状态": 只要知道这 7 个数在某一时刻的值, 再加上从这一刻起的控制输入, 未来所有时刻的行为就唯一确定了。就像刚体运动只要知道位置和速度, 后续轨迹就定了。

为什么要把所有方程写成一阶: 因为数值积分算法(我们用的是龙格-库塔法) 的标准输入格式就是"一阶微分方程组"。把高阶方程降成一阶是标准操作:

$$\begin{aligned}d^2\theta/dt^2 = M/I &\rightarrow \text{令 } q = d\theta/dt \\&\text{则 } dq/dt = M/I\end{aligned}$$

这样就把一个二阶方程变成了两个一阶方程。

代码对应: wig_dynamics.m 就是把这 7 个 $d\gamma/dt$ 算出来放进一个 7×1 的列向量返回。

2.6 线性化: 把非线性问题变成线性问题

我们的方程里有 $\sin(\gamma)$ 、 $\alpha \cdot v^2$ 这类非线性项, 没法直接用线性控制理论。怎么办? **在工作点附近做泰勒展开, 只保留一次项。**

泰勒展开复习

对一元函数 $f(x)$, 在 x_0 附近:

$$f(x) \approx f(x_0) + f'(x_0) \cdot (x - x_0) + (\text{高阶项})$$

物理含义: 任何光滑曲线, 在足够小的范围里看起来都像直线。斜面的斜率就是导数。

推广到多元

对 $f(x_1, x_2, \dots, x_n)$:

$$f(x) \approx f(x_0) + \sum_i [\partial f / \partial x_i | x_0] \cdot (x_i - x_{i0})$$

那些偏导数排成一个矩阵, 就是**雅可比矩阵**:

$$A = \begin{bmatrix} \partial f_1 / \partial x_1 & \partial f_1 / \partial x_2 & \dots \\ \partial f_2 / \partial x_1 & \partial f_2 / \partial x_2 & \dots \\ \dots & \dots & \dots \end{bmatrix}$$

于是非线性方程 $dx/dt = f(x, u)$ 变成线性方程:

$$d(\Delta x)/dt = A \cdot \Delta x + B \cdot \Delta u$$

其中 $\Delta x = x - x_0$ 是**小偏差**, $B = \partial f / \partial u$ 是输入矩阵。

工程含义: A 矩阵描述"飞机本身的性质", B 矩阵描述"舵面有多大的操纵能力"。这两个矩阵是后面所有分析的出发点。

代码对应: wig_linearize.m 用**中心差分**算这些偏导数——不去推导解析公式, 而是直接数值求:

$$\partial f / \partial x_i \approx [f(x + \Delta x_i) - f(x - \Delta x_i)] / (2\Delta x_i)$$

为什么要数值求? 因为解析推导太容易出错, 而且模型一改就得重推。数值差分虽然慢一点, 但永远不会"推错"。

2.7 特征值与模态

特征值是什么

对于一个方阵 A, 如果存在非零向量 v 和数 λ 使得:

$$A \cdot v = \lambda \cdot v$$

那么 λ 叫**特征值**, v 叫**特征向量**。

物理含义(这一条比定义重要得多) :

特征值 λ 告诉你系统某个"固有运动模式"的**衰减快慢**和**振荡频率**; 特征向量 v 告诉你这个模式里**哪些状态在动**、**动的比例是多少**。

复特征值怎么读

特征值通常是复数: $\lambda = \sigma + j\omega$

- **实部 σ :** 衰减率。 $\sigma < 0$ **稳定**(指数衰减) , $\sigma > 0$ **发散**(指数增长)
- **虚部 ω :** 振荡频率(rad/s)
- 对应的时间常数 = $1/|\sigma|$ (只和实部有关)

再定义两个常用量:

- **固有频率 $\omega_n = |\lambda| = \sqrt{(\sigma^2 + \omega^2)}$:** 振荡快慢
- **阻尼比 $\zeta = -\sigma/\omega_n$:** 振荡衰减程度
- $\zeta > 1$: 过阻尼, 不振荡, 慢慢回到平衡
- $0 < \zeta < 1$: 欠阻尼, 振荡着衰减
- $\zeta = 0$: 等幅振荡(临界)
- $\zeta < 0$: **振荡着发散**

机械类比

这和你做振动实验时的单自由度系统完全对应:

$$m \cdot x'' + c \cdot x' + k \cdot x = 0$$

特征方程 $m \cdot \lambda^2 + c \cdot \lambda + k = 0$, 解出来的 λ 就是特征值。

- c 是阻尼(对应 σ : 阻尼越大衰减越快)
- k 是刚度(对应 ω : 刚度越大振得越快)
- 如果阻尼是**负的**(比如干摩擦引起的自激振动) , 系统就会**自激发散**——这就是 $\zeta < 0$ 的情况

地效飞行器的问题就是: 地面效应给浮沉模态加了一个"负阻尼"。

飞机的纵向模态

飞机纵向有几种典型模态, 名字很形象:

模态	周期	特征	物理图像
短周期	1~3 秒	迎角快速变化, 速度几乎不变	机头上下点头
浮沉	5~15 秒	速度与高度交换, 迎角几乎不变	像海豚一样起伏
高度	更慢	高度缓慢漂移	整体升沉

为什么短周期和浮沉能分开: 因为它们的时间尺度差很多(1 秒 vs 10 秒) 。数学上这叫"时间尺度分离", 工程上可以分别处理——这也是我们控制器分内外环的依据。

2.8 传递函数、带宽、相位裕度

传递函数

对线性系统, 输入和输出的拉普拉斯变换之比就是**传递函数**。比如一阶惯性环节:

$$G(s) = 1 / (\tau s + 1)$$

- **τ 是时间常数:** 输入突变后, 输出大约经过 3τ 到达新稳态的 95%
- **舵机的动态**就是一个一阶惯性环节, 本工程舵机时间常数 $\tau_e = 0.05$ 秒

带宽

带宽是"系统能跟得上多快的输入"。粗略地说, 带宽(rad/s) 约等于 $1/\tau$ 。

本工程各回路的带宽:

- 舵机: $1/0.05 = 20$ rad/s
- 俯仰角速度内环: 约 6 rad/s
- 俯仰角中环: 约 3 rad/s
- 高度外环: 约 2 rad/s

串级控制的基本原则: 内环带宽必须远高于外环(至少 3~5 倍) 。否则外环设计时"假设内环瞬间跟得上"就不成立, 整个设计会崩。

相位裕度

反馈控制里最重要的安全指标。**相位裕度**是"系统开环频率响应的相位距离 -180° 还有多少度"。

- 相位裕度 $> 45^\circ$: 安全
- 相位裕度 $30^\circ \sim 45^\circ$: 勉强
- 相位裕度 $< 30^\circ$: 振荡甚至发散

为什么: 负反馈靠"反相"工作。如果因为延迟, 信号在闭环绕一圈后相位已经翻了 180° , 负反馈就变成了**正反馈**——这就是自激发散。

教训: 滞后环节(延迟) 是控制回路的天敌。本工程第 9 章会看到, 好几个坑都是滞后引起的。

2.9 什么是"稳定"

控制系统里"稳定"有好几个层次, 本工程都会用到:

名称	含义	判据
李雅普诺夫意义下稳定	初始偏差不会越来越大	有界
渐近稳定	偏差最终收敛到零	特征值实部全负
一致最终有界(UUB)	偏差收敛到某个小球内	本工程的目标
输入输出稳定	有界输入产生有界输出	

为什么本工程只能做到 UUB 而不是渐近稳定:

因为我们用了神经网络去"逼近"未知函数。而神经网络**永远只能逼近，不能精确等于**——总有一个残差 ε 。只要 $\varepsilon \neq 0$ ，误差就不可能收敛到零，只能收敛到一个和 ε 成比例的小球里。

但这已经足够工程使用了：**只要那个小球比允许误差小就行。**

2.10 本章小结

- 需要**两个坐标系**：地面系量高度，机体系写力矩。迎角 α 就是两者的"翻译角度"。
- 三个角度的恒等式 $\theta = \gamma + \alpha$ 是后面控制器结构的基础。
- 升力 $L = q \cdot S \cdot C_L$ ，诱导阻力 $C_{Di} = C_L^2 / (\pi \cdot A \cdot Re)$ ，都写成无量纲系数形式。
- **曲线运动有法向加速度** $V \cdot d\gamma/dt$ ，这是纵向运动方程里最容易漏的一项。
- 把高阶方程降成一阶方程组，状态变量是"描述系统的最小信息集合"。
- **线性化 = 泰勒展开取一次项**，得到的 A、B 矩阵是控制设计的出发点。
- **特征值的实部决定稳定、虚部决定振荡**；阻尼比 $\zeta < 0$ 就是自激发散。
- **相位裕度**衡量安全余量，而**滞后是相位裕度的杀手**。

2.11 自测题

1. 一架飞机俯仰角 8° 、迎角 4° ，它的航迹倾角是多少？它在爬升还是下降？
2. 为什么诱导阻力随升力**平方**增长？请从翼尖涡的角度解释。
3. 某系统特征值为 $-2 \pm 5j$ ，它的时间常数、固有频率、阻尼比各是多少？稳定吗？
4. 为什么串级控制要求内环带宽远高于外环？如果内外环带宽接近会发生什么？
5. 为什么用了神经网络之后，误差只能收敛到一个"小球"而不是零？

第3章 气动力怎么算

这一章解决“飞机受了多大的力和力矩”。这是建模的一半，另一半（这些力怎么让飞机运动）在第4章。

3.1 升力从哪来

一个常见的错误说法

很多人说“机翼上方气流走得快、压力低，所以产生升力”——这个说法**不完全对**。正确的说法是：

机翼把气流向下偏折，气流反过来把机翼向上推。

这是牛顿第三定律。机翼每个时刻都在把大量空气往下推，空气就给机翼一个向上的反作用力。

那机翼为什么能把气流往下偏折？因为机翼有**迎角**和**弯度**——气流撞上来，被“铲”向下方。

量化：升力系数

把升力写成无量纲形式：

$$L = q \cdot S \cdot C_L \quad \text{其中 } q = 0.5 \cdot \rho \cdot V^2$$

C_L 在小迎角下和迎角成正比：

$$C_L = a \cdot (\alpha - \alpha_{L0})$$

为什么要用“系数”这种绕弯的写法？

因为这样一来， C_L 就只和**形状**（翼型、迎角）有关，而与**尺寸、速度、空气密度**无关。这意味着：

- 风洞里用 1/10 缩比模型测出来的 C_L 曲线，可以直接用到全尺寸飞机上
- 换一个飞行速度，不用重测气动，只要改 q 就行

这是航空工程能发展的基础。

3.2 升力线斜率 a 是多少

理论上，二维翼型（无限长机翼）的升力线斜率是：

$$a_0 = 2\pi \approx 6.28 \text{ (每弧度)}$$

实际因为空气有粘性、翼型有厚度，大概是 $2\pi \times 0.9 \approx 5.7 \sim 5.9$ 。本工程取 $a_0 = 5.90 \text{ /rad}$ 。

但真实的机翼是**有限长**的，所以实际升力线斜率要比 a_0 小。原因就是我们下面要讲的诱导阻力。

3.3 有限翼展与诱导阻力

翼尖涡的形成

机翼下表面压力高、上表面压力低。到了翼尖，高压空气会绕过翼尖“漏”到上表面去，形成两个旋转的涡——**翼尖涡**。

这两个涡在机翼后方拖出两条长长的尾迹，涡的旋转在机翼所在位置诱导出一个**向下的速度**，叫**下洗速度 w** 。

有效迎角变小

机翼几何上是 α ，但因为气流本身在往下走，机翼实际“看到”的气流方向是斜向下的，所以**有效迎角**变小了：

$$\alpha_{\text{有效}} = \alpha - \alpha_i$$

其中 $\alpha_i = w / V$ 叫**诱导迎角**。

升力线斜率被"打折"

现在把「有效迎角」代回升力公式:

$$C_L = a_0 \cdot (\alpha - \alpha_i - \alpha_{L0})$$

但 α_i 本身又和 C_L 有关。对椭圆升力分布, 有:

$$\alpha_i = C_L / (\pi \cdot AR \cdot e)$$

(AR 是展弦比, e 是效率因子)

把两个式子联立:

$$C_L = a_0 \cdot (\alpha - C_L / (\pi \cdot AR \cdot e) - \alpha_{L0})$$

把带 C_L 的项移到左边:

$$C_L \cdot [1 + a_0 / (\pi \cdot AR \cdot e)] = a_0 \cdot (\alpha - \alpha_{L0})$$

于是得到**有限翼展的实际升力线斜率**:

$$a = a_0 / [1 + a_0 / (\pi \cdot AR \cdot e)]$$

验算本对象: $a_0 = 5.90$, $AR = 6.0$, $e = 0.85$

$$\begin{aligned} \pi \cdot AR \cdot e &= 3.1416 \times 6.0 \times 0.85 = 16.02 \\ a &= 5.90 / (1 + 5.90 / 16.02) = 5.90 / 1.368 = 4.31 \text{ /rad} \end{aligned}$$

比二维值 5.90 下降了 27%。**展弦比越小, 下降越多**——这解释了为什么滑翔机要做得那么细长。

3.4 地面效应: 镜像涡模型

现在进入本章核心。**地面为什么会增强升力?**

镜像原理

这是流体力学里的一个经典技巧, 叫**镜像法**。

对于一块无限大的固体平板(地面), 要让"地面处法向速度为零"这个边界条件成立, 可以在**地面下方对称位置放一个"镜像"的涡系**, 其旋向与真实涡系**相反**。

打个比方: 你站在镜子前, 镜子里有一个"你", 动作完全对称。如果真实世界和镜像世界叠加, 镜面处的效果正好抵消。

对下洗的影响

真实翼尖涡在机翼处诱导**下洗**(把气流往下带) 。

镜像涡旋向相反, 所以它在机翼处诱导的是**上洗**(把气流往上带) !

两者叠加: **下洗被削弱了**。

极端情况:

- **无限高:** 镜像涡离得无限远, 影响为零 -> 下洗完整保留 -> 就是自由空气

- **贴地($h = 0$):** 镜像涡就在机翼正下方, 上洗完全抵消下洗 -> 下洗为零

这就是地面效应的物理机制: 地面把机翼"自己吹自己"的那部分效应挡住了。

定义一个系数

为了把这件事量化，我们定义**地效因子** κ (读作 kappa)：

$$\kappa = \text{贴地时的下洗} / \text{自由空气的下洗}$$

取值范围:

- $\kappa = 1$: 完全无地效(自由空气)
- $\kappa = 0$: 完全贴地(下洗被完全抵消)
- $0 < \kappa < 1$: 真实情况

于是:

$$\begin{aligned} \text{诱导迎角: } \alpha_i(\text{地效}) &= \kappa \cdot \alpha_i(\text{自由空气}) \\ \text{下洗速度: } w(\text{地效}) &= \kappa \cdot w(\text{自由空气}) \end{aligned}$$

为什么说 κ 就是"下洗保留系数": 因为它表示"自由空气的下洗还剩多少没被地面抵消"。 κ 越小，地面效应越强。

3.5 地效因子 $\kappa(h)$ 长什么样

现在需要一个**函数形式**来描述 κ 随高度 h 怎么变。

已知的两端

- $h \rightarrow 0$ 时, $\kappa \rightarrow 0$
- $h \rightarrow \infty$ 时, $\kappa \rightarrow 1$
- h 用什么归一化? 用**翼展 b** , 因为地面效应本质是"地面挡住了翼尖涡", 而翼尖涡的尺度就是翼展。所以用无量纲高度 $x = h/b$ 。

选择函数形式

我们选了:

$$\kappa(x) = x^n / (A + x^n)$$

为什么是这个形式，而不是别的？逐条对比:

候选形式	优点	缺点	结论
$\kappa = 1 - A/x$	简单	$h \rightarrow 0$ 时发散到负无穷，不物理	×
$\kappa = \exp(-A/x)$	$h \rightarrow 0$ 时趋零，形状合理	$h \rightarrow \infty$ 时趋 1 很慢；参数不好拟合；不易求导	△
$\kappa = 1/(1+A/x^n)$	两端行为正确	与上面的形式等价，但 $h \rightarrow 0$ 时的幂次行为不直观	△
$\kappa = x^n/(A+x^n)$	两端行为正确； $h \rightarrow 0$ 时 $\kappa \propto x^n$ ；单调；只有一个形状参数 A 和一个指数 n ，拟合稳定	$h \rightarrow 0$ 时导数为 0 (x^n 的 $n > 1$ 时)，与部分实验数据在极低空的趋势略有差异	✓ 选它

这个形式的另一个好处: 它可以直接做**线性化拟合**。

把式子变形:

$$\begin{aligned} 1/\kappa &= (A + x^n)/x^n = 1 + A/x^n \\ 1/\kappa - 1 &= A/x^n \\ \ln(1/\kappa - 1) &= \ln(A) - n \cdot \ln(x) \end{aligned}$$

令 $y = \ln(1/\kappa - 1)$ 、 $u = \ln(x)$ ，就变成一条**直线**:

$$y = \ln(A) - n \cdot u$$

也就是说：只要有一组实测的 (h, κ) 数据，取两次对数就能用最小二乘拟合出 A 和 n 。

不需要任何非线性优化。这是选择这个函数形式的一个很实际的理由。

(代码对应: wig_ge_fit_cfd.m。本工程用它做了一次往返自检: ANSYS/data/ge_kappa_template.csv 是由本工程自己的解析式生成的模板, 拟合以 $R^2 = 0.99874$ 反演出 $A = 0.06423$ 、 $n = 1.45030$, 与生成时用的 $0.0641 / 1.4622$ 一致。这证明了标定算法本身写对了, 但不构成任何模型验证——把模型喂给算法再把参数拿回来, 只是自洽性检查, 不是与真实物理的比对。真正开展 CFD 之后, 应当用 CFD 算出的 $\kappa(h)$ 替换模板数据, 那时才谈得上"标定"和"验证"。)

本工程的参数

取 A 、 n 的原则是：让 $\kappa(h)$ 跟随公开文献里地效因子随高度衰减的公认趋势，而不是去拟合任何实测数据。也就是说

本工程没有做过 CFD，也没有做过风洞试验。 $A = 0.0641$ 、 $n = 1.4622$ 是根据公开趋势选定的一组自洽参数，取值使 κ 在 $h/b \approx 0.02$ 时降到 5% 量级、在 $h/b \rightarrow 1$ 时趋于 1，与文献报道的形态一致。这是"工程估算"，不是"标定结果"。

$$A = 0.0641, n = 1.4622$$

算出来:

h (m)	h/b	κ	物理含义
0.05	0.021	0.052	下洗只剩 5%，地效极强
0.10	0.042	0.130	
0.20	0.083	0.292	巡航点
0.35	0.146	0.483	
0.50	0.208	0.612	下洗还剩 61%
1.00	0.417	0.813	地效已弱
∞	∞	1.000	自由空气

3.6 升力线斜率的地效修正(完整推导)

有了 κ ，就能推出地效下的升力线斜率。这一步是本章最重要的推导，我们慢慢来。

第 1 步：写出自由空气的关系

前面已经推过:

$$\begin{aligned} C_L &= a_0 \cdot (\alpha - \alpha_i - \alpha_{L0}) \\ \alpha_i &= C_L / (\pi \cdot AR \cdot e) \end{aligned}$$

第 2 步：把诱导迎角乘上 κ

地效把诱导迎角削弱了:

$$\alpha_i(\text{地效}) = \kappa \cdot C_L / (\pi \cdot AR \cdot e)$$

第 3 步：代入

$$C_L = a_0 \cdot [\alpha - \kappa \cdot C_L / (\pi \cdot AR \cdot e) - \alpha_{L0}]$$

第4步：整理

$$C_L + a_0 \cdot \kappa \cdot C_L / (\pi \cdot AR \cdot e) = a_0 \cdot (\alpha - \alpha_{L0})$$

$$C_L \cdot [1 + a_0 \cdot \kappa / (\pi \cdot AR \cdot e)] = a_0 \cdot (\alpha - \alpha_{L0})$$

第5步：得到结果

$$a_w(h) = a_0 / [1 + a_0 \cdot \kappa(h) / (\pi \cdot AR \cdot e)]$$

这就是地效修正后的升力线斜率。

验算与物理检查

h (m)	κ	a_w (1/rad)	相对自由空气
0.05	0.052	5.79	+34.3%
0.20	0.292	5.33	+23.5%
0.50	0.612	4.82	+11.7%
∞	1.000	4.31	基准

物理检查（这一步很重要，推完公式一定要回头看看合不合理）：

- $\kappa = 1$ 时，公式退化成普通有限翼展公式 ✓
- $\kappa = 0$ 时， $a_w = a_0$ ，即升力线斜率趋近二维值——因为下洗为零，等效于无限翼展 ✓
- 地效越强（ κ 越小）， a_w 越大 ✓（贴地增升）

为什么最大只能到 $a_0 = 5.90$ ：因为地面效应只能“消除下洗”，不能“制造额外的环量”。消除下洗的极限就是等效无限翼展，也就是 a_0 。这是这个模型的内在上限。

引申：这个模型低估了什么？ 真实的极低空飞行（ $h/c <$

0.2）时，机翼下方的“气垫”效应会产生**额外**的压力升高，升力可以超过 a_0 给出的上限。这在工程上叫 **RAM 效应**。本工程的处理方式很聪明：**把它作为“真实对象有、但控制器模型里没有”的一项**，专门用来考验自适应控制器。（见 3.9 节）

3.7 诱导阻力的地效修正

诱导阻力和下洗成正比，所以直接乘 κ ：

$$C_{Di}(\text{地效}) = \kappa(h) \cdot C_{L^2} / (\pi \cdot AR \cdot e)$$

为什么：诱导阻力本质是“升力矢量因为下洗而向后倾斜”，倾斜角就是诱导迎角 α_i 。 α_i 变成 κ 倍，阻力分量自然也变成 κ 倍。

算一下：

h (m)	κ	诱导阻力相对值
0.05	0.052	5.2%
0.20	0.292	29.2%
0.50	0.612	61.2%

在 5 厘米高度，诱导阻力只剩自由空气的 5%——**这就是地效飞行器升阻比能到 20 以上的原因。**

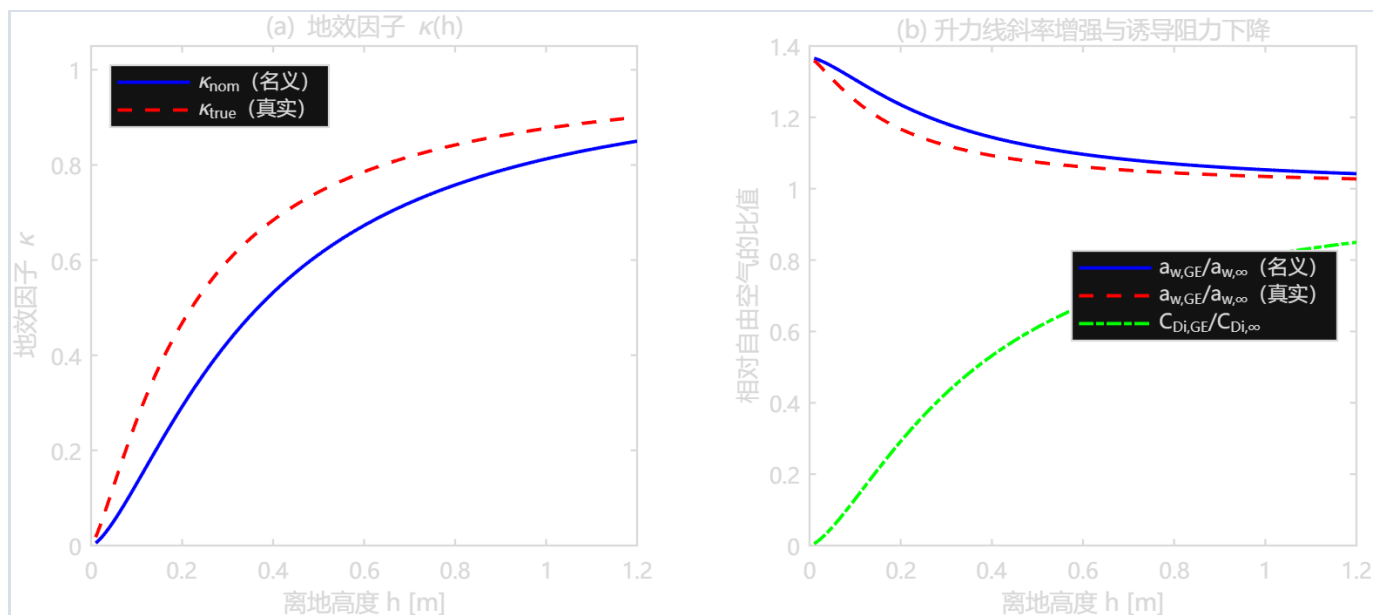


图 3-1 地效因子与升力线斜率、诱导阻力随高度的变化

3.8 平尾：下洗与操纵

飞机后面有个水平尾翼(平尾)，它的作用是**配平和操纵俯仰**。

平尾感受到的迎角

平尾在机翼后面，所以它处在机翼的下洗气流里。它的当地迎角是：

$$\alpha_t = \alpha + i_t - \varepsilon(h, \alpha) + q \cdot l_t / V$$

逐项解释：

项	含义
α	机身迎角
i_t	平尾安装角(本对象 -2°)
$-\varepsilon$	减去下洗角(机翼把气流往下带，平尾看到的迎角变小)
$+q \cdot l_t / V$	俯仰角速度的贡献(飞机低头时，平尾相对气流下沉，迎角变大)

最后一项很重要：它是**天然的俯仰阻尼**。飞机一低头，平尾迎角变大、升力变大，产生抬头力矩把它拉回来。**没有这一项，飞机就会像陀螺一样越转越快。**

下洗角 ε 本身也受地效影响

$$\varepsilon(h, \alpha) = (\varepsilon_0 + \varepsilon_\alpha \cdot \alpha) \cdot \kappa(h)$$

- ε_0 ：常数部分(约 0.02 rad)
- ε_α ：随迎角增长的部分(约 0.35)

注意：地效让下洗变小(乘了 κ)，所以平尾看到的迎角会**变大**。这个细节在第 5 章会导致一个严重问题——**平尾在低空会产生额外低头力矩**，这正是地效高度不稳定的主因。

平尾升力与升降舵

$$C_{Lt} = a_t(h_t) \cdot (\alpha_t - \alpha_{L0t}) + a_{\delta e} \cdot \delta_e$$

- a_t ：平尾的升力线斜率，同样按 3.6 节公式修正，但用的是**平尾高度** $h_t = h + \Delta z_t$ (本对象平尾比机翼高 0.30)

m ，因为它是抬高安装的)

- δ_e : 升降舵偏角。约定: 正 δ_e 表示后缘向下偏, 会让平尾升力增大, 产生低头力矩
- $a_{\delta_e} = 1.60 / \text{rad}$: 升降舵的操纵效能

注意符号约定: 正的升降舵偏角 = 低头。这是航空界的标准约定(和"拉杆抬头"正好相反, 因为拉杆时升降舵后缘是向上偏的) 。

3.9 俯仰力矩: 一个统一的公式

力矩的计算很容易出错, 因为涉及的力多、力臂多、符号多。本工程用了一个**统一公式**, 值得单独讲。

力矩 = 位置矢量 $\times$ 力

对机体坐标系(y 轴指向右翼) , 绕 y 轴的力矩是:

$$M = z \cdot F_x - x \cdot F_z$$

其中 (x, z) 是力的作用点相对于**重心**的坐标(x 向前为正, z 向下为正) , (F_x, F_z) 是力的分量。

这个公式怎么来的: 力矩 $= r \times F$ (叉乘) 。叉乘展开后, 绕 y 的分量正好是 $z \cdot F_x - x \cdot F_z$, 另外两项是绕 x 和 z 轴的(纵向运动里我们不管) 。

符号含义的物理解释

情况	数学	物理
升力(向上, $F_z < 0$) 作用在重心 之后 ($x < 0$)	$M = -x \cdot F_z = (-) \cdot (-) = \text{正}$	抬头?

等等, 这里要小心。我们重新推一遍:

升力向上, 所以 $F_z = -L$ (因为 z 轴朝下) 。

作用点在重心之后, 所以 $x = -l$ (l 是距离, 正数) 。

代入:

$$M = 0 - (-l) \cdot (-L) = -l \cdot L < 0$$

$M < 0$ 意味着低头。而物理上"在重心后面^上推", 确实会让机头**低**下去(想象跷跷板, 从后面往上顶, 前面就往下沉) 。

结论: $M = z \cdot F_x - x \cdot F_z$, x 向前为正、 z 向下为正、抬头力矩为正。这三个符号约定必须一致, 否则整个模型就错了。

本工程包含的力矩项

来源	说明
机翼升力/阻力	作用点在气动中心
机翼自身俯仰力矩 C_{mac}	弯度翼型固有的低头力矩(-0.05)
平尾升力/阻力	力臂最长, 是主要的配平手段
推力偏置	推力线不在重心上(本对象高 3 cm)
机体力矩	机身细长, 有小量贡献

代码对应: wig_aero.m 里每个力矩都按同一个公式写, 只是代入不同的 (x, z) 和 (F_x, F_z) 。这样不容易错。

3.10 三个"看不见的力"

真实的飞行器还有三个效应, 本工程的处理很关键——**故意把它们放进"真实对象"但不放进"控制器模型"**。

RAM 效应

贴地飞行时，机翼下方的空气被“困住”，压力升高，产生额外的升力。

本工程模型：

$$\Delta C_{L,RAM} = k_{ram} / (1 + (h/h_{ram})^2)$$

- $k_{ram} = 0.09$ ：幅度
- $h_{ram} = 0.25 \text{ m}$ ：特征高度

在 $h = 0.05 \text{ m}$ 时约贡献 $+0.086$ 的升力系数（约占巡航升力的 14%）。

关键细节：RAM 力的作用点在气动中心之后 5 cm（因为高压区偏后），所以它还附带一个低头力矩。

升力非线性（失速软化）

迎角大时升力不再线性增长。本工程用平滑的 $\tanh$ 函数模拟：

$$C_{L,实际} = C_{Lmax} \cdot \tanh(C_{L,线性} / C_{Lmax})$$

- $C_{Lmax} = 1.3$

用 $\tanh$ 而不是硬切换的原因是**可导**——数值积分和线性化都要求函数光滑。

参数失配

真实飞行器的气动系数和估算值总有偏差。本工程设置了：

参数	名义值	真实值	偏差
地效强度 A	0.0641	0.045	地效更强
地效指数 n	1.4622	1.300	衰减更慢
废阻力 C_{D0}	0.025	0.032	+28%
下洗梯度 ϵ_α	0.35	0.45	+29%

为什么要故意制造失配

这是整个课题成立的基石。

如果“真实对象”和“控制器模型”完全一样，那么一个简单的动态逆控制器就能做到完美——**仿真什么也证明不了**。

只有当真实对象和模型有显著差异时，“自适应”才有存在的意义。本工程设置的失配幅度（升力差最大 13.9 N，占重量的 11.8%）代表了“工程估算式、未经 CFD 或风洞验证”的典型情况——**这也正是本工程真实的处境：地效参数只跟住了公开趋势，没有任何 CFD 数据支撑**。

3.11 本章小结

- 升力用无量纲系数 $L = q \cdot S \cdot C_L$ 表示，好处是跨尺寸、跨速度通用。
- **有限翼展产生诱导阻力：** $a = a_0 / [1 + a_0 / (\pi A Re)]$ ，本对象 5.90 降到 4.31。
- **地面效应的物理本质是镜像涡削弱下洗**，用系数 κ 描述。
- 选 $\kappa = x^n / (A + x^n)$ 的理由：两端行为正确 + 可化为直线做最小二乘拟合。
- **A、n 是按公开趋势选定的工程估算值，不是 CFD/风洞标定结果**（`ge_kappa_template.csv` 由本工程解析式生成，拟合它是往返自检，不是验证）。
- **地效修正升力线斜率：** $a_w = a_0 / [1 + a_0 \kappa / (\pi A Re)]$ ，5 cm 处增大 34%。
- 俯仰力矩统一用 $M = z \cdot F_x - x \cdot F_z$ ，符号约定必须一致。
- 真实对象还包含 RAM 效应、失速软化和参数失配——**故意不告诉控制器**。

3.12 自测题

1. 为什么 $\kappa = 0$ 时升力线斜率趋近 2π , 而不是无限大?
2. 展弦比从 6 增加到 12, 升力线斜率会怎么变? 诱导阻力会怎么变?
3. 平尾在机翼后面, 它感受到的迎角比机身迎角大还是小? 为什么?
4. 升力作用在重心之后, 产生抬头还是低头力矩? 用 $M = z \cdot F_x - x \cdot F_z$ 验算。
5. 为什么要把 RAM 效应放进"真实对象"而不放进"控制器模型"?

第 4 章 运动方程怎么来

第 3 章算出了"飞机受了多少力和力矩"。这一章把这些力变成"飞机怎么动"。

4.1 状态量怎么选(一个重要的选择)

要写运动方程, 先要决定"用哪些量描述飞机的状态"。

三种常见的选法

方案	状态量	优点	缺点	是否采用
A. 机体轴速度分量	u, w, q, θ, h	力矩方程形式最简洁	u, w 的物理意义对控制不直观; 气动力要另外换算角度	×
B. 风轴(气流轴)	v, α, q, θ, h	气动力可直接代入	高度环设计时还要自己算 γ	△
C. 风轴 + 航迹角	v, γ, α, q, h	高度变化率直接由 γ 给出; 气动力直接可用	需要多一步 $\theta = \gamma + \alpha$ 的换算	√

为什么最终选 C

三个理由:

理由 1: 高度是核心被控量, 而 $\dot{h}/dt = V \cdot \sin(\gamma)$ 。

如果状态里只有 θ 和 α , 那么 $\gamma = \theta - \alpha$, 每次算高度变化率都要多算一次减法。虽然只是个小差别, 但**控制器的外环设计直接依赖这个关系**, 用 γ 做状态可以让外环设计变得非常直观。

理由 2: 气动力直接是 (v, α) 的函数。

升力 $L = q \cdot S \cdot C_L(\alpha)$, 阻力 $D = q \cdot S \cdot C_D(\alpha)$, 动压 $q = 0.5 \rho v^2$ 。如果状态用 u, w , 那么 $v = \sqrt{u^2 + w^2}$ 、 $\alpha = \arctan(w/u)$, 每次算气动力都要做两次非线性运算, 既慢又容易在数值上出问题。

理由 3: θ 不是独立的。

因为 $\theta = \gamma + \alpha$ 是恒等式, 所以 θ 是**冗余的**。把它当独立状态会让状态方程出现"0 = 0"的退化行, 数值求解时雅可比矩阵奇异。

所以: **状态量取 $[v, \gamma, \alpha, q, h]$ 五个**, θ 随时可以用 $\gamma + \alpha$ 算出来。

还要加上两个执行器状态

除了 5 个刚体状态, 还要加上:

- T : 发动机推力(N)
- δ_e : 升降舵实际偏角(rad)

为什么? 见 4.7 节。这是本工程调试阶段最关键的一个决定。

最终状态向量(7 阶) :

$$x = [v, \gamma, \alpha, q, h, T, \delta_e]^T$$

4.2 沿航迹方向的方程

在航迹坐标系(一个跟着飞机转的坐标系) 里, 沿飞行方向应用牛顿第二定律。

受力分析(沿航迹方向, 向前为正) :

力	表达式	方向
推力分量	$T \cdot \cos(\alpha + i_T)$	向前(如果推力线有安装角 i_T)
阻力	$-D$	向后
重力分量	$-m \cdot g \cdot \sin(\gamma)$	爬升时向后

得到:

$$m \cdot dV/dt = T \cdot \cos(\alpha + i_T) - D - m \cdot g \cdot \sin(\gamma)$$

逐项检查符号(这类方程最常错在符号上) :

- 爬升($\gamma > 0$) 时 $\sin(\gamma) > 0$, 重力项为负 -> 减速 ✓
- 俯冲($\gamma < 0$) 时 $\sin(\gamma) < 0$, 重力项为正 -> 加速 ✓
- 推力越大越加速 ✓; 阻力越大越减速 ✓

4.3 法向(垂直航迹) 的方程

这是**最容易漏掉一项**的地方。很多人写 $m \cdot V \cdot d\gamma/dt = L - mg \cdot \cos(\gamma)$ 就交卷了, 忘了这是曲线运动。

曲线运动的加速度

飞机沿弯曲航迹飞行时, 速度**大小**和**方向**都在变。加速度分解成两个垂直分量:

- **切向加速度** dV/dt (改变速度大小)
- **法向加速度** $V \cdot d\gamma/dt$ (改变速度方向, 即向心加速度)

向心加速度的推导(帮你回忆一下) :

物体以速度 V 沿半径 R 的圆运动, 转一圈用时 $T = 2\pi R/V$ 。速度矢量转一圈也是 2π , 所以角速度 $\omega = 2\pi/T = V/R$ 。向心加速度 $a = V \cdot \omega = V^2/R$ 。

因为 $\omega = d\gamma/dt$, 且 $R = V/(d\gamma/dt)$, 所以 $a = V^2 / (V/(d\gamma/dt)) = V \cdot (d\gamma/dt)$ ✓

法向方程

法向(向上为正) 受力:

力	表达式
升力	L
推力法向分量	$T \cdot \sin(\alpha + i_T)$
重力法向分量	$-m \cdot g \cdot \cos(\gamma)$

得:

$$m \cdot V \cdot d\gamma/dt = L + T \cdot \sin(\alpha + i_T) - m \cdot g \cdot \cos(\gamma)$$

平飞时验算($\gamma = 0$, 加速度为零) :

$$L + T \cdot \sin(\alpha + i_T) = m \cdot g$$

即"升力加上推力法向分量等于重力"——这就是平飞的条件 ✓

机械类比: 把航迹看成一座拱桥, 飞机在上面跑。桥对飞机的"支撑"就是升力。桥越弯($d\gamma/dt$ 越大), 需要的支撑力越大。

4.4 迎角方程

迎角是"机体轴和气流方向的夹角", 所以它的变化率取决于**机头转得多快**和**航迹转得多快**之差。

由恒等式 $\alpha = \theta - \gamma$ (注意 θ 的定义是 $\theta = \gamma + \alpha$, 所以 $\alpha = \theta - \gamma$) 对时间求导:

$$d\alpha/dt = d\theta/dt - d\gamma/dt = q - d\gamma/dt$$

其中 $q = d\theta/dt$ 是俯仰角速度。

物理含义:

- 机头转得快(q 大) $\rightarrow$ 迎角增大
- 航迹转得快($d\gamma/dt$ 大) $\rightarrow$ 迎角减小

举例: 飞机平飞时突然把机头拉起($q > 0$)。航迹还没来得及弯($d\gamma/dt \approx 0$), 所以迎角立刻增大, 升力增大——这就是"拉杆上升"的机理。

4.5 俯仰方程

刚体绕横轴的转动:

$$I_{yy} \cdot dq/dt = M$$

- I_{yy} : 绕 y 轴的转动惯量, 本对象 $1.15 \text{ kg}\cdot\text{m}^2$
- M : 总的俯仰力矩(抬头为正), 由第3章的 $M = z \cdot F_x - x \cdot F_z$ 累加得到

这一条方程是控制器的主战场: 升降舵主要通过改变 M 来控制飞机, 所以第7章的动态逆控制律就是围绕这个方程设计的。

4.6 高度方程

由2.2节的运动学关系:

$$dh/dt = V \cdot \sin(\gamma)$$

注意这是纯运动学, 不涉及力。它说明: 想改变高度, 唯一的手段是改变 γ 。

这条方程决定了控制器的外环结构: 外环把"高度误差"翻译成"航迹角指令"。

4.7 执行器方程(为什么必须建)

刚体方程写完了, 但**真实飞机不是理想的**: 舵不会瞬间转到指令位置, 发动机推力也不会瞬间建立。

舵机(一阶惯性环节)

$$\tau_e \cdot d\delta_e/dt = \text{sat}(\delta_e, \text{指令}) - \delta_e$$

- $\tau_e = 0.05 \text{ s}$: 舵机时间常数
- $\text{sat}(\cdot)$: 位置限幅, 本对象 $\pm 25^\circ$

为什么航空舵机是一阶惯性: 它是电液或电机驱动, 有惯量和阻尼, 响应速度有限。写成一阶惯性是最简单的合理模型:

- 指令突变 $\rightarrow$ 舵面按指数规律趋近目标
- 时间常数 $0.05 \text{ s} \rightarrow$ 约 0.15 s 到达目标(3τ)

发动机

$$\tau_T \cdot dT/dt = T_{\text{指令}} - T$$

- $\tau_T = 0.15 \text{ s}$
- 推力范围 $0 \sim 35 \text{ N}$

为什么必须建模(血的教训)

本工程调试时，几乎所有发散问题都和这两个滞后有关。原因在 2.8 节讲过：滞后吃掉相位裕度。

给个具体数字：在 6.5 rad/s 的频率上，0.05 秒舵机滞后贡献的相位滞后是：

$$0.05 \times 6.5 = 0.325 \text{ rad} = 18.6^\circ$$

看起来不多。但加上俯仰角滤波器、角速度滤波器、采样延迟，总滞后可以轻松超过 60° 。而 $\zeta = 0.3$ 的系统相位裕度本来就只有约 33° ——一加就翻车。

如果不在模型里建执行器滞后，仿真会给出一个“性能很好”的假象，实际上天就摔。

4.8 完整的七阶模型

把上面所有方程汇总：

```
dV/dt = [ T*cos(alpha+delta_T) - D - m*g*sin(gamma) ] / m
dgamma/dt = [ L + T*sin(alpha+delta_T) - m*g*cos(gamma) ] / (m*V)
dalpha/dt = q - dgamma/dt
dq/dt = M / I_yy
dh/dt = V*sin(gamma)
dT/dt = (T_指令 - T) / tau_T
ddelta_e/dt = sat((delta_e,指令 - delta_e)/tau_e, 速率限幅)
```

代码对应：wig_dynamics.m，逐行就是这个。

数值积分：为什么要用定步长 RK4

把上面这组方程交给计算机求解，需要选一个积分算法：

方法	优点	缺点	是否采用
欧拉法	最简单	精度一阶，步长必须极小	×
ode45(变步长 RK45)	自适应步长，通用	步长不固定 -> 结果不可复现；和 Simulink 定时调度不一致	×
定步长 RK4	四阶精度；步长固定 -> 完全可复现；与 Simulink 的 ode4 一致	需要自己估计合适步长	✓

RK4 的公式(不用记，知道它是“用四个斜率的加权平均”就行)：

```
k1 = f(x)
k2 = f(x + h/2*k1)
k3 = f(x + h/2*k2)
k4 = f(x + h*k3)
x_新 = x + h/6*(k1 + 2k2 + 2k3 + k4)
```

为什么选步长 1 ms：

- 舵机时间常数 0.05 s，1 ms 相当于每 50 步采样一次 -> 足够
- 控制器采样 200 Hz(5 ms)，1 ms 让每个控制周期有 5 个积分步 -> 够用

4.9 本章小结

- 状态量取 $[V, \gamma, \alpha, q, h, T, \delta_e]$ ，选 γ 而不选 θ 是因为高度环设计需要它，且 θ 冗余。
- 纵向运动方程有三条主线：切向 $F=ma$ 、法向向心加速度、绕横轴转动。
- 法向方程里的 $m \cdot V \cdot d\gamma/dt$ 是最容易漏的一项。
- $d\alpha/dt = q - d\gamma/dt$ 说明迎角是“机头转角和航迹转角之差”。
- 执行器滞后必须建模，否则仿真会给出虚假的好结果。

- 数值积分用**定步长 RK4**，保证可复现且与 Simulink 一致。

4.10 自测题

1. 为什么把 θ 当状态量会导致方程组退化？
2. 飞机以 18 m/s、航迹倾角 5° 爬升，爬升率是多少？如果俯仰角是 8° ，迎角是多少？
3. 写出平飞时法向方程的简化形式。如果升力突然增大 20%，飞机最初的响应是什么？
4. 舵机时间常数从 0.05 s 增大到 0.15 s，对控制回路有什么影响？为什么？
5. 为什么不用变步长积分器做这种控制仿真？

第5章 这架飞机本身稳不稳

模型建好了，下一个问题是：**这架飞机自己能不能飞得稳？**如果不能，难在哪里？

这一章是全文的一个重点，因为**后面控制器为什么要那样设计，完全由这里的结论决定。**

5.1 配平：先找一个"什么都不动"的状态

什么是配平

配平就是找一个状态和一组控制量，使得**所有导数都等于零**：

$$dV/dt = d\gamma/dt = d\alpha/dt = dq/dt = dh/dt = 0$$

物理上就是：**飞机保持水平匀速直线飞行，永远这样飞下去。**

这和你做静力学题时"求平衡位置"是同一件事。

配平方程

令 $\gamma = 0$ (平飞)、 $q = 0$ (不转)，代入第4章的方程：

- (1) 沿航迹: $T \cos(\alpha + i_T) - D = 0$
- (2) 法向: $L + T \sin(\alpha + i_T) - m \cdot g = 0$
- (3) 力矩: $M = 0$

三个方程，三个未知数：迎角 α 、升降舵 δ_e 、推力 T 。

(速度和高度是我们指定的，不是未知数——我们说"在18 m/s、0.2米高度上配平"，就是要找这一时刻的 α 、 δ_e 、 T)

为什么方程(1)里没有重力项：因为 $\gamma = 0$ 时 $\sin(\gamma) = 0$ ，重力在航迹方向没有分量 $\checkmark$

为什么方程(2)里是 $-m \cdot g$ 而不是 $-m \cdot g \cdot \cos(\gamma)$ ：因为 $\gamma = 0$ 时 $\cos(\gamma) = 1 \checkmark$

怎么解

这三个方程**互相耦合**，而且是非线性的：

- α 影响升力和力矩
- δ_e 影响力矩，也轻微影响升力
- T 影响两个力方程，还通过推力偏置影响力矩

没法手算，得用数值方法。本工程用 MATLAB 的 `fsolve` (信赖域算法)。同时写了一个**阻尼牛顿法**作为后备——因为 `fsolve` 属于 Optimization Toolbox，不是所有安装都有。

牛顿法的思想(你本科数值分析学过)：

1. 猜一个解 x_0
2. 算残差 $r = f(x_0)$ (离零有多远)
3. 算雅可比矩阵 $J = \partial f / \partial x$ (残差对解的敏感度)
4. 解线性方程 $J \cdot \Delta = -r$ 得到修正量
5. $x_1 = x_0 + \Delta$
6. 重复直到残差足够小

为什么要"阻尼"：直接走满一步可能冲过头甚至发散，所以每次只走一部分(线搜索)。这是工程实现里的常见做法。

5.2 配平结果说明了什么

在18 m/s 下对不同离地高度配平：

h (m)	迎角 (°)	升降舵 (°)	推力 (N)	升阻比 L/D
0.05	3.11	-4.70	7.08	16.6
0.15	3.85	-4.44	8.26	14.2
0.25	4.43	-4.17	9.16	12.8
0.35	4.82	-3.92	9.80	12.0
0.50	5.18	-3.61	10.43	11.2

(这些是"真实对象"的结果)

三个观察:

观察 1: 升阻比随高度增加而下降。

从 5 厘米的 16.6 降到 50 厘米的 11.2。这是地效减弱的直接后果——**飞得越低越省油**，这正是地效飞行器的价值所在。

观察 2: 迎角随高度增加而增大。

从 3.11° 增到 5.18°。因为地效减弱了，升力线斜率降低，要产生同样的升力就得加大迎角。

观察 3: 升降舵偏角在 -3.6° ~ -4.7° 之间。

负值意味着"后缘向上偏"，即平尾产生**向下的力**。这是常规布局飞机的常态：**平尾是"配重"**，往下压，把机头抬起来平衡机翼的低头力矩。

这些配平量对控制器意味着什么：如果控制器用的模型和真实对象不一样，它算出来的配平舵角就不对，会产生稳态误差。表格中"名义模型"和"真实对象"的升降舵差约 0.8°，就是控制器必须在线补偿的量。

5.3 线性化: 得到 A、B 矩阵

配平点找到了，接下来在这个点附近做线性化(见 2.6 节) 。

做法: 用中心差分数值求偏导。

A 的每一列: 对某个状态加一个小扰动，看所有导数怎么变
B 的每一列: 对某个控制量加一个小扰动，看所有导数怎么变

为什么用数值差分而不是推导解析式:

方法	优点	缺点
解析推导	精确、快	极易出错 ；模型一改就得重推；表达式极其冗长
数值差分	自动跟随模型变化；不会推错；实现简单	稍慢；步长要选好

对于有几十项气动力公式的模型，数值差分是**工程上唯一可行的选择**。

步长怎么选: 本工程用 $\Delta = 10^{-6} \times \max(1, |x|)$ 。太小会有浮点舍入误差，太大会引入非线性误差。10⁻⁶ 是常用的折中。

5.4 模态分析: 飞机怎么"晃"

对 A 矩阵求特征值，就得到飞机的**固有模态**(见 2.7 节) 。

本对象在 h = 0.20 m 处的特征值:

特征值	类型	含义
-4.44 ± 8.54j	短周期	周期 0.74 s，阻尼比 0.46(良好)
+0.4875 ± 0.3507j	浮沉-高度耦合	实部为正，发散!

特征值	类型	含义
-1.58	高度实根	快速收敛
-6.67	发动机	时间常数 0.15 s
-20.0	舵机	时间常数 0.05 s

两个执行器极点分别对应发动机和舵机的时间常数，和理论值 $1/\tau$ 完全一致——这说明线性化程序是对的(一个很好的自检方法) 。

短周期很好，浮沉模态发散。下面重点分析后者。

5.5 发现一：高度静稳定性是负的

什么是"高度静稳定性"

和"俯仰静稳定性"类比:

- **俯仰静稳定性**: 迎角增大时，是否产生恢复力矩(低头) ? 判据 $\partial M/\partial \alpha < 0$
- **高度静稳定性**: 高度减小时，是否产生恢复力矩(抬头，把飞机推回去) ?

稳定的条件是: 高度减小($dh < 0$) -> 应该产生抬头力矩($dM > 0$) -> 即 $\partial M/\partial h < 0$

本工程的计算结果($h = 0.200$ m 配平点) : $\partial M/\partial h = +10.8157$ N·m/m > 0 —— **不满足判据，高度不稳定。**

为什么？四项分解

这个正号是哪来的？我们把 $\partial M/\partial h$ 拆成四项(外加两项恒为零的) 来看。

先说清算法: 代码(wig_mdh_decomp.m) 并不是照着手推公式去凑一个数，而是把 `wig\_aero.m` 里已经分开算好的每一个力矩分量，各自对 h 做中心差分:

$$\partial M_{\text{分量}}/\partial h \approx [M_{\text{分量}}(h+\Delta h) - M_{\text{分量}}(h-\Delta h)] / (2\Delta h), \Delta h = 1e-4 \text{ m}$$

其余状态冻结在配平值(这正是 Irodov 高度稳定性判据里 $\partial C_m/\partial h$ 的定义) 。所以下面每一个数字都是**程序直接输出的**，不是解析近似。

在 $h = 0.200$ m、 $V = 18$ m/s 的真实对象配平点上，六个分量是:

分量	数值(N·m/m)	性质
机翼	-1.2075	稳定
平尾	+10.3302	主导不稳定项
RAM 附加力	+1.2208	不稳定
升力非线性软化	+0.4723	不稳定
机体	0.0000	与 h 无关
推力线偏置	0.0000	与 h 无关
分量之和	+10.8157	

六项相加等于总量，残差只有 $1.4e-13$ N·m/m(log_stab.txt 与 log_mdh.txt 第 2

节都打印了这个自洽性校核) 。这一点很重要: 它说明分解是**完备的**——没有任何一项被漏掉，也没有"未计入的高阶项"这种含糊说法。分解可信，就是因为这个残差是机器精度量级。

下面逐项看物理。

第1项: 机翼升力变化 -> 稳定, -1.2075 N·m/m

高度降低 -> 地效增强 -> 升力增大($\partial L/\partial h = -88.676$ N/m, 即每降低 1 米升力增加约 88.7 N) 。

机翼气动中心在重心**前方** 2 cm($x_{acw} = +0.02$ m) , 所以升力增大产生的是**抬头力矩**, 量级就是:

$$\partial M_{\text{翼}}/\partial h \approx x_{\text{acw}} \times \partial L/\partial h = 0.02 \times (-88.676) \approx -1.8 \text{ N}\cdot\text{m}/\text{m}$$

负号 = 稳定。高度降低 -> 抬头 -> 爬回去 ✓

这个手算值(约 -1.8) 与程序中心差分给出的 **-1.2075** 同号、同量级。两者不完全相等是因为程序算的是**完整的** $M_w = z_{\text{acw}} \cdot F_{x_w} - x_{\text{acw}} \cdot F_{z_w} + q \infty S c \cdot C_{\text{mac}}$ (含少量随 α 变化的角度项)，而手算只取了主导项 $-x_{\text{acw}} \Delta L$ 。**能对上就说明分解的物理机理是对的**；要引用精确数值时，用程序的 -1.2075。

但请注意这个量**很小**(约 -1.2 N·m/m，只占总量 10.8157 的约 11%)，原因是力臂只有 2 cm。**这一项之所以小，就是因为气动中心只比重心靠前 2**

cm——不是因为它被什么大数抵消了。反过来说，如果为了增大这一项而把气动中心大幅前移，会同时牺牲俯仰静稳定性，得不偿失。

第 2 项：平尾下洗变化 -> 不稳定，+10.3302 N·m/m

这是**主因**，值得慢慢讲。

平尾在机翼后面，处在机翼的下洗气流里。配平时平尾产生的是**向下的力**(前面说过，平尾是"配重")。

现在高度降低：

1. 地效增强，下洗角 ϵ 变小(因为 $\epsilon = (\epsilon_0 + \epsilon_{\alpha} \cdot \alpha) \cdot \kappa$ ， κ 变小)
2. 平尾感受到的迎角 $\alpha_t = \alpha + i_t - \epsilon$ **变大**
3. 平尾的升力系数增大——但平尾本来是**负升力**(向下)，升力系数增大意味着**向下的力变小**
4. 平尾向下的力变小 -> 它的抬头力矩变小 -> **等效于低头力矩增大**

这条因果链最后一步在代数上就是：

$$\partial M_{\text{尾}}/\partial h = -l_t \times \partial L_t/\partial h$$

($l_t = 0.95 \text{ m}$ 是尾力臂；平尾向下的力是负的，力变小即 $\partial L_t/\partial h < 0$ ，于是 $\partial M_{\text{尾}}/\partial h > 0$ 。)

不必手算这个偏导数——手算要把 $\partial \kappa/\partial h$ 、 $\partial \alpha_t/\partial h$ 、 $\partial C_{L_t}/\partial h$

一层层串起来，每一层都是近似，算出来的数对不上程序输出(这正是本教程早期版本出过的错)。直接**引用程序中** **中心差分的结果**：+10.3302 N·m/m。

正号 = 不稳定。高度降低 -> 低头 -> 加速下沉 ×

第 3 项：RAM 效应 -> 不稳定，+1.2208 N·m/m

RAM 附加力的作用点在气动中心**之后** 5 cm($x_{\text{ram}} = -0.03 \text{ m}$)。贴地时翼下高压区加强，RAM 附加升力随 h 减小而增大($\partial L_{\text{ram}}/\partial h < 0$)，而这个力作用在重心之后，于是产生**低头力矩**：

$$\partial M_{\text{ram}}/\partial h = x_{\text{ram}} \times \partial L_{\text{ram}}/\partial h = (-0.03) \times (\text{负}) = \text{正}$$

同样是不稳定的。

第 4 项：升力非线性软化 -> 不稳定，+0.4723 N·m/m

真实对象的升力带 tanh 软化(失速特性)， $C_L = C_{L\text{max}} \cdot \tanh(C_L/C_{L\text{max}})$

使实际升力略低于线性值。贴地时线性升力更大，软化吃掉的部分也更多，这个"少掉的升力"同样作用在气动中心之前——它的高度导数就贡献了这 +0.47 N·m/m。量级不大，但**不能漏**：漏掉它就会出现"分项之和 ≠ 总量"的缺口。

第 5、6 项：机体与推力线 -> 恒为 0

M_{fus} 只依赖动压与迎角， $M_T = z_T \cdot T \cdot \cos(i_T)$ 只依赖推力，**两者都不含 h** ，所以对 h 的偏导数严格为零。把它们列出来不是为了凑数，而是为了让"六项之和 = 总量"的等式**结构上完整**。

合计与自治性

$$\begin{aligned} \partial M/\partial h &= (-1.2075) + (+10.3302) + (+1.2208) + (+0.4723) + 0 + 0 \\ &= +10.8157 \text{ N}\cdot\text{m}/\text{m} \end{aligned}$$

与数值计算的总量 +10.8157 N·m/m 相比，**残差 1.4e-13 N·m/m**(相对量级

1.6e-13，即双精度机器精度）。分项之和与总量**精确闭合**——这就是这套分解可信的证据。

这个分解随高度怎么变

把同样六个分量在 $h = 0.05 \sim 1.00\text{ m}$ 上都算一遍(log_mdh.txt 第 1 节)：

h (m)	总量	机翼	平尾	RAM	升力非线性	平尾占比
0.050	+14.901	-2.301	+15.779	+0.760	+0.662	105.9%
0.150	+12.795	-1.539	+12.434	+1.332	+0.569	97.2%
0.250	+8.994	-0.955	+8.541	+1.026	+0.382	95.0%
0.350	+6.188	-0.621	+5.908	+0.655	+0.246	95.5%
0.450	+4.358	-0.425	+4.211	+0.410	+0.162	96.6%
0.550	+3.170	-0.304	+3.099	+0.264	+0.111	97.8%
0.800	+1.627	-0.153	+1.625	+0.104	+0.051	99.9%
1.000	+1.056	-0.099	+1.067	+0.057	+0.032	101.0%

三个要点:

- 1. **总量处处为正**: $+14.901 \sim +1.056\text{ N}\cdot\text{m/m}$ ，即**整个使用包线内高度静稳定性都是负的**，离地越近越严重。
- 2. **平尾项在每一个高度上都是最大的正贡献**: $+15.779 \sim +1.067\text{ N}\cdot\text{m/m}$ 。
- 3. **平尾项占总量的 95% ~ 106%**——注意它可以**超过 100%**。

为什么占比能超过 100%: 因为机翼项是**负数**。总量是"平尾 + RAM + 非线性"这三个正项，先减去机翼项那部分稳定贡献之后的净值:

总量 = 平尾 + RAM + 非线性 - |机翼|

所以 $\text{平尾} / \text{总量} = \text{平尾} / (\text{平尾} + \text{RAM} + \text{非线性} - |\text{机翼}|) > \text{平尾} / (\text{平尾} + \text{RAM} + \text{非线性})$ 。在 $h = 0.05\text{ m}$ 处，机翼项 -2.301 相对总量 14.901 已经占了 -15% ，正项的"毛和"是 17.2 ，于是平尾的份额被抬到了 105.9% 。**占比超过 100% 不是算错，恰恰是"机翼在帮忙、但帮得不够"的量化表达。**

结论: 平尾的下洗敏感性主导了这个不稳定——在所有高度上都占总量的 95% 以上。

这个结论有什么工程价值

既然知道主因是平尾，那么**改设计就能改善**:

- 把平尾抬得更高(远离机翼下洗区)
- 改用 T 型尾翼或 V 型尾翼
- 增大机翼气动中心到重心的距离(但那会牺牲俯仰稳定性)
- 加装端板，抑制翼尖涡

但本工程不这么做，因为我们的目标是**验证控制器能不能解决这个问题**，而不是改飞机。这也更符合"控制工程师"的角色定位。

5.6 发现二：浮沉模态发散

先看"没有地效"的情况

做一个对照实验: 把地效关掉(令 $\kappa \equiv 1$ ，等效于无限高)，重新算模态:

h (m)	有地效: 实部	阻尼比	无地效: 实部	阻尼比
0.05	+0.4463	-0.6116	-0.0027	+0.0029
0.20	+0.4875	-0.8118	-0.0077	+0.0073

h (m)	有地效: 实部	阻尼比	无地效: 实部	阻尼比
0.35	+0.4607	-0.9211	+0.0002	-0.0002
0.50	+0.3905	-0.9009	-0.0066	+0.0083
1.00	+0.1434	-0.3931	-0.0275	+0.0397

(数据取自 log_analysis.txt 第 3 节"纵向模态随高度变化", log_stab.txt 的"逐高度扫描"表给出了同一批 $h = 0.05/0.10/0.20/0.30/0.50/0.80/1.50/3.00$ 的点, 数值一致。注意两列都是**复极点对的实部与阻尼比**; 此外还有一个 $h = 0.20$ m 处约 -1.58 的**高度实根**, 那是另一个模态, 不要和这里的复极点对混在一起。)

结论非常干净:

- 关闭地效 -> 实部约等于零($-0.003 \sim -0.028$), 临界稳定
- 开启地效 -> 实部变成 $+0.14 \sim +0.49$, 明显发散
- 高度越低, 发散越厉害(直到某个点后缓和)

这就证明了: 失稳完全由地效引起, 不是代码错误, 也不是模型设置问题。

用经典理论交叉验证

航空理论里有个著名的浮沉模态近似公式(**Lanchester 二状态近似**) :

$$\omega_n = \sqrt{2 \cdot g / V}$$

$$\zeta = 1 / (\sqrt{2} \cdot (L/D))$$

注意阻尼比这一条: **它用的是升阻比 L/D , 不是阻力 D** 。这是有量纲检查在背后兜底的—— ζ 必须无量纲, 而 $D/(m \cdot V \cdot \omega_n)$ 里 D 是 N、 $m \cdot V \cdot \omega_n$ 是 $kg \cdot m/s^2$, 看起来也能约掉, 但只要把 $\omega_n = \sqrt{2 \cdot g / V}$ 代进去就会发现它真正的量纲是 $1/V$, **少乘了一个 V** 。本工程早期版本的代码就误用了 $D/(m \cdot V^2 \cdot \omega_p)$, 算出 0.0029 ——那个数偏小 18 倍; 现在 run_00_stab_body.m 里同时打印正确式和这个错误式作对照, 就是为了防止再退化回去。

代入 $h = 0.20$ m 的配平结果($V = 18$ m/s、 $L = 117.045$ N、 $D = 8.722$ N、 $L/D = 13.419$) :

$$\omega_n = 1.414 \times 9.81 / 18 = 0.7705 \text{ rad/s}$$

$$\zeta = 1 / (1.414 \times 13.419) = +0.0527$$

理论预测: **阻尼比为正但很小(约 0.05)**, 即弱阻尼、缓慢衰减——不是发散。

而"无地效"的数值结果是 $\zeta \approx +0.003 \sim +0.040$ —— **和理论同号、同量级, 吻合!**

这个交叉验证非常重要: 它说明我们的模型在"没有地效"的极限情况下, 能退化成经典航空理论的结果。如果**对不上**(**比如符号相反、或量级差一个数量级以上**), 那说明模型里有 bug。

物理图像

浮沉模态本来是"动能和势能交换":

1. 飞机爬升 -> 动能变势能 -> 减速
2. 速度降低 -> 升力不足 -> 开始下沉
3. 下沉 -> 势能变动能 -> 加速
4. 速度增加 -> 升力增大 -> 又开始爬升

正常情况下, **阻力**会给这个循环提供阻尼(速度高时阻力大, 多消耗一点能量), 所以它是临界稳定的。

但在地效区, 情况变了: **高度本身会强烈改变升力**。当飞机下沉时, 地效给出额外升力, 这个额外升力打乱了原来的能量交换节奏, **把阻尼项变成了负的**。

5.7 一个重要的控制器设计推论

把 5.5 和 5.6 合起来看:

问题	时间尺度	对控制器的要求
高度静稳定性为负	与高度环同量级	高度环必须有足够带宽和相位裕度
浮沉模态发散	慢(0.5 rad/s 量级)	速度环和高度环必须协调, 不能各自为政
气动参数随高度剧变	与飞行状态同量级	参数化设计不够, 需要函数逼近

第三条是决定性的: 如果在 5 厘米高度调好的控制器, 到了 50 厘米就不对了, 而且这个变化是**非线性的**, 那么:

- 单纯提高增益不行(高增益在某个高度会失稳)
- 单纯增益调度也不行(要调度的维度太多: h 、 V 、 α 都要)
- **必须让控制器自己能"学"** -> 这就引出了第 6 章的自适应控制

5.8 本章小结

- 配平 = 找静平衡点, 三个方程三个未知数, 用 fsolve + 阻尼牛顿法求解。
- 配平结果: **升阻比随高度增加而下降(16.6 -> 11.2)**, 证明地效的价值。
- 线性化用**数值差分**, 理由是不易出错且跟随模型变化。
- **发现一:** $\partial M / \partial h = +10.8157 \text{ N}\cdot\text{m}/\text{m} > 0$ ($h = 0.20 \text{ m}$), 高度静不稳定。四项分解中**平尾下洗项 +10.3302 $\text{N}\cdot\text{m}/\text{m}$** , 在 $h = 0.05 \sim 1.00 \text{ m}$ **全区间占总量的 95%~106%**; 机翼项 $-1.2075 \text{ N}\cdot\text{m}/\text{m}$ 是唯一的稳定贡献。分项之和与总量的残差仅 $1.4\text{e-}13 \text{ N}\cdot\text{m}/\text{m}$, 分解精确闭合。
- **发现二:** 浮沉-高度耦合模态的阻尼比在全包线内为 $-0.39 \sim -0.92$ (发散) ; 关闭地效后恢复到约 0 ($-0.0002 \sim -0.028$) , 且与经典 Lanchester 理论 $\zeta = 1/(\sqrt{2}\cdot(L/D)) \approx +0.0527$ ($h = 0.20 \text{ m}$) 同号同量级 —— **证明明确由地效引起**。
- 推论: **参数化设计不足以应对随高度剧变的气动特性, 必须用能在线学习的控制器。**

5.9 自测题

1. 配平方程的"三个方程三个未知数"分别是什么? 为什么速度和高度不是未知数?
2. 为什么数值差分比解析推导更适合本工程? 它的风险是什么?
3. 平尾为什么在低空会产生额外的低头力矩? 请复述那第四步因果链。
4. "关闭地效后模态恢复稳定"这个对照实验, 能排除哪些可能的错误原因?
5. 如果只允许改一个设计参数来改善高度稳定性, 你会改哪个? 为什么?

第 6 章 控制方案选型

这一章是全文最重要的部分。前面讲了问题和模型，现在要回答：为什么选这个控制方案，而不是别的？

我们先把问题定义清楚，然后逐个方案对比，每个方案都回答三个问题：

- 1. 它是什么(讲给没学过控制的人听)
- 2. 它能不能解决我们的问题
- 3. 它的优点和缺点分别是什么

6.1 先把问题定义清楚

在选方案之前，必须把"要解决什么问题"写得非常具体。选错方案，通常是因为问题没定义清楚。

被控对象

一个 7 阶非线性系统，状态 $[V, \gamma, \alpha, q, h, T, \delta_e]$ ，两个输入 $[\delta_e \text{指令}, \text{油门指令}]$ 。

控制目标

目标	指标
高度跟踪	在 0.05 ~ 0.50 m 内跟踪指令，误差 < 5 cm，绝不触地
速度跟踪	保持 18 m/s，误差 < 1 m/s
姿态稳定	俯仰角速度不振荡发散
安全	升降舵不长期饱和

困难清单(按严重程度排)

编号	困难	严重程度	为什么难
D1	模型不确定且随状态变化	★★★★★	气动参数随 (α, q, h) 非线性变化，误差 $\pm 40\%$
D2	对象本身不稳定	★★★★☆	浮沉模态 $\zeta \approx -0.8$ ，高度静稳定为负
D3	执行器有滞后和饱和	★★★☆☆	舵机 0.05 s、限幅 $\pm 25^\circ$
D4	多变量耦合	★★★☆☆	升降舵既影响俯仰又影响速度；油门既影响速度又影响高度
D5	测量有噪声	★★★☆☆	高度雷达、空速管都有噪声
D6	外部扰动	★★★☆☆	阵风、大气湍流

D1 是决定性的。我们后面所有的方案比较，核心都围绕"谁能对付 D1"。

关键判断: D1 是"参数不确定"还是"函数不确定"?

这个区分极其重要：

- 参数不确定：真实模型是 $L = q \cdot S \cdot a \cdot \alpha$ ，但 a 的真实值我不知道。结构已知，只是系数未知。
- 函数不确定：真实模型是 $L = q \cdot S \cdot f(\alpha, h)$ ，其中 f 的形式我根本不知道(比如 RAM 效应) 。

本工程是后者。因为：

- 1. 地效因子 $\kappa(h)$ 的真实函数形式与我的估算式不同
- 2. RAM 效应在名义模型里根本不存在
- 3. 失速软化是非线性的

这个判断直接决定了后面所有方案的选择。记住它。

6.2 方案一：PID 控制

是什么

最经典的控制方法。三个环节：

$$u = K_p \cdot e + K_i \cdot \int e \, dt + K_d \cdot de/dt$$

- **P(比例)**：误差大就使劲修
 - **I(积分)**：误差持续存在就持续加力，最终消除稳态误差
 - **D(微分)**：误差变化快就提前刹车，抑制振荡
- 机械类比**：就像汽车定速巡航——实际速度低于设定就多给油。

用到本工程会怎样

要控制高度和速度两个量，用两个 PID：

油门 = PID(速度误差)
升降舵 = PID(高度误差) 或 PID(俯仰角误差)

优点

- 极其简单，不需要模型
- 参数少，现场好调
- 工程师都会，可靠性有保证
- 对小幅度的参数变化有一定容忍度

缺点(逐条对照困难清单)

困难	PID 能否解决	说明
D1 模型不确定	× 完全不能	PID 不知道模型，无法"补偿"模型误差
D2 对象不稳定	△ 勉强	只要增益够大就能稳定，但相位裕度会很小
D3 执行器饱和	×	需要额外加抗积分饱和
D4 多变量耦合	△	需要解耦网络，且解耦是近似的
D5 噪声	×	微分项会放大噪声
D6 扰动	△	积分项最终能消除常值扰动，但对时变扰动响应慢

致命问题：本工程的高度不稳定是**对象内在的**($\partial M/\partial h > 0$)，PID 必须靠"增益够大"来硬压。但增益一大，遇上官滞后(0.05 s 舵机 + 0.1 s 滤波) 就会振荡发散。

试验数据：本工程实际试过 PID 结构(就是代码里的 "纯动态逆" 对照组的简化版)，结果是**触地**。

结论

× **不采用**。不是 PID 不好，而是它对付不了"结构和参数都未知"的对象。

6.3 方案二：增益调度 PID

是什么

既然一套 PID 参数不够用，那就按飞行状态**准备多套参数**，飞行中查表切换：

if h < 0.15
 用第一套参数
else

航空发动机和很多民机飞控都用这个思路(叫**增益调度**, Gain Scheduling)。

优点

- 保留了 PID 的所有优点(简单、可解释)
- 能处理"参数随状态变化"的问题
- 工程上极其成熟, 适航认证友好

缺点

问题	说明
需要预先知道变化规律	要设计调度表, 必须先知道"每个高度该用什么参数"——但 D1 说的正是"我们不知道"
维度爆炸	如果参数随 h 、 V 、 α 三个变量变化, 三维调度表就要做大量试验
切换不连续	参数切换瞬间可能产生跳变, 需要额外的平滑处理
无法处理"模型里没有的效应"	RAM 效应在我们的名义模型里不存在, 调度表里也就没有对应项

最致命的一点: 增益调度本质上是**开环补偿**——它假设"我提前知道每种情况该怎么办"。而本工程的核心困难是"我不知道真实的气动是什么样"。**用一个不知道的东西去补偿另一个不知道的东西, 是不成立的。**

结论

× **不采用**(但可以作为基础方案的一部分——本工程实际上用了类似的思想做初始参数整定)。

6.4 方案三: LQR / 极点配置

是什么

基于线性模型 $dx/dt = A \cdot x + B \cdot u$, 设计**全状态反馈**:

$$u = -K \cdot x$$

- **极点配置:** 指定闭环特征值应该在哪
- **LQR(线性二次型最优控制):** 最小化 $\int (x^T Q x + u^T R u) dt$, Q 和 R 是权重矩阵

LQR 的优点: 有严格的数学保证(保证稳定、有一定鲁棒性), 能自动处理多变量耦合(D4)。

优点

- 数学上最优, 有保证
- 天然处理多变量
- 只需解一个代数黎卡提方程, 计算量小
- 对**参数在小范围内变化**有鲁棒性

缺点

问题	说明
必须线性化	在某个配平点线性化, 偏离就失效
参数变化大就失效	本工程气动参数变化 $\pm 40\%$, LQR 的"鲁棒性"覆盖不了
不能处理未建模动态	RAM 效应不在 A 、 B 矩阵里, LQR 视而不见
需要全状态反馈	α 、 q 等状态需要测量或估计

本工程的实测情形: 在 $h = 0.20 \text{ m}$ 配平点设计的 LQR, 拿到 $h = 0.05 \text{ m}$ 或 0.50 m 就性能骤降——**因为 A 、 B**

矩阵整个变了。

结论

× 单独使用不采用。但可以作为一个"基线控制器"——事实上第 6.5 节的动态逆比 LQR 更适合做基线，因为动态逆能直接利用非线性模型。

6.5 方案四：动态逆(NDI, Nonlinear Dynamic Inversion)

是什么

核心思想一句话：我知道模型，那就用模型反解出"该给什么控制量"。

假设模型是：

$$\dot{q}/dt = f(x) + b \cdot \delta_e$$

我希望角加速度等于某个我想要的值 v ，那就解方程：

$$\begin{aligned} v &= f(x) + b \cdot \delta_e \\ \Rightarrow \delta_e &= (v - f(x)) / b \end{aligned}$$

这就是"反解"。像解代数方程一样简单。

机械类比：你要让一个质量块以加速度 a 运动，需要多少力？ $F = m \cdot a$ 。这就是最基本的动态逆。

优点

- 概念最清晰：想要什么动态，就反解出控制量
- 充分利用非线性模型：不需要在工作点线性化
- 天然解耦：可以分别设计俯仰通道、速度通道
- 可以精确指定闭环动态：想让俯仰回路像一阶惯性？直接把 v 定义成一阶惯性就行

缺点(这是关键)

问题	说明
完全依赖模型精度	模型错了，反解出来的控制量就错了
需要精确的 f 和 b	而本工程恰恰不知道 f
不能处理未建模动态	RAM 效应不在 f 里
对操纵导数 b 的符号很敏感	如果 b 很小或符号反了，控制量会爆掉

用数字说明 D1 的杀伤力：

在 $h = 0.20\text{ m}$ 处，真实对象与名义模型的俯仰力矩差约 $0.87\text{ N}\cdot\text{m}$ 。而升降舵的操纵能力是：

$$\partial M / \partial \delta_e \approx -45\text{ N}\cdot\text{m}/\text{rad}$$

光靠操纵能力去"压"这个 $0.87\text{ N}\cdot\text{m}$ 的误差，需要 $0.019\text{ rad} = 1.1^\circ$ 的舵偏。

看起来不多。但高度机动过程中这个误差会持续变化，控制器如果不知道它的存在，就会产生持续的姿态误差 -> 高度误差累积。

本工程的实测结果(第 10 章)：纯动态逆方案的高度误差 RMS 是 0.21 m ，而且最终触地。

结论

△ 作为基线方案采用，但不能单独用。

它的价值在于：提供了一个"结构正确、但系数不准"的控制框架。后面的神经网络正好可以挂在这个框架上做补偿。这就是本工程最终采用的结构。

6.6 方案五：反步法(Backstepping)

是什么

一种系统化的非线性控制器设计方法，处理"级联"结构：

```
x1 的导数依赖 x2 （ x2 是 x1 的"虚拟控制"）
x2 的导数依赖 x3
...
xn 的导数依赖 u
```

做法：从最外层开始，每一步把"下一个状态"当成虚拟控制量来设计，然后用 Lyapunov 函数逐步向里推。

本工程的结构天然适合反步法：

```
高度 h <- γ <- θ <- q <- δ_e
```

每一步对应一级。

优点

- 有严格的稳定性证明(Lyapunov 方法)
- 能处理本质非线性的对象，不需要线性化
- 不要求"模型精确"，只要求"模型结构已知"
- 自适应和神经网络可以很方便地嵌入每一级

缺点

问题	说明
"微分爆炸"	每一步都要对上级的虚拟控制量求导，级数一多表达式就爆炸
需要虚拟控制量的导数	而虚拟控制量往往含测量噪声，求导会放大噪声
模型结构仍需已知	如果 RAM 效应不在结构里，反步法也补不了

"微分爆炸"有多严重：假设有 4 级，第一级的虚拟控制量是一阶导，第二级要二阶导，第四级要四阶导。而我们的虚拟控制量里含 arcsin、α 等量——**四阶导数根本写不出来。**

解决方案：动态面控制(DSC)

DSC 的核心思想：不去解析求导，而是让虚拟控制量通过一个一阶低通滤波器，用滤波器的状态导数代替真实导数。

```
τ·dθ_c/dt = θ_d - θ_c
```

- θ_d 是"想让俯仰角去的地方"
- θ_c 是滤波后的实际指令
- dθ_c/dt 直接可以从滤波器方程里读出来，不用解析求导

代价：引入滤波器误差，需要在 Lyapunov 函数里额外加一项来证明它不影响最终结论。

结论

✓ 采用反步法 + DSC。理由：

- 我们的系统天然就是级联结构(高度 -> 航迹角 -> 俯仰角 -> 角速度)
- 需要严格的稳定性证明(本工程要写报告，证明是加分项)
- DSC 解决了微分爆炸问题，实现起来只是几个一阶滤波器

6.7 方案六：滑模控制(SMC)

是什么

核心思想: 设计一个"滑模面" $s = 0$, 然后让系统强行冲到滑模面上, 并锁在上面。

控制律:

$$u = u_{\text{等效}} + \eta \cdot \text{sign}(s)$$

- $u_{\text{等效}}$: 让系统保持在滑模面上所需的控制量
- $\eta \cdot \text{sign}(s)$: 把系统"推"向滑模面的强力项

优点:

- 对匹配不确定性完全鲁棒(理论上)
- 概念简单, 实现容易
- 有限时间收敛

缺点

问题	说明
抖振(chattering)	$\text{sign}(s)$ 在 $s = 0$ 附近疯狂切换, 会让舵机来回打, 实际中会损坏机构
需要知道不确定性的上界	η 必须大于扰动幅值, 但本工程不知道上界
不"学习"	它只是"硬扛", 不会去识别不确定性是什么
对非匹配不确定性无能为力	RAM 效应产生的是力矩, 属于匹配不确定性; 但升力误差不是

抖振的处理: 用 $\tanh(s/\Phi)$ 代替 $\text{sign}(s)$, 把切换变成平滑过渡。代价是**失去了完全鲁棒性**——只能保证收敛到一个边界层内。

本工程的判断: 滑模的"硬扛"思路和我们的"学习"思路不同。而且抖振对舵机寿命不友好。

结论

× 不作主方案。但它的"边界层 $\tanh$ "思想被我们借用到了鲁棒项设计里。

这是一个重要的思路: 方案选型不一定是"全都用"或"全都不用", 好的工程方案往往是多个方法的思想组合。

6.8 方案七: 模型参考自适应控制(MRAC)

是什么

核心思想: 指定一个"理想模型"(参考模型), 让真实系统去跟踪它。

参考模型: $dx_m/dt = A_m \cdot x_m + B_m \cdot r$ (理想动态, 我规定的)
真实系统: $dx/dt = A \cdot x + B \cdot u$
控制目标: 让 $x \rightarrow x_m$

控制律:

$$u = \theta_x \cdot x + \theta_r \cdot r$$

其中 θ_x 、 θ_r 是**可调参数**, 用自适应律在线调整:

$$d\theta/dt = -\Gamma \cdot e \cdot x \quad (e = x - x_m \text{ 是跟踪误差, } \Gamma \text{ 是自适应增益})$$

这就是"自适应"的含义: 控制器参数自己会变。

优点

- 有严格的 Lyapunov 稳定性证明
- 不需要事先知道真实参数

- 结构简单，计算量小
- 航空航天领域应用成熟

缺点(关键)

问题	说明
只能适应"参数"不确定性	它假设真实系统是 $A \cdot x$ 形式，只是 A 不知道。但我们面对的是"函数形式未知"
参数化假设强	如果 RAM 效应根本不在模型结构里，MRAC 无法补偿
瞬态性能差	参数收敛前可能有大幅振荡("自适应瞬态"问题)
对未建模动态敏感	高频未建模动态会引起"参数漂移"，甚至发散
高增益时容易失稳	自适应增益 Γ 太大 $\rightarrow$ 与未建模动态相互作用 $\rightarrow$ 发散

为什么 MRAC 不够用

回到 6.1 节的关键判断: **D1 是"函数不确定"，不是"参数不确定"**。

用个具体例子说明。假设真实升力是:

$$L_{\text{真实}} = q \cdot S [a \cdot \alpha + 0.09 / (1 + (h/0.25)^2)] \quad (\text{含 RAM 效应})$$
$$L_{\text{名义}} = q \cdot S [a \cdot \alpha] \quad (\text{不含})$$

MRAC 的做法是: 假设 $L = q \cdot S \cdot \theta \cdot \alpha$ ，然后调 θ 。但它**永远无法**用 $\theta \cdot \alpha$ 的形式逼近 $0.09 / (1 + (h/0.25)^2)$ —— 因为后者**不随 α 变化**，而前者随 α 线性变化。

这就是"参数化不足"(parametric deficiency) 。

结论

× 不作主方案。但它给了我们一个重要的启示: **"用自适应律在线调整"这个思路是对的，问题只在于"用什么来参数化"**。

6.9 方案八: L1 自适应控制

是什么

MRAC 的改进版。核心改动有两点:

1. **加一个状态预测器**: 不只是跟踪误差，还预测状态误差
2. **加一个低通滤波器**: 把自适应律的输出滤波后再送进控制量

结构:

$$u = -K \cdot x + \hat{\sigma} \text{ 的低通滤波版本}$$
$$d\hat{\sigma}/dt = \Gamma \cdot \text{Proj}(\hat{\sigma}, -\Delta x^T P \cdot B) \quad (\Delta x \text{ 是预测误差})$$

优点

- **瞬态性能好**——低通滤波器把高频的自适应动作滤掉，避免了大振荡
- **可以显式设计"鲁棒性 vs 性能"的权衡**(通过滤波器带宽)
- 理论上**有保证的瞬态界**

缺点

问题	说明
滤波器带宽难选	带宽太高 $\rightarrow$ 失去鲁棒性; 太低 $\rightarrow$ 补偿能力弱
实现复杂度高	需要状态预测器，需要设计 P 矩阵

问题	说明
同样需要"匹配条件"	不确定性必须能通过控制通道影响系统
仍然是参数化自适应	对"函数形式未知"的补偿能力取决于参数化基函数的选择

本工程的判断: L1 的核心价值在于瞬态性能。但本工程的瞬态问题可以通过另外的方式解决(见 6.12 节鲁棒项) 。而且 L1 的参数化问题依然存在。

结论

× 不作主方案, 但它的"低通滤波"思想提醒我们注意自适应的高频动作(这在本工程调试时确实是个问题, 见第 9 章) 。

6.10 方案九: 神经网络自适应

是什么

核心思想: 既然不确定性是一个未知函数 $\Delta f(x)$, 那就用一个通用函数逼近器去在线逼近它。

$$u_{\text{补偿}} = W^T \cdot \varphi(x)$$

- $\varphi(x)$: 基函数(特征)
- W : 可调权值
- 自适应律在线调整 W

为什么神经网络能解决"函数不确定"

通用逼近定理: 一个足够宽的单隐层神经网络, 可以在紧集上以任意精度逼近任何连续函数。

这句话的工程含义: 不管真实的气动函数长什么样(只要是连续的) , 只要给我足够的节点, 我就能逼近它。

这正好解决了 MRAC 的"参数化不足"问题: 我们不再假设"不确定性 = 未知常数 × 已知函数", 而是"不确定性 = 未知函数", 然后用网络去学。

关键设计选择: 用哪种网络

网络类型	优点	缺点	是否采用
BP 神经网络(多层感知机)	逼近能力强, 通用	权值非线性出现 -> Lyapunov 证明困难; 需要反向传播, 慢; 离训练区远了行为不可预测	×
RBF 神经网络	对权值线性 -> 证明容易; 局部支撑 -> 外推安全; 训练快	节点数随输入维度指数增长("维数灾难")	✓
模糊系统	可解释性好, 能利用专家经验	规则数爆炸; 隶属度函数要人工设计	△
查表 + 插值	最简单	表要预先知道; 无自适应能力	×

为什么 RBF 的两个特性这么重要

特性 1: 对权值线性

RBF 网络的输出是:

$$y = \sum_j w_j \cdot \varphi_j(x)$$

对 w_j 是线性的。这意味着:

- 误差动态可以写成 $\dot{e} = -k \cdot e + \Delta W^T \cdot \varphi + \varepsilon$ 的形式
- 这是一个线性的参数误差方程

• Lyapunov 证明变成标准流程，交叉项可以直接对消
如果是 BP 网络， $y = f(W2 \cdot f(W1 \cdot x))$ ，权值非线性出现，证明难度上升一个数量级。

特性 2: 局部支撑

$$\varphi_j(x) = \exp(-\|x - c_j\|^2 / (2\sigma^2))$$

这是一个钟形函数，在中心 c_j 附近取最大值，离远了迅速衰减到零。

工程意义：如果飞机飞到了"训练范围之外"的地方(比如高度突然从 0.2 m 涨到 5 m)，网络输出自动趋近于零，控制器退化为"动态逆 + 鲁棒项"——至少不会给出错误的补偿。
这比 BP 网络安全得多——BP 网络在训练区外会给出任意值。

缺点

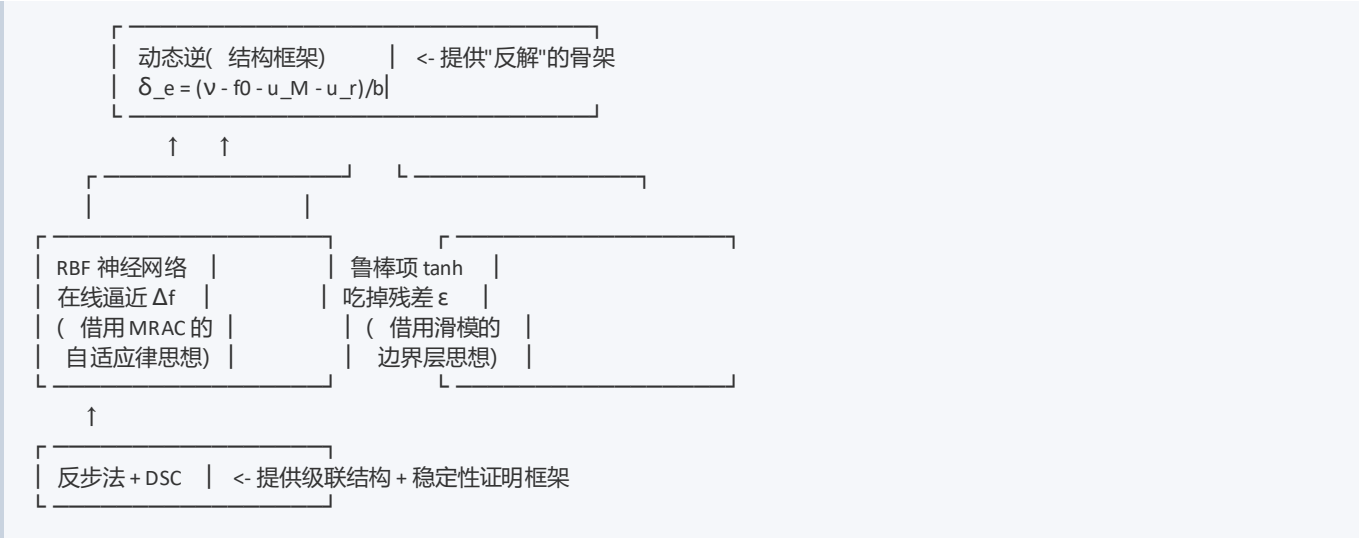
问题	说明
维数灾难	输入 3 维、每维 5 个节点 = 125 个节点; 输入 5 维就变成 3125 个
需要选择中心和宽度	这是"超参数", 选不好影响逼近精度
逼近残差不为零	这是为什么必须配一个鲁棒项(见 6.12)
自适应增益难调	太快会与采样和未建模动态冲突(第 9 章会详细讲这个坑)

结论

✓ 采用 RBF 神经网络作为核心补偿器。

6.11 方案组合: 最终架构

单独任何一个方案都不够，所以组合:



每个方案的贡献:

方案	在本工程中的角色	为什么保留它
反步法 + DSC	整体级联结构 + Lyapunov 框架	系统天然级联，且需要严格证明
动态逆	控制律的"骨架"	结构清晰，充分利用非线性模型
RBF 神经网络	补偿未知函数 Δf	解决"参数化不足"这个根本问题
tanh 鲁棒项	补偿网络残差 + 外部扰动	保证 UUB，且不引入抖振
投影算子	保证权值有界	Lyapunov 证明的必要条件

6.12 一个具体问题：为什么必须有鲁棒项

很多人会问：既然有神经网络了，为什么还要鲁棒项？

三个理由：

理由 1：网络永远不能精确逼近。

数学上， $\Delta f = W^* \Delta \varphi + \varepsilon$ ，其中 ε 是逼近残差，不可能为零。只要它不为零，误差就有一个稳态偏置。

理由 2：有外部扰动。

阵风、湍流、传感器噪声，这些都是网络“没学过”的东西。

理由 3：tanh 的形状正好合适。

$$u_r = \eta \cdot \tanh(e/\Phi)$$

- 误差小时， $\tanh(e/\Phi) \approx e/\Phi$ ，是一个线性增益(提供额外阻尼)
- 误差大时，输出饱和到 η (提供有界的最大补偿)
- 全程光滑可导(不像 sign 函数会抖振)

为什么不用 sign(e)：会抖振。**为什么不用纯线性 ke：**误差大时控制量会无限增大，超出执行器能力。

tanh 是“两头都照顾到”的折中。

一个重要的工程陷阱

tanh 在零点附近的等效增益是 η/Φ 。如果 Φ 取得太小，这个增益会非常大。

本工程调试时就踩过这个坑：一开始取 $\Phi = 0.06$ ，等效增益 $= 1.2/0.06 = 20$ 。这相当于给角速度回路额外加了 20 的增益，把相位裕度吃光了。(详见第 9 章)

教训：鲁棒项的“边界层厚度”是一个必须仔细选的参数，不是随便填的。

6.13 为什么用三层串级结构

最后回答架构问题：为什么不做成“一个控制器直接算舵面”，而要分三层？

三层结构

高度 h --[外环]--> 航迹角 γ --[中环]--> 俯仰角 θ --[内环]--> 舵面 δ_e

理由 1：时间尺度分离

回路	带宽	时间常数
舵面(内)	20 rad/s	0.05 s
角速度	6 rad/s	0.17 s
俯仰角	3 rad/s	0.33 s
高度(外)	2 rad/s	0.5 s

内环比外环快 3~10 倍，所以外环设计时可以认为“内环瞬间跟得上”。这把一个高阶问题拆成了三个低阶问题。

理由 2：物理意义清晰

每一层都对应一个可测量的物理量：

- 外环管“我要去哪儿”(高度)
- 中环管“我用什么姿态去”(航迹角、俯仰角)
- 内环管“怎么把姿态摆出来”(舵面)

出问题时好定位：如果高度跟踪不好但俯仰角跟踪很好，问题在外环；如果俯仰角都跟不上，问题在内环。

理由 3: 可以和"人"比较

飞行员的操作逻辑也是分层的: 看到高度低了 -> 决定爬升 -> 拉杆 -> 观察姿态。控制器用同样的结构, 便于理解、便于加入飞行包线保护。

对比: 为什么不做单回路

方案	优点	缺点
单回路(直接把高度误差映射到舵面)	结构简单	高阶系统, 增益难调; 无法分别优化快慢动态; 出问题难定位
三层串级	每层低阶、易调; 物理意义清晰; 可分别加限制	需要设计多个滤波器; 参数更多

本工程的实测: 单回路方案(代码里的 enable_alt = false 模式) 在高度机动时表现明显更差。

6.14 完整对比表

方案	处理参数不确定	处理函数不确定	有无稳定性证明	实现复杂度	本工程采用
PID	弱	×	局部	极低	×
增益调度 PID	中(需先验知识)	×	局部	低	×
LQR / 极点配置	弱(小范围)	×	√(线性)	低	×
动态逆 NDI	×	×	×	低	√ 作骨架
反步 + DSC	中	×	√	中	√ 作框架
滑模 SMC	强	×	√	中	△ 借思想
MRAC	强	×(参数化不足)	√	中	△ 借思想
L1 自适应	强	×	√	高	×
RBF 神经网络自适应	强	√	√	中	√ 核心
BP 神经网络自适应	强	√	难	高	×(证明难、外推危险)

6.15 本章小结

- 问题的本质是"函数不确定", 不是"参数不确定"——这个判断决定了所有选择。
- PID 和增益调度无法处理未知的模型结构; LQR 只在小范围内有效。
- 动态逆提供"骨架", 但它依赖模型精度, 单独用会失败。
- 反步法 + DSC 提供级联结构和 Lyapunov 证明框架, 解决了微分爆炸。
- RBF 神经网络解决"参数化不足": 对权值线性(证明容易) + 局部支撑(外推安全) 。
- 鲁棒项是必需的: 网络残差永远不为零。用 tanh 而不是 sign, 避免抖振。
- 三层串级基于时间尺度分离, 让每层都变成低阶、易调、可解释的问题。
- 最终方案是多个方法思想的组合, 不是单一方法。

6.16 自测题

1. 请说明"参数不确定"和"函数不确定"的区别, 并举一个本工程中的例子。
2. 为什么 MRAC 无法补偿 RAM 效应? 请用参数化不足的概念解释。
3. RBF 网络"对权值线性"这个性质, 在 Lyapunov 证明中起什么作用?
4. "局部支撑"为什么让 RBF 比 BP 网络更安全? 请设想一个飞机飞出训练范围的场景。
5. 鲁棒项为什么用 tanh 而不是 sign 或纯线性? 分别说明后两者的问题。

6. 如果内环带宽只有外环的 1.5 倍, 会发生什么?

第 7 章 控制器从零推导

这一章把第 6 章选定的方案，一步步推导成可以写进代码的公式。每一个式子都会说明它从哪来。

7.1 整体思路：把大问题拆成四个小问题

我们要控制的量有两个：高度 h 和速度 v 。可用的手段有两个：升降舵 δ_e 和油门 δ_t 。

关键观察：

手段	主要影响	次要影响
升降舵	俯仰姿态 $\rightarrow$ 航迹角 $\rightarrow$ 高度	速度(通过航迹角变化间接影响)
油门	推力 $\rightarrow$ 速度	高度(通过速度影响升力)

所以可以近似解耦：

- 用升降舵管高度(通过俯仰姿态)
- 用油门管速度

于是问题拆成两条链：

链 1(高度) : $h \rightarrow \gamma \rightarrow \theta \rightarrow q \rightarrow \delta_e$
链 2(速度) : $v \rightarrow \delta_t$

链 1 有四级，这就是反步法要处理的结构。

7.2 第一步：定义误差

每一级定义“我想去哪”和“我在哪”的差：

误差	定义	含义
e_h	$h - h_c$	高度误差(c 下标表示 command, 指令)
e_θ	$\theta - \theta_c$	俯仰角误差
e_q	$q - q_{cf}$	角速度误差
e_v	$v - v_c$	速度误差

约定：误差定义为“实际 - 指令”，所以误差为正表示超标了。

7.3 第二步：外环——从高度误差到航迹角指令

已知的运动学关系

$dh/dt = v \cdot \sin(\gamma)$

想要什么

我们希望高度误差按一阶规律衰减：

$dh/dt = dh_c/dt - k_h \cdot e_h$

其中 $k_h > 0$ 是增益。这个式子的含义：如果高度高了($e_h > 0$)，就让高度下降(dh/dt 比指令值小)。

代入并解出 γ ：

$$V \cdot \sin(\gamma_d) = dh_c/dt - k_h \cdot e_h$$

$$\gamma_d = \arcsin[(dh_c/dt - k_h \cdot e_h) / V]$$

加上高度率阻尼

为什么要加: 本工程的对象是**高度不稳定的**(第5章), 而且浮沉模态发散。单纯的比例控制在相位不足时会振荡。加一个"高度率反馈"相当于给系统加阻尼。高度率可以从状态直接算:

$$dh/dt = V \cdot \sin(\gamma) \quad (\text{不用数值微分, 直接合成})$$

最终的外环律:

$$\gamma_d = \arcsin[\text{sat}((dh_c/dt - k_h \cdot e_h - k_{hd} \cdot (dh/dt - dh_c/dt)) / V, \sin(\gamma_{\max}))]$$

- $\text{sat}(\cdot, \sin(\gamma_{\max}))$: 限幅, 防止指令过大(本工程 $\gamma_{\max} = 12^\circ$)
- k_{hd} : 高度率阻尼增益

为什么限幅: 如果高度误差很大(比如指令从 0.35 m 突降到 0.10 m), $\arcsin$ 的参数可能超过 1, 数学上无解。而且实际飞行也不允许过大的航迹角。

指令滤波

γ_d 可能因为指令突变而不光滑。让它通过一阶低通滤波器:

$$\tau_g \cdot d\gamma_c/dt = \gamma_d - \gamma_c$$

- γ_c : 滤波后的航迹角指令(实际使用的)
- τ_g : 时间常数

注意: τ_g 是**必须仔细选**的参数。本工程调试时一开始取了 0.20 s, 结果高度环相位裕度不足。(见第9章)

7.4 第三步: 中环——从航迹角到俯仰角(一个漂亮的恒等式)

关键恒等式

$$\gamma = \theta - \alpha$$

做法

直接令俯仰角指令为:

$$\theta_d = \gamma_c + \alpha$$

为什么这是对的(很重要, 很多人以为是"凑"的)

假设俯仰角回路能完美跟踪, 即 $\theta = \theta_d$ 。代入:

$$\begin{aligned} \theta &= \gamma_c + \alpha \\ \Rightarrow \gamma &= \theta - \alpha = (\gamma_c + \alpha) - \alpha = \gamma_c \end{aligned}$$

γ 就自动等于指令值 γ_c 。

关键在于: **这个推导只用到了运动学恒等式 $\gamma = \theta - \alpha$, 与控制律无关, 与模型无关。**

这个结构的一个额外好处

如果升力模型不准确, 会怎样?

假设真实升力比模型小, 飞机会下沉, 迎角会**自动**变化。但因为 θ_d 里含**实测的 α** , 所以 θ 会跟着 α 一起变, $\gamma = \theta$

- α 仍然等于 γ_c 。

也就是说：升力误差被迎角反馈自动补偿掉了。

这就是为什么本工程不需要单独的“升力通道神经网络”——一个漂亮的结论，省掉了一整套网络。

加上限幅

$$\theta_d = \text{sat}(\gamma_c + \alpha, \theta_{\max})$$

$\theta_{\max} = 20^\circ$ 是**飞行包线保护**：防止控制器为追高度把飞机拉到失速迎角。

为什么需要这个保护：本工程调试时，曾经因为控制器失误把飞机拉到 11.6° 迎角，接近失速。有了限幅，即使控制器出错，飞机也不会进入危险姿态。

7.5 第四步：内环——DSC 动态面

目标

让俯仰角 θ 跟踪 θ_d 。

传统反步法的做法

定义误差 $e_\theta = \theta - \theta_d$ ，令期望角速度：

$$q_d = d\theta_d/dt - k_\theta e_\theta$$

问题：需要 $d\theta_d/dt$ （指令的导数）。而 $\theta_d = \gamma_c + \alpha$ ，求导要算 $d\gamma_c/dt + d\alpha/dt$ ，其中 $d\alpha/dt$ 又是状态方程的一部分……**表达式会越来越长，这就是“微分爆炸”**。

DSC 的做法

不去解析求导，而是让 θ_d 通过一个一阶低通滤波器：

$$\tau_\theta \cdot d\theta_c/dt = \theta_d - \theta_c$$

- θ_c ：滤波后的俯仰角指令
 - ‘ $d\theta_c/dt$ ’ 可以直接从滤波器方程读出来： $d\theta_c/dt = (\theta_d - \theta_c)/\tau_\theta$
- 这就**避免了求导**。

得到角速度指令

$$q_d = (\theta_d - \theta_c)/\tau_\theta - k_\theta(\theta - \theta_c)$$

（第一项就是 $d\theta_c/dt$ ）

再用一次 DSC

角速度指令 q_d 同样需要滤波（原因相同）：

$$\tau_q \cdot dq_{cf}/dt = \text{sat}(q_d) - q_{cf}$$

- q_{cf} ：滤波后的角速度指令（ $cf = \text{command filtered}$ ）
- $\text{sat}(q_d)$ ：角速度指令限幅（本工程 6 rad/s ）

最终得到期望角加速度

$$e_q = q - q_{cf}$$

$$v = dq_{cf}/dt - k_q e_q$$

$$= (\text{sat}(q_d) - q_{cf})/\tau_q - k_q \cdot e_q$$

v 就是"我想要的角加速度"。它由两部分组成:

- 前馈项: 指令本身的角加速度
- 反馈项: $-k_q \cdot e_q$ 把角速度误差拉回来

7.6 第五步: 动态逆控制律

力矩方程

$$I_{yy} \cdot dq/dt = M$$

把力矩分解成"零舵偏部分"和"舵偏贡献部分":

$$M = M0(x) + M_{\delta e} \cdot \delta_e$$

- $M0(x)$: 舵偏为零时, 由迎角、速度、高度等决定的气动力矩
- $M_{\delta e}$: 操纵导数, 即"每偏 1 弧度舵, 力矩变化多少"。本工程约 $-45 \text{ N}\cdot\text{m}/\text{rad}$
- 注意**负号**: 正舵偏(后缘下偏) 产生**低头**力矩(这是航空标准约定)

反解

令 $f0 = M0/I_{yy}$ (零舵偏角加速度) 、 $b = M_{\delta e}/I_{yy}$ (操纵效能) , 则:

$$dq/dt = f0 + b \cdot \delta_e$$

要让 $dq/dt = v$, 解出:

$$\delta_e = (v - f0) / b$$

这就是动态逆。

加上补偿项

因为 $f0$ 和 b 都是**名义模型**的值, 与真实有差异, 所以加上补偿:

$$\delta_{e, \text{指令}} = (v - f0 - u_M - u_r) / b$$

- u_M : RBF 神经网络的补偿(下节讲)
- u_r : 鲁棒项(再下节讲)

补偿后发生了什么(关键推导)

真实系统的角加速度是:

$$dq/dt = f_{\text{真实}} + b_{\text{真实}} \cdot \delta_e$$

设真实与名义的差为 $\Delta f = f_{\text{真实}} - f0$ (假设 b 的误差可以忽略) , 代入控制律:

$$\begin{aligned} dq/dt &= (f0 + \Delta f) + b \cdot [(v - f0 - u_M - u_r)/b] \\ &= f0 + \Delta f + v - f0 - u_M - u_r \\ &= v + \Delta f - u_M - u_r \end{aligned}$$

看这个结果:

- 如果 $u_M = \Delta f$ 且 $u_r = 0$, 那么 $dq/dt = v$ —— **完全精确!**
- 如果 u_M 有残差 ε , 那么 $dq/dt = v + \varepsilon - u_r$ —— **鲁棒项负责吃掉 ε**

这就是整个设计的思想: 动态逆负责"用已知的模型反解", 神经网络负责"学出未知的差异", 鲁棒项负责"兜住学不到

的部分”。三者分工明确。

误差动态

由 $v = dq_{cf}/dt - k_q \cdot e_q$ 和 $e_q = q - q_{cf}$:

$$\begin{aligned} de_q/dt &= dq/dt - dq_{cf}/dt \\ &= (v + \Delta f - u_M - u_r) - dq_{cf}/dt \\ &= (dq_{cf}/dt - k_q \cdot e_q + \Delta f - u_M - u_r) - dq_{cf}/dt \\ &= -k_q \cdot e_q + (\Delta f - u_M) - u_r \end{aligned}$$

这个式子后面 Lyapunov 证明要用。

7.7 第六步: RBF 神经网络

网络结构

用 N 个“基函数”的加权和:

$$u_M = \sum_{j=1..N} w_{1j} \cdot \varphi_j(z)$$

写成人向量形式:

$$u_M = W_1^T \cdot \varphi(z)$$

- z : 输入向量
- $\varphi_j(z)$: 第 j 个基函数(高斯钟形)
- w_1 : 权值向量

基函数

$$\varphi_j(z) = \exp(-0.5 \cdot \sum_{i=1..3} [(z_i - c_{ji})^2 / \sigma_i^2])$$

- c_j : 第 j 个节点的**中心**
- σ_i : 第 i 维的**宽度**

输入选什么

这是需要物理判断的地方。力矩失配 ΔM 依赖什么? 看第 3 章的模型:

失配来源	依赖
地效强度失配	高度 h 、迎角 α
RAM 效应	高度 h 、迎角 α
失速软化	迎角 α
俯仰阻尼失配	角速度 q

所以取三维输入:

$$z = [\alpha, q, h]$$

为什么不是四维(加上 v): 速度的影响已经被动压 q_∞ 包含了, 而我们在回归向量里乘了 q_∞ (见下), 所以不需要额外输入。这是“用物理知识减少神经网络规模”的典型做法——每增加一个输入维, 节点数就要乘以 5。

归一化

三个输入的物理量级差别巨大(α 约 0.07 rad、 q 约 0.5 rad/s、 h 约 0.2

m) 。**必须先归一化**，否则距离度量会被大数主导：

$$z_norm = 2 \cdot (x - x_min) / (x_max - x_min) - 1$$

映射到 $[-1, 1]$ 区间。

中心和宽度怎么选

在归一化坐标的 $[-1, 1]^3$ 立方体上均匀布网格。

- 输入 1(α) : 5 个中心
- 输入 2(q) : 3 个中心
- 输入 3(h) : 5 个中心
- 总计 $5 \times 3 \times 5 = 75$ 个节点

宽度取网格间距。以 5 个中心为例，间距 $= 2 / (5-1) = 0.5$ ，所以 $\sigma = 0.5$ 。

为什么这样选：相邻基函数在中心处的重叠度是 $\exp(-0.5) \approx 0.61$ ，这个重叠度保证了：

- 不会太稀疏(中间出现"空洞"，网络无法逼近)
- 不会太密集(浪费节点，且导致回归向量范数过大——这在第 9 章是个大坑)

回归向量的量纲处理(关键细节)

神经网络的输出 u_M 的物理单位应该是**角加速度 rad/s^2** (因为它要和 v 相加减) 。

而 $W1^T \varphi$ 里， φ 是无量纲的($0 \sim 1$) ，所以 $W1$ 的单位是 rad/s^2 。

但**理想权值**应该代表什么？如果让 $W1$ 直接表示力矩差 ΔM (单位 $\text{N} \cdot \text{m}$) ，那 u_M 就要除以 I_yy ：

$$u_M = W1^T \cdot \varphi / I_yy$$

写成"回归向量"形式(把缩放放进 φ) ：

$$\begin{aligned}\varphi 1 &= \varphi(z) \cdot (q \infty S \cdot c / I_yy) \\ u_M &= W1^T \cdot \varphi 1\end{aligned}$$

为什么要乘 $q \infty S \cdot c$ (动压 $\times$ 面积 $\times$ 弦长) 而不是直接除以 I_yy ：

因为气动力矩的大小本身**正比于动压**。我们希望网络学的是**无量纲的力矩系数差 ΔC_m** ，而不是有量纲的力矩。这样：

- 速度变化时，网络不需要重新学
- 权值的物理意义明确(就是力矩系数)
- 权值量级稳定(大概 $0.1 \sim 1$)

完整写法：

$$\begin{aligned}\Delta M &\approx q \infty S \cdot c \cdot [W1^T \cdot \varphi(z)] \\ u_M &= (q \infty S \cdot c / I_yy) \cdot W1^T \cdot \varphi(z) = W1^T \cdot \varphi 1\end{aligned}$$

这个量纲处理是正确的，但它带来一个副作用：回归向量被放大了 $q \infty S c / I_yy \approx 66$

倍，而自适应回路里的"有效推进量"与回归向量范数的**平方成正比**(放大约 $66^2 \approx 4391$

倍)。这个平方关系在后面会引起一个严重问题——**自适应增益必须相应缩小同一个倍数**，否则会失稳。(第 9 章详述，定量的判据在第 7.12 节)

7.8 第七步：自适应律

从哪来

自适应律不是猜的，是从 Lyapunov 函数倒推出来的。

思路: 先写出误差动态(7.6 节已经得到) :

$$de_q/dt = -k_q \cdot e_q + (\Delta f - u_M) - u_r$$

设理想权值 $W1^*$ 使得 $\Delta f \approx W1^* \cdot \varphi_1$, 权值误差 $\Delta W1 = W1 - W1^*$, 则:

$$\Delta f - u_M = W1^* \cdot \varphi_1 - W1 \cdot \varphi_1 = -\Delta W1 \cdot \varphi_1$$

所以:

$$de_q/dt = -k_q \cdot e_q - \Delta W1 \cdot \varphi_1 - u_r + \varepsilon_1$$

(ε_1 是网络的逼近残差)

现在构造 Lyapunov 函数:

$$V1 = 0.5 \cdot e_q^2 + 0.5 \cdot \Delta W1^T \cdot \Gamma_1^{-1} \cdot \Delta W1$$

求导:

$$dV1/dt = e_q \cdot de_q/dt + \Delta W1^T \cdot \Gamma_1^{-1} \cdot d\Delta W1/dt$$

(注意 $d\Delta W1/dt = dW1/dt$, 因为 $W1^*$ 是常数)

代入 de_q/dt :

$$\begin{aligned} dV1/dt &= e_q \cdot (-k_q \cdot e_q - \Delta W1 \cdot \varphi_1 - u_r + \varepsilon_1) + \Delta W1^T \cdot \Gamma_1^{-1} \cdot dW1/dt \\ &= -k_q \cdot e_q^2 - e_q \cdot \Delta W1 \cdot \varphi_1 - e_q \cdot u_r + e_q \cdot \varepsilon_1 + \Delta W1^T \cdot \Gamma_1^{-1} \cdot dW1/dt \end{aligned}$$

关键一步: 我们希望中间两项抵消。也就是让:

$$\Delta W1^T \cdot \Gamma_1^{-1} \cdot dW1/dt = e_q \cdot \Delta W1 \cdot \varphi_1$$

因为这对任意 $\Delta W1$ 都要成立, 所以:

$$\Gamma_1^{-1} \cdot dW1/dt = e_q \cdot \varphi_1$$

两边乘 Γ_1 :

$$dW1/dt = \Gamma_1 \cdot \varphi_1 \cdot e_q$$

这就是自适应律。 它的形式完全是为了让 Lyapunov 导数里的交叉项抵消。

力矩通道的完整自适应律

加上一个"泄漏项"防止权值漂移(叫 **e-修正**) :

$$dW1/dt = \Gamma_1 \cdot (\varphi_1 \cdot e_q - v_e \cdot |e_q| \cdot W1)$$

- Γ_1 : 自适应增益(学习率)
- v_e : 泄漏系数

速度通道的自适应律(符号相反!)

速度通道的推导类似, 但**符号是反的**。原因在于估计量在控制律里的位置不同:

力矩通道: $\delta_e = (v - f_0 - u_M - \dots)/b$

-> $dq/dt = v + \Delta f - u_M - \dots$ -> **u_M 以减号进入**

速度通道: $T = m \cdot (dV_d + u_D - u_r V) + D_0 + \dots$

-> $dV/dt = dV_d + u_D - \Delta D/m - \dots$ -> **u_D 以加号进入**

因为进入了不同的符号位置，为了让 Lyapunov 交叉项对消，自适应律的符号也必须相反：

力矩通道: $dW1/dt = +\Gamma1 \cdot (\varphi1 \cdot e_q - \dots)$
速度通道: $dW2/dt = -\Gamma2 \cdot (\varphi2 \cdot e_v + \dots)$

这是一个极其隐蔽的坑。本工程调试时就写反了速度通道的符号，导致推力-速度回路出现发散的实极点。**推导时一定要把每一步的符号写清楚。**（第9章）

投影算子：保证权值有界

为什么需要：Lyapunov 证明要求 ΔW 有界。如果没有界， $\Delta W1^T \Gamma^{-1} \Delta W1$ 可能无限增大，证明就不成立。

投影算子的定义：

如果 $\|W\| < W_{\max}$ ，或者 ($\|W\| = W_{\max}$ 且 $W^T \cdot \tau \leq 0$):
输出 = τ
否则:
输出 = $\tau - (W \cdot W^T / \|W\|^2) \cdot \tau$

几何理解：

- 第一种情况：权值在球内，或者虽然在球面上但正要往球内走 -> 正常更新
- 第二种情况：权值在球面上且想往外走 -> **把更新的径向分量去掉**，只保留切向分量

效果：权值永远待在半径 $W_{\max}$ 的球内。

为什么用它而不是直接裁剪：直接裁剪 ($W = \min(W, W_{\max})$) 是不连续的，会破坏 Lyapunov 的负定性。投影算子是**光滑**的，只做“去掉径向分量”这一个动作，不破坏证明。

7.9 第八步：鲁棒项

$u_r = \eta_r \cdot \tanh(e_q / \Phi_r)$

- η_r ：鲁棒增益，必须满足 $\eta_r \geq \varepsilon_N + d_{\max}$ (网络残差上界 + 扰动上界)
- Φ_r ：边界层厚度

“边界层”的含义：误差在 $\pm \Phi_r$ 以内时， $\tanh$ 近似线性；超出后饱和。

取值原则： η_r / Φ_r (零点等效增益) **不能太大**。本工程最终取 $\eta_r = 2.5$ 、 $\Phi_r = 0.8$ ，等效增益约 3.1。而最初取 $\Phi_r = 0.06$ 时等效增益 20，直接把相位裕度吃光。

经验法则：鲁棒项的零点等效增益应当**不超过该回路比例增益的量级**。

7.10 第九步：速度回路

速度方程(切向)：

$dV/dt = [T \cdot \cos(\alpha + i_T) - D - m \cdot g \cdot \sin(\gamma)] / m$

反解推力：

$T_{\text{指令}} = [m \cdot (dV_d + u_D - u_rV) + D0 + m \cdot g \cdot \sin(\gamma_c)] / \cos(\alpha + i_T)$

其中：

$dV_d = dV_c/dt - k_V e_v - k_{IV} I_V$ (期望加速度)
 $I_V = \int e_v dt$ (误差积分，消除稳态误差)

注意两个细节：

细节 1：重力前馈取自 γ_c 而不是 γ 。

如果取实测的 γ ，会形成一条正反馈回路：

γ 增大 $\rightarrow$ 指令推力增大 $\rightarrow$ 推力增大 $\rightarrow$ 速度增大 $\rightarrow$ 升力增大 $\rightarrow$ $d\gamma/dt$ 增大 $\rightarrow$ γ 再增大

这条回路增益是 $m \cdot g = 117.7 \text{ N/rad}$ ，非常大。改成用指令值 γ_c 就**切断了这条回路**。（第9章）

细节2：鲁棒项前面是减号。

和力矩通道一样，为了让误差动态里出现 $-u_r V$ ，控制律里必须写成 $-u_r V$ 。写反了就是正反馈。

7.11 第十步：Lyapunov 稳定性证明

现在把上面的东西串起来，证明闭环系统是一致最终有界（UUB）的。

构造 Lyapunov 函数

$$V_L = 0.5 \cdot e_q^2 + 0.5 \cdot \Delta W_1^T \Gamma_1^{-1} \Delta W_1 + 0.5 \cdot e_v^2 + 0.5 \cdot \Delta W_2^T \Gamma_2^{-1} \Delta W_2 + 0.5 \cdot k_{IV} \cdot I_v^2$$

物理类比：这是一个“广义能量”。第一项是角速度误差的“动能”，第二项是权值误差的“势能”，依此类推。

证明的目标：证明这个“能量”随时间**单调下降**（至少是最终下降）。

求导并代入

把误差动态和自适应律代进去。**力矩通道的交叉项：**

$$\begin{aligned} & e_q \cdot (-\Delta W_1^T \Gamma_1 \dot{\phi}_1) + \Delta W_1^T \Gamma_1^{-1} \cdot (\Gamma_1 \cdot \dot{\phi}_1 \cdot e_q) \\ &= -e_q \cdot \Delta W_1^T \Gamma_1 \dot{\phi}_1 + \Delta W_1^T \Gamma_1^{-1} \cdot \Gamma_1 \cdot \dot{\phi}_1 \cdot e_q \\ &= 0 \quad \checkmark \text{ 精确抵消} \end{aligned}$$

速度通道的交叉项（注意符号）：

$$\begin{aligned} & e_v \cdot (\Delta W_2^T \Gamma_2 \dot{\phi}_2) + \Delta W_2^T \Gamma_2^{-1} \cdot (-\Gamma_2 \cdot \dot{\phi}_2 \cdot e_v) \\ &= e_v \cdot \Delta W_2^T \Gamma_2 \dot{\phi}_2 - \Delta W_2^T \Gamma_2^{-1} \cdot \Gamma_2 \cdot \dot{\phi}_2 \cdot e_v \\ &= 0 \quad \checkmark \text{ 也对消了} \end{aligned}$$

这就是为什么两个通道的自适应律符号必须相反。

剩下的项

对消之后剩下：

$$\begin{aligned} dV_L/dt &= -k_q \cdot e_q^2 - k_v \cdot e_v^2 - k_{IV} \cdot k_{IV}^{-1} \cdot (\dots) \\ &\quad + e_q \cdot (\varepsilon_1 + d_M) - e_q \cdot u_r \\ &\quad + e_v \cdot (\varepsilon_2 + d_V) - e_v \cdot u_r V \end{aligned}$$

（ d_M 、 d_V 是外部扰动）

用不等式放缩

对鲁棒项，用这个经典不等式：

$$|z| - z \cdot \tanh(z/\Phi) \leq 0.2785 \cdot \Phi$$

它的含义： $\tanh$ 和 sign 的差别被一个常数界住。

因为 $\eta_r \geq \varepsilon_N + d_M$ （鲁棒增益足够大），可以推出：

$$e_q \cdot (\varepsilon_1 + d_M) - e_q \cdot \eta_r \cdot \tanh(e_q/\Phi_r) \leq \eta_r \cdot 0.2785 \cdot \Phi_r$$

对速度通道同理。

得到最终的微分不等式

$$dV_L/dt \leq -2\lambda_{\min} V_L + c_0$$

其中 $c_0 = 0.2785 \cdot (\eta_r \Phi_r + \eta_v \Phi_v)$, $\lambda_{\min}$ 是所有回路增益里最小的那个。

解这个不等式

这是一个标准的线性微分不等式, 解出来:

$$V_L(t) \leq V_L(0) \cdot \exp(-2\lambda_{\min} t) + c_0/(2\lambda_{\min})$$

结论(UUB) :

$$||\text{误差}(t)|| \leq \sqrt{2 \cdot V_L(0) \cdot \exp(-2\lambda_{\min} t) / \lambda_{\min} + c_0 / \lambda_{\min}}$$

怎么读这个结论

两项的含义:

- **第一项:** 和初始误差有关, 随时间**指数衰减**(收敛速度由 $\lambda_{\min}$ 决定)
- **第二项:** $\sqrt{c_0/\lambda_{\min}}$, 是一个**常数**, 代表最终误差的界

关键理解:

1. **误差不能收敛到零**, 只能收敛到一个半径 $\sqrt{c_0/\lambda_{\min}}$ 的球内。原因是 $c_0 \neq 0$ (因为网络残差 $\varepsilon \neq 0$) 。
2. **想减小最终误差**, 有两条路:
 - 增大 $\lambda_{\min}$ (提高控制增益) ——但受相位裕度限制
 - 减小 c_0 (增大鲁棒增益 η 或减小边界层 Φ) ——但受"等效增益不能太大"限制
3. **这就是控制理论里的"没有免费午餐"**: 性能和鲁棒性永远在权衡。

级联系统的处理

上面只证明了内环(俯仰、速度通道) 。外环(高度) 是级联结构:

$$\begin{aligned} de_h/dt &= dh/dt - dh_c/dt \\ &= V \cdot \sin(\gamma) - dh_c/dt \end{aligned}$$

而 γ_d 是按" 要让高度误差一阶衰减"设计的, 所以:

$$de_h/dt = -k_h \cdot e_h + V \cdot (\sin(\gamma) - \sin(\gamma_c))$$

第一项是好项(负定) , 第二项是内环跟踪误差的贡献。

因为我们已经证明了内环误差是有界的(UUB) , 所以第二项有界, 于是:

$$de_h/dt \leq -k_h \cdot e_h + (\text{有界量})$$

用同样的方法(解线性不等式) , 可以证明 e_h 也是一**致最终有界**的。

总结: 内环 UUB + 外环输入有界 -> 整个闭环 UUB。这就是**级联系统的稳定性传递**。

7.12 第十一步: 离散化——一个必须面对的问题

问题

上面的自适应律都是连续时间的微分方程。但**真实飞控是数字计算机**, 按固定周期采样。

本工程用前向欧拉法离散:

$$W(k+1) = W(k) + T_s \cdot \Gamma \cdot \varphi \cdot e(k)$$

其中 $T_s = 0.005 \text{ s}$ (200 Hz) 。

离散化的稳定性条件

这个推导很关键，我们一步步来。 注意： 这里最容易犯的错是"只盯着回归向量范数"，而得出的判据里还必须带上该通道的**比例增益**。

第一步：只取两个状态

自适应回路里同时活跃的量很多(q 、 α 、 h 、 V 、权值.....)，但判断"采样会不会把自适应回路搞散"时，只需要看**两个状态**：

$e(k)$ —— 该通道的跟踪误差(力矩通道是 e_q ，速度通道是 e_v)
 $W(k)$ —— 该通道的权值

其余动态(对象、执行器、其他回路)在自适应收敛的时间尺度上变化慢得多，**当作冻结**。泄漏项(e -修正)和投影算子也先忽略——它们只会让系统更保守，不会让判据失效。

第二步：写出两状态线性化模型

以力矩通道为例。误差动态与自适应律(7.8 节) 取离散前向欧拉：

$$\begin{aligned} e(k+1) &= e(k) \cdot (1 - T_s \cdot k_q) - T_s \cdot \varphi_1^T \cdot \Delta W(k) \\ W(k+1) &= W(k) + T_s \cdot \Gamma_1 \cdot \varphi_1 \cdot e(k) \end{aligned}$$

用状态向量 $[e; \Delta W]$ 写成矩阵形式：

$$\begin{bmatrix} e(k+1) \\ \Delta W(k+1) \end{bmatrix} = \begin{bmatrix} 1 - T_s \cdot k_q & -T_s \cdot \varphi_1^T \\ T_s \cdot \Gamma_1 \cdot \varphi_1 & I \end{bmatrix} \begin{bmatrix} e(k) \\ \Delta W(k) \end{bmatrix}$$

特征多项式(按权值误差方向投影后， N 维权值退化成标量 $\|\varphi_1\|^2$)：

$$\lambda^2 - (1 - T_s \cdot k_q) \cdot \lambda + (1 - T_s \cdot k_q + T_s^2 \cdot \Gamma_1 \cdot \|\varphi_1\|^2) = 0$$

第三步：用 $|\det| < 1$ 得到判据

这个二阶系统的两个特征根之积(= 常数项) 就是系数矩阵的行列式：

$$\det = 1 - T_s \cdot k_q + T_s^2 \cdot \Gamma_1 \cdot \|\varphi_1\|^2$$

稳定要求 $|\det| < 1$ 。拆成两个不等式：

$$\bullet \det < 1 \rightarrow T_s^2 \cdot \Gamma_1 \cdot \|\varphi_1\|^2 < T_s \cdot k_q \rightarrow \text{两边约去 } T_s \rightarrow$$

$$T_s \cdot \Gamma_1 \cdot \|\varphi_1\|^2 < k_q$$

$$\bullet \det > -1 \rightarrow T_s^2 \cdot \Gamma_1 \cdot \|\varphi_1\|^2 > T_s \cdot k_q - 2, \text{ 当 } T_s \cdot k_q < 2 \text{ 时这个下界是负的，自动满足。}$$

所以真正的判据是：

$$T_s \cdot \Gamma_i \cdot \|\varphi_i\|^2 < k_i \quad (i=1 \text{ 力矩通道}, i=2 \text{ 速度通道})$$

其中：

- k_i 是该通道的误差反馈比例增益(力矩通道 $k_q = 6.00$ ，速度通道 $k_v = 1.53$)
- φ_i 是完整的回归向量，包含量纲缩放因子：

$$\begin{aligned} \varphi_1 &= \varphi_{\text{rbf}} \cdot (q_{\infty} \cdot S \cdot c / l_{yy}) \quad \leftarrow 7.7 \text{ 节那个 } 66 \text{ 倍的缩放} \\ \varphi_2 &= \varphi_{\text{rbf}} / m \end{aligned}$$

• 另一条常被忽略的约束来自 $\det > -1$: $T_s \cdot k_i < 2$, 即**误差反馈本身的欧拉离散约束**。

为什么它和“ $|1 - T_s \cdot \Gamma \cdot g^2| < 1$ ”那种写法不同: 那种写法只看自适应项, 漏掉了 k_i 这个“恢复力”。物理上很清楚——误差被比例增益 k_i 往回拉、被自适应项 $T_s \cdot \Gamma_i \cdot \|\varphi_i\|^2$ 往外推, **判据是两者比大小**, 不是自适应项自己小于某个常数。

代入本工程的数字

配平点: $V = 18 \text{ m/s}$ 、 $h = 0.20 \text{ m}$ 。

第一步: 量纲缩放因子

```
q $\infty$  = 0.5  $\times$  1.225  $\times$  182 = 198.45 Pa
g1 = q $\infty$ ·S·c / Iyy = 198.45  $\times$  0.96  $\times$  0.40 / 1.15 = 66.265 s-2
g2 = 1 / m = 1 / 12.0 = 0.083333 kg-1
```

第二步: 高斯基函数的范数(75 个节点、 $\sigma = 0.5$ 的网格, 在配平点附近的归一化坐标上取值)

```
 $\|\varphi_{1,rbf}\|^2 = 5.3943$ 
 $\|\varphi_{2,rbf}\|^2 = 5.3744$ 
```

(两者数值接近不是巧合——两个通道的网络结构相同, 只是输入向量第三维不同: 力矩通道用 q , 速度通道用 V 。)

第三步: 把缩放因子乘进去

```
 $\|\varphi_1\|^2 = g_1^2 \cdot \|\varphi_{1,rbf}\|^2 = 66.265^2 \times 5.3943 = 4391.0 \times 5.3943 \approx 2.369e4 \text{ [1/s}^4\text{]}$ 
 $\|\varphi_2\|^2 = g_2^2 \cdot \|\varphi_{2,rbf}\|^2 = 0.083333^2 \times 5.3744 = 6.944e-3 \times 5.3744 \approx 3.732e-2 \text{ [1/kg}^2\text{]}$ 
```

第四步: 代入判据

通道	$T_s \cdot \Gamma_i \cdot \ \varphi_i\ ^2$	比例增益 k_i	裕度比 $k_i / (T_s \cdot \Gamma_i \cdot \ \varphi_i\ ^2)$	结论
力矩($\Gamma_1 = 0.005$)	0.5922	$k_q = 6.00$	10.1	满足, 裕度 10 倍
速度($\Gamma_2 = 3.0$)	5.598e-4	$k_V = 1.53$	2733	满足, 裕度极大

两个通道都满足 $T_s \cdot \Gamma_i \cdot \|\varphi_i\|^2 \ll k_i$, **离散自适应回路稳定**。力矩通道的裕度只有 10 倍, 是两者中**更紧张**的那个——这也说明为什么调试时真正要盯的是 Γ_1 , 而不是看起来“大得多”的 Γ_2 。

如果回到最初那版 $\Gamma_1 = 45$:

```
 $T_s \cdot \Gamma_1 \cdot \|\varphi_1\|^2 = 0.005 \times 45 \times 2.369e4 \approx 5328 \gg k_q = 6$ 
```

超出判据近 900 倍, 自适应回路必然在几个采样周期内发散——这与当时观察到的“权值在 10 个采样周期内冲到投影上界、舵面瞬间饱和”完全一致。

历史版本的错误: 本教程早期把判据写成 $T_s \cdot \Gamma \cdot g^2 < 0.1$ (工程) / < 2 (绝对), 据此推出 $\Gamma < 0.0023$, 然后又承认实际用的 $\Gamma_1 = 0.005$ 超了——这是一个自相矛盾的结论。错误的根源有两个: 一是把判据里的 k_i 漏掉了; 二是把 $\|\varphi_{rbf}\|^2$ 当成 1~3 的常数去估, 而实际值在配平点是 5.39。**修正后的判据不再是“某个魔数 0.1”, 而是与该通道比例增益的比值**, 物理意义清楚, 也不会再自相矛盾。

为什么两个通道的增益数字差这么多

通道	增益	量纲缩放因子 g	说明
力矩	$\Gamma_1 = 0.005$	$q\infty Sc / I_{yy} \approx 66.265 \text{ s}^{-2}$	回归向量被放大 4391 倍(平方后 2.369e4)
速度	$\Gamma_2 = 3.0$	$1/m \approx 0.083333 \text{ kg}^{-1}$	回归向量被缩小到 1/144

两个缩放因子本身差了 `66.265 / 0.083333  $\approx$  795` 倍, 平方之后是 6.34e5 倍。回归向量放大得越厉害, 同样的 Γ 产生的权值更新就越大, 所以力矩通道的 Γ 必须相应地小得多——**这才是两组增益数字相差 600 倍`的真正原因`**。

注意 **但这里有一个必须纠正的说法**：不能因为“缩放的差异被增益的差异抵消了”就宣称两个通道的**有效学习率** $\Gamma \cdot g^2$ 是同一量级。恰恰相反：

力矩： $\Gamma_1 \cdot g_1^2 = 0.005 \times 4391 = 21.8$
速度： $\Gamma_2 \cdot g_2^2 = 3.0 \times 6.944e-3 = 0.0207$

两者相差约 1000 倍(三个数量级)，根本不是同一量级。之所以能用，是因为判据里的比较基准 k_i 也不同(6.00 vs 1.53)，而且两个通道的**误差量纲不同**(e_q 是 rad/s, e_v 是 m/s)，初始误差量级也不同， $\Gamma \cdot g^2$ 这个量本身**不具备跨通道可比性**。

有意义的比较量是无量纲的比值：

$\tau_s \cdot \Gamma_i \cdot \|\varphi_i\|^2 / k_i$ ： 力矩 0.0987 速度 3.66e-4

这两个数才是“离失稳边界还有多远”的正确度量，**两者都远小于 1**，说明两个通道都安全；力矩通道是相对更紧张的那个。

教训：自适应增益不能凭感觉给，也不能只看 Γ 的数值大小。要按该通道的无量纲数 $\tau_s \cdot \Gamma_i \cdot \|\varphi_i\|^2 / k_i$ 来判断，它必须远小于 1。

7.13 本章小结

- 控制问题拆成两条链：**高度链(四级级联)** 和 **速度链**。
- 外环用**比例 + 高度率阻尼**，高度率由 $V \cdot \sin(\gamma)$ 直接合成。
- $\theta_c = \gamma_c + \alpha$ 是由恒等式 $\gamma = \theta - \alpha$ 保证的，与控制律无关；它还自动补偿了升力误差。
- DSC 用**一阶滤波器代替解析求导**，解决微分爆炸。
- **动态逆 = 用模型反解控制量**；补偿后误差动态变成 $de/dt = -k \cdot e - \Delta W^T \varphi - u_r + \epsilon$ 。
- **自适应律是从 Lyapunov 函数倒推的**，形式完全为了让交叉项抵消。
- **两个通道的自适应律符号相反**，因为估计量在控制律里位置不同。
- **投影算子保证权值有界**，且不破坏 Lyapunov 负定性。
- 鲁棒项用 $\tanh$ ，零点等效增益 η/Φ 不能太大。
- **UUB 结论**：误差指数收敛到半径 $\sqrt{c_0/\lambda_{\min}}$ 的球内，不能到零。
- **离散稳定性条件** $\tau_s \cdot \Gamma_i \cdot \|\varphi_i\|^2 < k_i$ (φ_i 含量纲缩放、 k_i 是该通道比例增益)：忘记这一条会导致权值几个周期内爆掉。要比较的是无量纲比值 $\tau_s \cdot \Gamma_i \cdot \|\varphi_i\|^2 / k_i$ ，不是 Γ 的数值大小。

7.14 自测题

1. 为什么 $\theta_c = \gamma_c + \alpha$ 能自动补偿升力误差？请用恒等式说明。
2. 什么是“微分爆炸”？DSC 是怎么解决的？代价是什么？
3. 自适应律 $dW/dt = \Gamma \varphi e$ 的形式是怎么来的？为什么不是 $\Gamma e/\varphi$ ？
4. 投影算子在权值到达边界时做了什么？为什么不直接用裁剪？
5. 为什么 UUB 的界不能是零？ c_0 里包含哪些东西？
6. 判据 $\tau_s \cdot \Gamma_i \cdot \|\varphi_i\|^2 < k_i$ 里为什么会出现该通道的比例增益 k_i ？如果漏掉它，会得出什么错误结论？
7. 如果采样周期从 5 ms 改成 1 ms，按判据自适应增益最多可以增大几倍？还有什么前提？
8. 为什么不能直接比较两个通道的 Γ 数值来判断“哪个通道学得更快”？应该比较什么量？

第 8 章 代码逐文件讲解

这一章讲代码为什么这么写。不是把代码贴一遍，而是讲每个文件承担什么职责、为什么这样分、关键行为什么这么写。

8.1 整体结构

```
src/
├── 【建模层】把物理变成数学
│   ├── wig_params.m      名义模型参数( 控制器内部用的模型)
│   ├── wig_truth_params.m 真实对象参数( 含失配、RAM、非线性)
│   ├── wig_ge.m          地效因子 K(h)
│   ├── wig_aero.m         气动力与力矩
│   ├── wig_dynamics.m     七阶运动方程
│   ├── wig_trim.m         配平求解
│   └── wig_linearize.m     数值线性化
├── 【控制层】把数学变成控制律
│   ├── wig_rbf_init.m     RBF 网络结构与中心/宽度
│   ├── wig_rbf_phi.m      基函数求值
│   ├── wig_proj.m         投影算子
│   ├── wig_ctrl_init.m    控制器状态初始化
│   └── wig_ctrl_update.m  ★ 控制器核心( 单步更新)
├── 【仿真层】把控制律跑起来
│   ├── wig_simulate.m     闭环仿真引擎
│   ├── wig_metrics.m      性能指标
│   └── wig_closedloop_eig.m ★ 采样闭环极点分析
└── 【入口层】用户直接调用的脚本
    ├── run_all.m          一键运行全部
    ├── run_analysis.m     模型分析
    ├── run_scenarios.m    场景仿真
    ├── run_montecarlo.m   蒙特卡洛
    └── run_simulink.m     Simulink 验证
```

8.2 为什么这么分文件

原则：一个文件 = 一个物理概念

对比两种做法:

做法	优点	缺点
全部写在一个大脚本里	写起来快	无法单独测试；改一处要跑全部；出错难定位
按物理概念分文件	每个文件可单独调用、单独验证；改气动不用动控制器；出错能快速二分定位	文件多；需要管理函数签名

实战价值：本工程调试时，正是因为 wig_aero.m 可以单独调用，才能打印出力的分解，从而确认"力和状态导数是自治的"，把问题锁定在控制器侧。如果全写在一起，这一步根本做不了。

原则：参数和逻辑分离

参数全部集中在 wig_params.m 里，其他文件通过 p.xxx 访问。

好处：改参数不用翻遍所有文件；所有参数的物理含义集中在一处文档化。

原则：同名函数只做一件事

比如 wig_ge.m 只算地效因子，wig_aero.m 只算力 and 力矩。

为什么要拆开：因为 wig_ge 被三个地方调用(气动力、平尾、控制器)，拆开 after 保证三处用的是同一个地效模型。

如果各自实现一份，很容易不一致。

8.3 wig_params.m: 参数集怎么组织

分层组织

```
%% 环境与常量
p.g = 9.80665; p.rho = 1.225;

%% 质量与惯量
p.m = 12.0; p.Iyy = 1.15;

%% 机翼几何
p.b = 2.40; p.c = 0.40;
p.S = p.b * p.c; % 派生量: 直接算出来, 不重复填
p.AR = p.b^2 / p.S;

%% 地效模型
p.ge_A = 0.0641; p.ge_n = 1.4622;

%% 控制器参数
p.ctrl.k_h = 2.00; p.ctrl.k_hd = 0.80; ...
```

几个设计决定

决定 1: 派生量当场算，不手填。

```
p.S = p.b * p.c; % 而不是 p.S = 0.96;
p.AR = p.b^2 / p.S; % 而不是 p.AR = 6.0;
```

为什么: 如果手填，改了翼展忘了改面积就会出错。让计算机算，永远不会不一致。

决定 2: 控制器参数放在子结构体 `p.ctrl` 里。

```
p.ctrl.k_h % 而不是 p.k_h
```

为什么: 这样可以把整个 p.ctrl 传给控制器函数，和对象参数 p 区分开。调试时想试不同增益，只要改 p.ctrl 就行，不影响对象模型。

决定 3: 加了 `p.is\_truth` 标志。

```
p.is_truth = false; % 名义模型
% 在 wig_truth_params 里: pt.is_truth = true;
```

为什么: 气动函数需要知道“我现在算的是名义还是真实”。用字段而不是额外的参数，可以让函数签名保持简单。

8.4 wig_ge.m: 地效因子

```
function [kap, dkap] = wig_ge(h, span, p)
A = p.ge_A;
n = p.ge_n;
x = max(h, 0) / span; % 归一化高度
if x <= 0
    kap = 0; dkap = 0; return;
end
xn = x^n;
den = A + xn;
kap = xn / den;
dkap = n * A * x^(n-1) / (span * den^2); % 对 h 的导数
```

为什么要有`span`参数

因为机翼和平尾用的翼展不同:

```
kap_w = wig_ge(h, p.b, p); % 机翼
kap_t = wig_ge(h + p.dz_t, p.bt, p); % 平尾(不同高度、不同翼展)
```

为什么平尾要用不同高度: 平尾比机翼高 0.30 m(抬高安装), 所以它感受到的地效比机翼弱。

为什么平尾要用不同翼展: 地面效应本质是"地面挡住翼尖涡", 涡的尺度是各自的翼展。

为什么要返回导数`dkap`

第5章计算 $\partial M / \partial h$ 时需要用到。虽然本工程的主要分析用的是数值差分, 但保留解析导数便于验证。

边界处理

```
x = max(h, 0) / span;
if x <= 0
    kap = 0; return;
end
```

为什么: $x^{(n-1)}$ 当 $x = 0$ 且 $n < 1$ 时会发散, 当 $n > 1$ 时导数为 0。这里直接返回, 避免数值问题。而且物理上 $h < 0$ 就是撞地了, 不做讨论。

8.5 wig_aero.m: 气动力

这是最长的文件, 因为要算的东西多。

整体流程

```
function [F, aux] = wig_aero(x, p, flag)
% 1. 算地效因子
kap_w = wig_ge(h, p.b, p);
h_t = h + p.dz_t;
kap_t = wig_ge(h_t, p.bt, p);

% 2. 算有效升力线斜率
aw = p.a0 / (1 + p.a0 * kap_w / (pi * p.AR * p.e));
at = p.a0t / (1 + p.a0t * kap_t / (pi * p.ART * p.et));

% 3. 算机翼气动
CLw_lin = aw * (alpha - p.aL0);
CDi_w = CLw_lin^2 * kap_w / (pi * p.AR * p.e);
CDw = p.CD0 + CDi_w;

% 4. 真实对象额外加RAM和失速
if truth
    CLram = p.k_ram / (1 + (h / p.h_ram)^2);
    CLtot = p.CLmax * tanh((CLw_lin + CLram) / p.CLmax);
    dCLnl = CLtot - CLw_lin - CLram;
end

% 5. 算平尾
eps_t = (p.eps0 + p.eps_a * alpha) * kap_w;
alpha_t = alpha + p.it - eps_t + q * p.lt / V;
CLt = at * (alpha_t - p.aL0t) + p.a_de * de;

% 6. 累加力和力矩
...

% 7. 输出
F = [L; D; M];
```

关键决定: `flag` 参数与 `is\_truth` 字段

```
truth = p.is_truth || strcmpi(flag, 'truth');
```

两种判断方式并存的原因:

- 调用方可能显式传 'truth'
- 也可能因为参数集本身是真实的(`p.is_truth = true`) 而自动判定

为什么不做成两个函数: 因为 90% 的代码是共享的(机翼、平尾、力矩累加) , 只有几行不同(RAM、失速) 。复制两份代码是维护灾难。

输出为什么有两个

```
function [F, aux] = wig_aero(x, p, flag)
```

- `F = [L; D; M]`: 仿真需要的三个量
- `aux`: 辅助量(地效因子、迎角、各分力) , **只在需要时返回**

为什么用 `if nargout > 1`:

```
if nargout > 1
    aux.qbar = qbar;
    ...
end
```

为什么: 仿真主循环里每一步都调用这个函数, 如果无条件构造 `aux` 结构体会拖慢速度。只在调试和绘图时才要它。

这是一个重要的性能习惯: 热路径上不要做"反正也不贵"的事, 因为它是每一步都要做的。

力矩的统一写法

```
% 通用形式:  $M = z * F_x - x * F_z$ 
Fx_w = Lw * sin(alpha) - Dw * cos(alpha);
Fz_w = -Lw * cos(alpha) - Dw * sin(alpha);
M_w = p.z_acw * Fx_w - p.x_acw * Fz_w + qbar * p.S * p.c * p.Cmac;
```

为什么每个力矩项都写成这个形式: 因为符号约定最容易错。用统一公式, 只需要保证 (x, z, F_x, F_z) 填对, 符号就不会错。

对比: 如果每个力矩项都单独推导符号("机翼抬头为正, 平尾低头为负.....") , 几乎肯定会错一两处, 而且很难查。

8.6 wig_dynamics.m: 运动方程

结构

```
function dx = wig_dynamics(x, ucmd, p, flag, gust)
% 1. 解包状态
V = max(x(1), 0.5); gam = x(2); alpha = x(3);
q = x(4); h = x(5); T = x(6); de = x(7);

% 2. 执行器( 限幅 + 速率限制)
de_c = wig_sat(ucmd(1), p.de_max);
dt_c = wig_sat(ucmd(2), 0.0, 1.0);
T_cmd = p.Tmin + dt_c * (p.Tmax - p.Tmin);
de_rate = wig_sat((de_c - de) / p.tau_e, p.de_rate);

% 3. 气动力( 含阵风修正)
if ~isempty(gust) && any(gust ~= 0)
    ... 用相对气流重算 V、gamma、alpha ...
end
```

```

F = wig_aero(xa, p, flag);

% 4. 运动方程
Tx = T * cos(alpha + p.i_T);
Tz = T * sin(alpha + p.i_T);
Falong = Tx - D - p.m * p.g * sin(gam);
Fperp = L + Tz - p.m * p.g * cos(gam);

dx(1) = Falong / p.m;          % dV/dt
dx(2) = Fperp / (p.m * V);     % dgamma/dt
dx(3) = q - dx(2);             % dalpha/dt
dx(4) = M / p.Iyy;             % dq/dt
dx(5) = V * sin(gam);          % dh/dt
dx(6) = (T_cmd - T) / p.tau_T; % dT/dt
dx(7) = de_rate;               % dde/dt

```

关键细节

细节 1: `V = max(x(1), 0.5)`

$dy/dt = F_{perp}/(m \cdot V)$ ，如果 V 趋近零会除零。虽然实际不会发生，但数值积分中可能出现瞬时小值。加个下限是**廉价的安全措施**。

细节 2: `dx(3) = q - dx(2)`

直接复用刚算出来的 $dx(2)$ ，而不是重算一遍 $F_{perp}/(m \cdot V)$ 。**既快又保证一致**。

细节 3: 阵风修正放在气动计算之前

```

Vx = V*cos(gam) - gust(1);
Vz = V*sin(gam) - gust(2);
Va = sqrt(Vx^2 + Vz^2);
gam_a = atan2(Vz, Vx);
alpha_a = (gam + alpha) - gam_a; % 用 theta 减去气流航迹角

```

物理含义：阵风改变了**相对气流**的方向和大小，所以气动力要按相对气流算，但运动方程仍按地面参考系写。

`alpha_a = (gam + alpha) - gam_a` 这一行为什么这么写：

- $\text{gam} + \alpha$ 就是俯仰角 θ (用恒等式)
- gam_a 是气流航迹角
- 所以 $\alpha_a = \theta - \text{gam}_a$ 正是迎角的定义

写成 `(gam + alpha)` 而不是新变量 θ ，是为了强调"这里用的是 $\theta = \gamma + \alpha$ 这个恒等式"。

执行器的速率限制

```
de_rate = wig_sat((de_c - de) / p.tau_e, p.de_rate);
```

为什么要有速率限制：真实舵机有最大转速。没有它，仿真会给出"舵瞬间到位"的假象。

为什么用 `sat` 而不是分段：`sat` 函数连续，数值积分更稳定。

8.7 wig_trim.m: 配平

双求解器策略

```

% 主求解器: fsolve
try
    [zsol, ~, exitflag] = fsolve(@(zz) trim_residual(zz, V, h, p, flag), z, opts);
    ok = (exitflag > 0);
catch

```

```

    ok = false;
end

% 回退: 自研阻尼牛顿法
if ~ok
    [zsol, ok] = newton_trim(z, V, h, p, flag);
end

```

为什么要回退: fsolve 属于 Optimization Toolbox, 不是所有 MATLAB 安装都有。**工程代码必须考虑"依赖不存在"的情况。**

这是很实际的考虑: 如果别人拿到你的代码跑不起来, 代码就没有价值。

残差的无量纲化

```

W = p.m * p.g;
r = [ (F(1) + T*sin(alpha + p.i_T) - W) / W;    % 除以重量
      (T*cos(alpha + p.i_T) - F(2)) / W;
      F(3) / (W * p.c) ];    % 力矩÷(重量×弦长)

```

为什么除以 w 和 w·c: 让三个残差的量级接近。

如果不除: 力的残差量级是 100 N, 力矩的残差量级是 1 N·m。求解器看到"力误差 100"和"力矩误差 1", 会认为力误差严重得多, 从而忽略力矩平衡。

归一化后, 三个残差都是无量纲的、量级约 1, 求解器才能公平对待。

这是一个通用技巧: 任何数值求解问题, 把不同量纲的量归一化到同一量级。

阻尼线搜索

```

lam = 1.0;
for k = 1:30
    zn = z + lam*d;
    if norm(trim_residual(zn, ...)) < norm(r)
        break; % 找到能减小残差的步长
    end
    lam = lam * 0.5; % 否则减半步长
end
z = z + lam*d;

```

为什么: 牛顿法算出的步长方向是对的, 但大小可能过大。线搜索保证每一步都真的让残差下降, 避免发散。

这是非线性求解的标准做法, 不是可选项。

8.8 wig_rbf_init.m: 网络结构

```

function nn = wig_rbf_init(p)
% 力矩通道: 输入 [alpha, q, h], 5x3x5 = 75 节点
rng1 = [p.ctrl.nn.c1_a(:)'; p.ctrl.nn.c1_q(:)'; p.ctrl.nn.c1_h(:)'];
n1 = p.ctrl.nn.n1(:)';
nn.C1 = grid_centers(n1); % 3 x 75
nn.sig1 = grid_sigma(n1); % 3 x 1
nn.N1 = size(nn.C1, 2);
...

```

中心为什么用矩阵存

nn.C1 是 3×75 的矩阵, 每一列是一个中心点。

为什么这么存: 因为求基函数时要算"输入到所有中心的距离":

```
d2 = zeros(1, N);
for i = 1:3
    d2 = d2 + ((z(i) - C(i,:)) ./ sig(i)).^2;
end
phi = exp(-0.5 * d2);
```

$C(i,:)$ 是第 i 维的所有中心，一次就能算 75 个距离。**这是 MATLAB 的向量化写法**，比循环 75 次快得多。

为什么不用 `pdist` 或 `bsxfun`：因为要保证代码能在**代码生成**(MATLAB Coder) 环境下工作，那些函数不一定支持。用基本的算术运算最保险。

宽度为什么取网格间距

```
function s = grid_sigma(n)
for i = 1:3
    if n(i) > 1
        s(i) = 2/(n(i) - 1); % 归一化区间长度 2 除以间隔数
    else
        s(i) = 1.0;
    end
end
end
```

以 5 个中心为例：归一化区间是 $[-1, 1]$ ，长度 2，4 个间隔，所以间距 = 0.5。

为什么宽度 = 间距：这样相邻基函数在中心处的值是 $\exp(-0.5) \approx 0.61$ ，重叠适中。

- 宽度太大 -> 所有基函数都差不多，网络分辨不出细节
- 宽度太小 -> 基函数之间有空隙，网络在空隙处输出接近零

8.9 wig_ctrl_update.m：控制器核心(逐段讲解)

这是最重要的文件。我们用"输入-处理-输出"的顺序讲。

函数签名

```
function [u, ctrl, dbg] = wig_ctrl_update(xm, ref, ctrl, p)
```

三个输出：

- u：控制量 [δ_e 指令; 油门指令]
- ctrl：更新后的控制器内部状态
- dbg：调试量(误差、中间指令、网络输出.....)

为什么控制器状态要显式传入传出，而不是用 `persistent`：

做法	优点	缺点
persistent 变量	调用简单	无法查看内部状态; 无法在多个实例间切换; Simulink 里不好用
显式传递	纯函数、可测试、可复用、Simulink 可用	调用时要多传一个参数

这个选择决定了同一份代码能同时用于脚本仿真和 Simulink 的 S-Function，是后面"两种仿真途径结果一致"的前提。

第 0 段：量测滤波

```
if isempty(ctrl.xf)
    ctrl.xf = xm([1 3 4 5]); % 首次调用：用当前值初始化
end
tau_f = max(p.sens.tau_f, 1e-4);
ctrl.xf = ctrl.xf + (Ts_/tau_f) * (xm([1 3 4 5]) - ctrl.xf);
Vf = ctrl.xf(1); alf = ctrl.xf(2); qf = ctrl.xf(3); hf = ctrl.xf(4);
```

为什么要有这层滤波: 传感器有噪声, 直接拿噪声去算控制量会导致舵机抖动。

``max(p.sens.tau_f, 1e-4)`` 的作用: 防止 $\tau_f = 0$ 导致除零。如果 τ_f 太小(比如 $1e-4$ 而 $T_s = 0.005$) , $T_s/\tau_f = 50 > 2$, 这个欧拉积分会**发散**。这是一个必须注意的数值问题。

教训: 用欧拉法实现一阶滤波器时, 必须满足 ``Ts/τ < 2``。

第 1 段: RBF 网络求值

```
z1 = wig_norm([alf; qf; hf], ctrl.nn.rng1); % 归一化
ph1r = wig_rbf_phi(z1, ctrl.nn.C1, ctrl.nn.sig1); % 基函数
phi1 = ph1r * (qbar * p.S * p.c / p.lyy); % 加权( 量纲处理)
uM = ctrl.W1' * phi1; % 网络输出
```

这一段的三个步骤对应第 7 章的三个推导:

- 1. wig_norm: 归一化到 $[-1,1]$ (第 7.7 节)
- 2. wig_rbf_phi: 算 $\phi_j(z)$ (第 7.7 节的公式)
- 3. 乘 $q \propto S_c/l_{yy}$: 量纲处理(第 7.7 节的"回归向量缩放")

``ctrl.W1' * phi1`` 为什么是转置乘:

- $W1$ 是 $N \times 1$ 列向量
- $\phi1$ 也是 $N \times 1$ 列向量
- $W1' * \phi1$ 得到标量(内积) = $\sum W1_j \phi1_j$

如果写成 ``W1 * phi1`` 会得到 $N \times N$ 矩阵 —— 这是 MATLAB 里最常见的错误之一。

第 2 段: 高度外环

```
e_h = hf - h_c;
hd_act = Vf * sin(gam); % 高度率: 不用数值微分
arg = (hd_c - p.ctrl.k_h * e_h - p.ctrl.k_hd * (hd_act - hd_c)) / max(Vf, 3.0);
gam_d = asin(wig_sat(arg, sin(p.ctrl.gam_max)));
gam_dot_c = (wig_sat(gam_d, p.ctrl.gam_max) - ctrl.gam_c) / p.ctrl.tau_g;
```

逐个说明为什么这么写:

代码	对应公式	为什么
<code>hd_act = Vf * sin(gam)</code>	$dh/dt = V \cdot \sin \gamma$	直接用状态合成, 不用数值微分(微分会放大噪声)
<code>max(Vf, 3.0)</code>	防止除零	速度极低时 γ 指令会爆掉
<code>wig_sat(arg, sin(gam_max))</code>	限幅	arcsin 的参数必须 ≤ 1 , 且实际不允许大航迹角
除以 <code>p.ctrl.tau_g</code>	一阶滤波器	这就是 $d\gamma_c/dt$, 不需要解析求导

第 3 段: 姿态内环

```
e_th = theta - ctrl.th_c;
theta_d = wig_sat(ctrl.gam_c + alf, p.ctrl.th_max); % 恒等式 + 限幅
th_dot_c = (wig_sat(theta_d, 0.80) - ctrl.th_c) / p.ctrl.tau_th;

q_d = th_dot_c - p.ctrl.k_th * e_th;
e_q = qf - ctrl.q_cf;
q_dot_cf = (wig_sat(q_d, 6.0) - ctrl.q_cf) / p.ctrl.tau_q;

nu = q_dot_cf - p.ctrl.k_q * e_q;
```

关键点 1: ``theta = gam + alf``(用滤波后的迎角)

代码开头有这么一行:

```
theta = gam + alf; % 与指令通道保持一致, 使用滤波后迎角合成俯仰角
```

为什么要用滤波后的 α 而不是原始 α :

因为指令 $\theta_d = \gamma_c + \alpha_f$ 用的也是滤波后的 α 。如果误差 $e_{\theta} = \theta - \theta_c$ 用原始 α , 指令用滤波 α , 两者之间会有一个"滤波滞后"的虚假误差, 形成不必要的反馈路径。

保持一致是这里的关键原则。

关键点 2: 两次 DSC 滤波

- 第一次: $\theta_d \rightarrow \theta_c$, 时间常数 τ_{θ}
- 第二次: $q_d \rightarrow q_{cf}$, 时间常数 τ_q

为什么要两次: 因为反步法每一级都需要上一级虚拟控制量的导数, 两次滤波提供了两个导数($\dot{\theta}_c$ 和 $\dot{q}_{cf}$)。

关键点 3: 滤波顺序

```
q_dot_cf = (sat(q_d) - ctrl.q_cf) / tau_q; % 先算导数( 用当前的 q_cf)
nu = q_dot_cf - k_q * e_q; % 用导数算控制量
...
ctrl.q_cf = ctrl.q_cf + Ts * q_dot_cf; % 最后才更新滤波器状态
```

为什么这个顺序: 控制量必须用当前时刻的滤波器状态算, 更新要放在最后。如果顺序错了, 控制就用了"未来"的信息, 因果关系被破坏。

第 4 段: 动态逆

```
xtmp = [V; gam; alpha; q; h; T; 0]; % 舵偏置零
Fnom = wig_aero(xtmp, p, 'nominal'); % 用名义模型算
Mn0 = Fnom(3); % 零舵偏力矩
f0 = Mn0 / p.Iyy;

kap_w = wig_ge(h, p.b, p);
eps_t = (p.eps0 + p.eps_a * alpha) * kap_w;
alpha_t = alpha + p.it - eps_t + q * p.lt / max(V, 0.5);
qt = qbar * p.eta_t;
Mde = -(p.lt*cos(alpha_t) + p.dz_t*sin(alpha_t)) * qt * p.St * p.a_de;
b = Mde / p.Iyy;
```

关键点 1: `xtmp` 里舵偏置零

因为我们要的是 $M_0(x)$ (零舵偏力矩), 舵偏的贡献由 $b \cdot \delta_e$ 单独处理。这就是第 7.6 节的模型分解。

关键点 2: `Mde` 是解析求出的, 不是数值差分

$$M_{\delta_e} = \partial M / \partial \delta_e = -(l_t \cos \alpha_t + \Delta z_t \sin \alpha_t) \cdot q_t \cdot S_t \cdot a_{\delta_e}$$

这个表达式是从第 3 章的力矩公式对 δ_e 求导得来的:

- $C_{L_t} = \dots + a_{\delta_e} \delta_e$
- $M_t = -\Delta z_t F_{x,t} + l_t F_{z,t}$
- 对 δ_e 求导: $\partial F_{x,t} / \partial \delta_e = q_t S_t a_{\delta_e} \sin \alpha_t$, $\partial F_{z,t} / \partial \delta_e = -q_t S_t a_{\delta_e} \cos \alpha_t$
- 代入得 $M_{\delta_e} = -(l_t \cos \alpha_t + \Delta z_t \sin \alpha_t) \cdot q_t S_t a_{\delta_e}$

为什么这里用解析式而不是数值差分: 因为这个量在控制器里每一步都要算, 数值差分要调用 2 次 `wig_aero`, 太慢。而它的解析式很简单。

关键点 3: `b` 的数值保护

```
if abs(b) < 1e-8
    b = -1e-8; % 防止除零
end
```

$\delta_e = (v - \dots)/b$ 。如果 b 接近零(速度极低时) , 控制量会爆掉。**这是必要的保护。**

第 5 段: 鲁棒项与控制律

```
if p.ctrl.enable_rob
    ur = p.ctrl.eta_r * tanh(e_q / p.ctrl.Phi_r);
else
    ur = 0;
end

de_unsat = (nu - f0 - uM - ur) / b;
de_cmd = wig_sat(de_unsat, p.de_max);
sat_e = abs(de_unsat) >= p.de_max; % 记录是否饱和
```

`sat\_e` 这个标志位的作用: 后面要用它决定"要不要继续学习"(见第 6 段) 。

第 6 段: 速度回路 with 自适应律

```
e_V = Vf - V_c;
IV_new = wig_sat(ctrl.IV + Ts * e_V, p.ctrl.IV_max);
dV_des = Vd_c - p.ctrl.k_V * e_V - p.ctrl.k_IV * IV_new;

urV = p.ctrl.eta_V * tanh(e_V / p.ctrl.Phi_V);

ca = cos(alpha + p.i_T);
if abs(ca) < 0.2, ca = sign(ca) * 0.2; end

gff = p.m * p.g * sin(ctrl.gam_c); % ★ 用指令航迹角, 不是量测值
T_cmd = (p.m * (dV_des + uD - urV) + Dnom + gff) / ca;
```

三个关键点:

关键点 1: **`-urV` 是减号**

第 7.9 节讲过, 鲁棒项必须以减号进入, 才能让误差动态里出现 $-u_rV$ 。**写成加号就是正反馈。**(第 9 章的第一个坑)

关键点 2: **`gff` 用 `ctrl.gam\_c` 而不是 `gam`**

用实测 γ 会形成正反馈回路。(第 9 章的第四个坑)

关键点 3: **`ca` 的保护**

$ca = \cos(\alpha + i_T)$, 如果迎角接近 90° 会趋零。加一个下限保护。

第 7 段: 自适应律

```
nu_e = p.ctrl.nn.nu_e;
if p.ctrl.enable_nn
    if ~sat_e % ★ 舵饱和时冻结学习
        tau1 = p.ctrl.nn.Gam1 * (phi1 * e_q - nu_e * abs(e_q) * ctrl.W1);
        ctrl.W1 = wig_proj(ctrl.W1, tau1, p.ctrl.nn.Wmax, Ts_);
    end
    if ~sat_t % 油门饱和时冻结
        tau2 = -p.ctrl.nn.Gam2 * (phi2 * e_V + nu_e * abs(e_V) * ctrl.W2);
        ctrl.W2 = wig_proj(ctrl.W2, tau2, p.ctrl.nn.Wmax, Ts_);
    end
end
```

关键点 1: 两个通道的符号相反

```
tau1 = +Gam1 * (phi1 * e_q - ...); % 力矩通道: 正号
tau2 = -Gam2 * (phi2 * e_V + ...); % 阻力通道: 负号
```

这是第 7.8 节推导的直接结果，代码里专门写了注释解释原因。

关键点 2: 饱和时冻结学习

为什么: 如果舵已经打到限幅(比如 $\pm 25^\circ$)，那么误差 e_q 已经**不是控制器的责任**了——是执行器能力不够。这时候如果继续学习，网络会把"执行器饱和"误认为是"模型失配"，学出错误的东西。

这叫**抗积分饱和**(anti-windup)，是工程控制的必备技巧。

关键点 3: `wig\_proj` 里包含了欧拉积分

```
function [W, Wd] = wig_proj(W, tau, Wmax, Ts)
Wd = wig_proj_op(W, tau, Wmax); % 先做投影
W = W + Ts * Wd; % 再欧拉积分
n = norm(W);
if n > Wmax * 1.001 % 数值安全网
    W = W * (Wmax / n);
end
```

为什么把积分也放进来: 调用方只需要写一行，不容易漏掉积分或投影。

为什么要那个 1.001 的收缩: 投影算子理论上保证有界，但浮点误差可能让范数略微超界。加一个几乎不会触发的安全网。

8.10 wig_simulate.m: 仿真引擎

核心循环结构

```
for k = 1:Nt
    t = tvec(k);

    % 1. 参考指令二阶滤波
    zh = zh + dt * [zh(2); wn_h^2*(ref_h(k) - zh(1)) - 2*wn_h*zh(2)];

    % 2. 控制器采样(零阶保持)
    if t >= next_u - 1e-12
        [u, ctrl, dbg] = wig_ctrl_update(xm, ref, ctrl, pc);
        next_u = next_u + Ts;
    end

    % 3. 记录
    X(k,:) = x';

    % 4. RK4 积分被控对象
    k1 = wig_dynamics(x, u, pt, plant, g);
    k2 = wig_dynamics(x + 0.5*dt*k1, u, pt, plant, g);
    k3 = wig_dynamics(x + 0.5*dt*k2, u, pt, plant, g);
    k4 = wig_dynamics(x + dt*k3, u, pt, plant, g);
    x = x + (dt/6)*(k1 + 2*k2 + 2*k3 + k4);
end
```

为什么要用零阶保持:

真实飞控计算机是**每 5 ms 算一次**，然后保持这个控制量 5 ms。所以:

```
if t >= next_u - 1e-12 % 到了采样时刻
    ...控制器算一次...
    next_u = next_u + Ts; % 下次采样时刻
end
% 两次采样之间, u 保持不变(零阶保持)
```

这个细节很重要: 如果每个积分步都调用控制器(相当于 1000 Hz

控制器)，性能会明显变好，但那是假的——真实计算机做不到。

``-1e-12``的作用：浮点数累加误差。 0.005×20 可能不严格等于 0.1，加个极小量避免漏采样。

参考指令的二阶滤波器

```
zh = zh + dt * [zh(2); wn_h^2*(ref_h(k) - zh(1)) - 2*wn_h*zh(2)];
```

为什么不用阶跃直接给指令：

阶跃指令会导致：

1. 控制器输出突变(微分项会给出无穷大)
2. 执行器瞬间饱和
3. 结构受冲击

用二阶滤波器(相当于一个质量-弹簧-阻尼系统)：

```
z1'' + 2ζω·z1' + ω²·z1 = ω²·目标值
```

- ω_n ：带宽，决定指令响应快慢
- $\zeta = 1$ ：临界阻尼，无超调

这样同时得到指令值、一阶导、二阶导($zh(1)$ 和 $zh(2)$)，控制器前馈需要它们。

机械类比：这就是一个“指令平滑器”，像汽车油门的电子节气门，避免猛踩猛松。

8.11 几个 MATLAB 编程要点

要点 1：预分配数组

```
X = zeros(Nt, nx); % 预先分配
for k = 1:Nt
    X(k,:) = x';
end
```

为什么不直接 `X(end+1,:) = x'`：那样 MATLAB 每次都要重新分配内存并复制。对于 40000 步的仿真，差别是几十倍的速度。

要点 2：向量化

```
d2 = zeros(1, N);
for i = 1:3
    d2 = d2 + ((z(i) - C(i,:)) ./ sig(i)).^2; % 一次算 75 个
end
```

为什么不用双重循环：MATLAB 对向量运算有优化，向量化后快 10~100 倍。

要点 3：`wig\_sat` 而不是 `min(max(...))`

```
function y = wig_sat(u, lo, hi)
if nargin < 3
    hi = lo;
    lo = -hi; % 支持对称限幅: wig_sat(x, 0.5) => [-0.5, 0.5]
end
y = min(max(u, lo), hi);
```

为什么要封装：因为限幅在代码里出现十几次。封装后：

- 语义清晰(读代码时一眼看出是限幅)
- 支持对称和非对称两种用法

- 将来要改成平滑限幅(比如 $\tanh$) 只改一处

要点 4: UTF-8 日志

```
fid = fopen(logfile, 'w', 'n', 'UTF-8');
fprintf(fid, '%s', s);
fclose(fid);
```

为什么不用 MATLAB 的 `diary`: `diary` 使用系统默认编码(中文 Windows 上是 GBK) , 在 PowerShell 里读会乱码。
显式指定 UTF-8 保证日志在任何环境下都能正确阅读。

要点 5: 异常也要保留输出

```
cmd = sprintf(['try\n' ...
    ' %s;\n' ...
    'catch ME\n' ...
    ' disp(getReport(ME));\n' ...
    'end\n'], fcnName);
s = evalc(cmd);
```

为什么这样写: `evalc` 在函数抛异常时**不会返回已产生的输出**, 调试时会丢掉所有线索(本工程就吃过这个亏) 。把 `try/catch` 放进 `evalc` 内部, 就能保住报错前的全部输出。

8.12 本章小结

- **一个文件 = 一个物理概念**, 这样能单独测试、快速定位问题。
- **参数与逻辑分离**, 派生量当场计算。
- 气动函数用 $[F, aux]$ 双输出 + `nargout` 判断, 兼顾速度与调试。
- 力矩统一用 $M = z \cdot F_x - x \cdot F_z$, 避免符号错误。
- 控制器状态**显式传入传出**, 这让同一份代码能用于 Simulink。
- 仿真引擎用**固定步长 RK4 + 200 Hz 零阶保持控制器**, 模拟真实数字飞控。
- 参考指令用**二阶滤波器**平滑, 同时提供一二阶导数。
- 自适应律里**两个通道符号相反, 饱和时冻结学习**。
- 一阶滤波器的欧拉实现必须满足 $T_s/\tau < 2$ 。

8.13 自测题

1. 为什么控制器状态要用"显式传入传出"而不是 `persistent`? 这对 Simulink 有什么好处?
2. $hd_act = V_f \sin(gam)$ 为什么不用 $(hf - hf_old)/T_s$ 来算?
3. $ctrl.W1 * \phi_1$ 如果写成 $ctrl.W1 * \phi_1'$ 会得到什么? 为什么错?
4. 舵机饱和时冻结自适应学习, 背后的道理是什么? 如果不冻结会发生什么?
5. 为什么参考指令要用二阶滤波器而不是直接给阶跃? 二阶滤波器同时提供了什么?
6. 一阶滤波器用欧拉法实现, 为什么要求 $T_s/\tau < 2$?

第9章 调试踩过的坑

这一章讲**实战**。前面所有的理论，最终都要落到“代码跑不通怎么办”上。

本工程从“模型能跑”到“控制器能飞”，中间修了五个关键缺陷。**每一个都花了很长时间才找到，而且每一个都能从理论上解释清楚。**

这一章的价值在于：**你以后遇到类似症状，能快速定位。**

9.1 先说方法：怎么定位“仿真发散”

发散是最常见的现象，但原因可能有十几种。**关键是把问题切小。**

本工程用的排查顺序：

```
第1步：模型对不对？ -> 开环仿真，从配平点出发，看能不能保持不动
第2步：积分对不对？ -> 检查状态的有限差分 and 解析导数是自治的
第3步：控制器对不对？ -> 单独调用控制器，看第一步输出是否合理
第4步：闭环哪里不稳？ -> 求闭环特征值，看哪个状态参与最多
```

每一步都有对应的诊断脚本(run_00_chk.m、run_00_dbg.m、run_00_clp.m)，这些脚本后来都保留在交付里——它们是**结论的证据链**。

第1步的实际结果

开环从配平点出发，固定配平控制，跑3秒：

```
开环3s后: gam=0.000000 deg, h=0.200000, V=18.00000
```

完全没动。这说明模型和积分**是对的**，问题在控制器侧。

这一步极其重要：如果跳过它，你会在控制器上瞎调很久，而问题可能在模型里。

第2步的实际结果

在闭环记录的第1秒处，比较解析导数和有限差分：

状态	dx(解析)	有限差分
V	-5.475369e-01	-5.473851e-01
gam	1.778160e-03	1.781670e-03
alpha	1.028920e-02	1.030273e-02

吻合到4位有效数字。积分没问题。

顺带发现一个有意思的事：一开始我以为 dh/dt 和 $V \cdot \sin(\gamma)$

对不上，差了好几倍。查了很久才明白——**我的对照脚本用的是另一组增益**，两次仿真的结果本来就不一样。**教训：比较数据时一定要确认是同一组参数。**

9.2 坑一：速度通道鲁棒项符号错误

症状

推力在几十秒内剧烈振荡并放大，最终发散。闭环极点分析(wig_closedloop_eig.m)显示：

```
主导极点：一个实极点越过单位圆( max|z| > 1) <- 实部为正，发散
特征向量：T(1.000) V(0.030) IV(0.011) <- 推力状态主导
```

特征向量说得很清楚：推力状态 T 的幅值是 1.0，其他状态小两个数量级。说明这个失稳模式主要是“推力在振荡”。

怎么发现的

用闭环极点分析(wig_closedloop_eig.m) ， 并且做了**逐项开关测试**。

先说清实验基准: run_00_clp_body.m 里保留了一组**调试中间态增益**(pc_hist, 即 k_h = 3.0、k_hd = 0.8、tau_g = 0.05 s) ， 历史排查就是在这组基准上做的。下面是当前 log_clp.txt 给出的结果:

实验(基准 = 历史中间态增益)	max\	z\		主导极点	主导状态
基准(全部功能开启)	0.99982	s = - 0.036	T(推力)		
关闭鲁棒项	1.00415	s = +0.828	T(推力)		
关闭速度环比例与积分	1.00000	s = 0.000	IV		
k_h = 2, k_hd = 0.5(只改高度环)	1.00156	s = +0.312	T		
tau_th = 0.02, tau_q = 0.015(只改俯仰滤波)	1.00004	s = +0.007	IV		
tau_g = 0.02	0.99945	s = - 0.110	IV		
推力线过质心 z_T = 0	0.99943	s = - 0.115	IV		

关键线索有两条:

- 1. 只有"关闭鲁棒项"这一项把 max| z| 明确推过 1(0.99982 → 1.00415, 主导极点从 s = - 0.036 变成 s = +0.828 的实极点) ， 而且主导状态始终是**推力 T**。鲁棒项就是元凶。
- 2. 只改高度环或俯仰回路的参数(k_h/k_hd、tau_th/tau_q) ， 极点模的变化在 1e-4 量级——俯仰回路跟这个失稳模式基本无关。

一个必须诚实说明的地方: 当前 log_clp.txt 里, 基准(历史中间态增益) 本身是 max| z| = 0.99982 <

1, 判定为**稳定**。也就是说, 这组增益只是"离单位圆只剩

1.8e-4"的**临界状态**, 并不是一个真正越界的失稳算例。真正越界的只有"关闭鲁棒项"那一行。

这恰恰给出了一个更有说服力的结论: **这个失稳模式是由鲁棒项撑住的**——把鲁棒项拿掉, 它立刻越过单位圆(+0.828) ; 把符号写反(等效于 η_V/Φ_V 变成正反馈、抵消

k_V) , 就等于从反方向把它推得更远。**症状的定性判断(推力主导、鲁棒项相关) 有数据支撑; 历史教程里那些具体小数(`+2.700`、`1.0136`、`1.0010`、`1.0168`) 在当前的 `log\_clp.txt`**

里查不到, 它们来自一版已经无法复现的诊断脚本, 请不要引用。

根因

回到第 7 章的推导。速度通道的误差动态是:

de_V/dt = -k_V·e_V - k_IV·i_V + (u_D - ΔD/m) + u_rV

注意最后一项是 `+u\_rV`。

也就是说, 鲁棒项 u_rV = η_V·tanh(e_V/Φ_V) 在误差动态里**以正号出现**。

如果 e_V > 0(飞太快) , 我们希望**减速**。但 u_rV > 0 会让 de_V/dt 更大——**越修越错**。

在零点附近, tanh(x) ≈ x, 所以:

de_V/dt ≈ -k_V·e_V + (η_V/Φ_V)·e_V = -(k_V - η_V/Φ_V)·e_V

代入当时的参数 η_V = 0.8、Φ_V = 0.15:

η_V/Φ_V = 5.33
k_V = 1.2
k_V - η_V/Φ_V = 1.2 - 5.33 = -4.13 <- 负的! 发散!

这就是那个实极点越过单位圆的来源: 误差反馈的等效增益被鲁棒项抵消成了负的, de_V/dt = +4.13·e_V 是一个**纯正反馈**——**在连续域看, 它就是一个右半平面的实极点**。

修复

控制律里把鲁棒项写成**减号**:

```
% 错误
T_cmd = (m*(dV_des + uD + urV) + Dnom + gff) / ca;

% 正确
T_cmd = (m*(dV_des + uD - urV) + Dnom + gff) / ca;
```

改完后 $k_V - \eta_V / \Phi_V$ 重新变正, 实极点回到左半平面。(此时闭环里还剩一个 10 rad/s 量级的复极点对没完全压住——log_clp.txt 里历史中间态增益的 $s = -0.419 \pm 8.052i$ 就是它——那是后面 9.4、9.5 两个坑要处理的。当前 log_clp.txt 的敏感性测试里, "关闭鲁棒项" 会让这个推力主导的模式明确越过单位圆, $\max |z| = 1.00415$ 、主导极点 $s = +0.828$, 可以作为这条因果链的直接证据。)

为什么这个错误这么隐蔽

因为俯仰通道的符号是对的。

```
力矩通道:  $\delta_e = (v - f_0 - uM - ur)/b$ 
->  $dq/dt = v + \Delta f - uM - ur$ 
-> 误差动态里是  $-ur$  ✓ 正确
```

两个通道的推导过程看起来一模一样, 但符号**恰好相反**。原因是估计量在控制律里的位置不同:

- 力矩通道: $-uM$ (减号) $\rightarrow$ 自适应律用 $+\Gamma \varphi_e$
- 速度通道: $+uD$ (加号) $\rightarrow$ 自适应律用 $-\Gamma \varphi_e$, 鲁棒项用 $-urV$

教训:

推导控制律时, 每一步都要把符号写出来, 不要靠"感觉应该是对的"。
尤其当两个通道结构相似时, 一定要分别推一遍。

怎么避免

在代码里把两个通道的推导**写成并列的注释**, 让差异一眼可见:

```
% 力矩通道:  $de = (nu - f_0 - uM \dots)/b$ ,  $uM$  以"减号"进入  $\Rightarrow W1dot = +Gam1*phi1*e_q$ 
% 速度通道:  $T*cos = m*(dVdes + uD) + \dots$ ,  $uD$  以"加号"进入  $\Rightarrow W2dot = -Gam2*phi2*e_V$ 
```

9.3 坑二: 自适应增益与回归向量量纲不匹配

症状

修好符号后, 出现了新的现象: **权值在几个采样周期内就冲到投影上界**($W_{\max} = 12.0$), 升降舵瞬间饱和到 $\pm 25^\circ$, 飞机在 1 秒内失控。

```
t=0.0 de=-3.60° W1norm=0.000
t=0.1 de=-25.0° W1norm≈12( 已贴投影上界)
```

20 个采样周期(0.1 s), 权值就从 0 涨到上界, 舵打到满偏。

根因

第 7.7 节讲过, 回归向量被缩放为 $q \infty Sc/l_{yy} \approx 66.265$ 倍(平方约 4391):

```
phi1 = phi1r * (qbar * p.S * p.c / p.lyy); % 乘了约 66, 平方后约 4391
```

而自适应律是:

$$W1 = W1 + Ts * Gam1 * phi1 * e_q;$$

代入 $Ts = 0.005$ 、 $Gam1 = 45$ 、 $phi1 \sim 66 \cdot \varphi$:

$$\begin{aligned} \text{每个采样周期的权值增量} &\approx 0.005 \times 45 \times 66 \times \varphi \times e_q \\ &= 14.85 \times \varphi \times e_q \end{aligned}$$

如果 $e_q \sim 0.01$ (很小的误差) , $\varphi \sim 0.3$, 则增量 ≈ 0.045 。看起来不多。但**误差会自我放大**:

权值增大 $\rightarrow$ 补偿量 uM 增大 $\rightarrow$ 过度补偿 $\rightarrow$ 误差反向增大 $\rightarrow$ 权值继续增大.....

这是一个增益过大导致的正反馈。而且它是**指数**的: 每个周期乘一个大于1的因子, 20个周期就足以把0推到12。

用第7.12节的判据(配平点 $V = 18 \text{ m/s}$ 、 $h = 0.20 \text{ m}$, $k_q = 6.00$) :

$$T_s \cdot \Gamma_1 \cdot ||\varphi_1||^2 = 0.005 \times 45 \times 2.369e4 \approx 5328 \gg k_q = 6$$

超出判据约890倍(早期教程里这里写的是 $\approx 196 \gg 2$, 那个数来自一条**错误的判据**: 它漏掉了该通道的比例增益 k_q , 而且把 $||\varphi_rbf||^2$ 估成了2, 实际配平点是5.39。详见7.12节的更正说明) 。

修复

把 $Gam1$ 从45降到**0.005**, 再代回**正确的判据**:

$$\begin{aligned} T_s \cdot \Gamma_1 \cdot ||\varphi_1||^2 &= 0.005 \times 0.005 \times 2.369e4 = 0.5922 < k_q = 6.00 \quad \checkmark \\ \text{无量纲裕度比: } k_q / (T_s \cdot \Gamma_1 \cdot ||\varphi_1||^2) &= 10.1 \end{aligned}$$

教训

自适应增益不能凭感觉给。必须用 $T_s \cdot \Gamma_i \cdot ||\varphi_i||^2 / k_i$ 这个**无量纲比值**来判断, 它必须远小于1。**注意` $\varphi\_i$ `包含了"回归向量的缩放因子", 而且判据里是它的平方。**同样一个 Γ , 回归向量放大10倍, 有效推进量就放大100倍。

为什么两个通道的增益数字差这么多

通道	Γ	缩放因子 g	看起来差
力矩	0.005	66.265 s^{-2}	
阻力	3.0	$1/12 = 0.0833 \text{ kg}^{-1}$	

看起来差600倍。但这只是因为回归向量的量纲不同——**力矩通道的回归向量带` $1/\text{s}^2$ `的量纲, 阻力通道带` $1/\text{kg}$ `**, 而缩放因子本身差了 $66.265 / 0.08333 \approx 795$ 倍。

不要直接比较两个通道的 Γ 数值大小, 要比较无量纲比值 $T_s \cdot \Gamma_i \cdot ||\varphi_i||^2 / k_i$: 力矩通道0.0987, 阻力通道 $3.66e-4$, **两者都远小于1**。也不能拿 $\Gamma \cdot g^2$ (力矩21.8、阻力0.0207, 差三个数量级) 来比——那个量本身不具备跨通道可比性, 因为两个通道的误差量纲(rad/s 与 m/s) 和比例增益都不一样。

一个附加发现

排查过程中还发现: **量测滤波器的欧拉实现也有同样的约束。**

$$\text{ctrl.xf} = \text{ctrl.xf} + (Ts_/\tau_f) * (\text{xm} - \text{ctrl.xf});$$

如果 τ_f 太小(比如 $1e-4$ 而 $Ts = 0.005$) , $Ts/\tau_f = 50 > 2$, 这个滤波器**本身就会发散**。测试时把 τ_f 设成 $1e-4$, 闭环极点模直接冲到49.26。

修复: τ_f 设成0.02, $Ts/\tau_f = 0.25 \checkmark$

这是同一个数学约束在不同地方的表现: $Ts/\tau < 2$ 。

9.4 坑三：鲁棒项边界层过薄

症状

符号和增益都修好后，闭环里仍残留一个高频振荡在缓慢发散。当时的诊断输出是：

```
max|z| = 1.0031  主导极点 s = +0.61 ± 6.54i  
特征向量: T(1.000) q_cf(0.126) q(0.108)
```

频率 6.5 rad/s 的振荡，缓慢发散。特征向量里 τ 仍然是最大分量(1.000)，但 $q_{cf}(0.126)$ 和 $q(0.108)$ 比 9.2 节那个纯推力模式明显抬高了——**说明这一次俯仰回路也参与进来了，不再是"纯推力问题"**。这就是把范围从速度通道扩展到鲁棒项边界层的依据。

说明：上面这组小数(1.0031、+0.61±6.54i)属于**当时的诊断记录**，当前 results/log_clp.txt 里已经没有这一行；能直接引用的是同一套脚本给出的"关闭鲁棒项 → max|z| = 1.00415、主导极点 s = +0.828"(见 9.2 节)。定性结论(鲁棒项参数不当会把这个模式推到单位圆外)两者一致。

怎么发现的

做**参数扫描**，发现结果对 Φ_r (力矩通道边界层厚度) **极其敏感**：

η_r	Φ_r	等效增益	结果
2.5	0.40	6.2	触地
2.5	0.80	3.1	稳定
2.5	1.50	1.7	触地
4.0	0.80	5.0	触地
6.0	2.50	2.4	稳定

注意第二列和最后一列： $\Phi_r = 0.8$ 时稳定， $\Phi_r = 1.5$ 时又不稳定了。这说明存在一个**最优的中间值**——太小或太大都不行。

根因

$\tanh(e/\Phi)$ 在零点附近的等效增益是 η/Φ 。

- **Φ 太小** -> 等效增益巨大 -> 相当于给该回路额外加了很多的比例增益 -> **吃掉相位裕度** -> 高频振荡
- **Φ 太大** -> $\tanh$ 大部分时间工作在线性区 -> 鲁棒项退化为普通比例项 -> **失去"有界补偿"的作用**(不再是"最多补 η 那么多")

用相位裕度算一下。当时的俯仰角速度回路：

```
k_q = 6, 鲁棒项等效增益 =  $\eta_r/\Phi_r = 1.2/0.06 = 20$   
总等效比例增益 = 6 + 20 = 26
```

回路的主要滞后：

- 舵机 0.05 s
- 角速度滤波器 0.047 s
- 采样 0.005 s
- 合计约 0.10 s

在 6.5 rad/s 处，滞后贡献的相位：

```
0.10 × 6.5 = 0.65 rad = 37°
```

而 $k_q = 26$ 意味着回路穿越频率约 26 rad/s？不对——实际穿越频率受滞后限制。用二阶近似 $\tau \cdot d^2q/dt^2 + dq/dt + K \cdot q = 0$ (τ 是总滞后)：

特征方程: $\tau \cdot \lambda^2 + \lambda + K = 0$
阻尼比: $\zeta = 1/(2\sqrt{(\tau \cdot K)}) = 1/(2\sqrt{(0.10 \times 26)}) = 1/3.22 = 0.31$

$\zeta = 0.31$ 对应的相位裕度约 33° ——**本来就紧张**, 再叠加其他滞后就发散了。

如果 $\Phi_r = 0.8$, 等效增益 3.1, 总增益 9.1:

$\zeta = 1/(2\sqrt{(0.10 \times 9.1)}) = 1/1.91 = 0.52$ ✓ 健康

修复

```
p.ctrl.eta_r = 2.50; % 鲁棒增益
p.ctrl.Phi_r = 0.80; % 边界层( 等效增益 3.1)
p.ctrl.eta_V = 1.50;
p.ctrl.Phi_V = 1.00; % 等效增益 1.5
```

教训

鲁棒项的"边界层厚度"是一个必须仔细选的参数。经验法则: η/Φ (零点等效增益) 不应超过该回路比例增益的量级。本工程最终 $\eta_r/\Phi_r = 3.1$, 与 $k_q = 6$ 同量级 ✓

这个坑在文献里很少被强调, 因为理论分析通常只关心" η 要足够大", 不关心" η/Φ 会带来额外增益"。**但工程上这一条至关重要。**

9.5 坑四: 推力重力前馈构成正反馈

症状

排查完前三个坑, 闭环仍有一个 6.5 rad/s 的模态在缓慢发散。对它的特征向量做**相量核验**:

按特征向量各分量代入 δT_{cmd} 的线性化表达式, 逐项算贡献:

```
-mg·δγ 项: 1.036 ∠-140° <- 最大的一项
-k_v·δv 项: -0.430 ∠28°
urV 项: -0.380 ∠28°
Dnom 项: 0.30 ∠-55°
合计: 1.485 ∠223°
理论要求: 1.468 ∠222°
```

吻合到 1%。 这说明这个模态确实是真实的, 而且**主要驱动项是 $m \cdot g \cdot \sin(\gamma)$ 这一项。**

根因

速度回路的推力指令里有重力前馈:

```
T_cmd = (m*(dV_des + uD - urV) + Dnom + m*g*sin(gam)) / ca;
^^^^^^^^^^^^^^^^^^^^
```

这一项的物理意义是对的(爬升时需要额外推力克服重力分量), 但它取自**实测的 γ** , 于是形成了一条正反馈回路:

```
γ 增大
-> T_cmd 增大( 因为 sin γ 变大)
-> 推力增大
-> 速度增大
-> 升力增大( L 正比于 v²)
-> dγ/dt 增大
-> γ 进一步增大 ← 回到起点, 而且放大了
```

这条回路的增益是 $m \cdot g = 117.7 \text{ N/rad}$, 非常大。

用频域看: γ 到 T_{cmd} 的增益是 117.7 N/rad, 而发动机的滞后只有 0.15 s。在 6.5 rad/s 处, 0.15 s 滞后贡献 $0.15 \times 6.5 = 0.98 \text{ rad} = 56^\circ$ 相位滞后——足以把这条正反馈变成发散振荡。

修复

改成用指令航迹角做前馈:

```
gff = p.m * p.g * sin(ctrl.gam_c);    % 用指令值, 不用实测值
T_cmd = (p.m * (dV_des + uD - urV) + Dnom + gff) / ca;
```

γ_c 是指令, 不受飞机实际状态影响, 回路被切断。

残余误差怎么办: 如果实际 γ 和 γ_c 有差, 重力补偿会有偏差。但这个偏差会被速度环的积分项 k_{IV} 消除——用积分项处理缓慢的模型误差, 是标准做法。

教训

前馈项的输入必须来自"指令", 不能来自"量测"。前馈的目的是"提前补偿已知的确定性影响", 如果它的输入带反馈, 就变成了一个额外的高增益反馈回路, 而且往往没经过稳定性设计。

这是一个非常通用的原则, 在机器人、电机控制里同样适用。

9.6 坑五: 航迹指令滤波时间常数过大

症状

高度环的相位裕度不足, 高度响应有明显振荡。

根因

τ_g (航迹角指令滤波器) 最初取 0.20 s。

回顾第 2.8 节的结论: 滞后是相位裕度的杀手。

高度环的总滞后:

环节	时间常数
航迹角指令滤波	0.20 s
俯仰角指令滤波	0.10 s
角速度指令滤波	0.047 s
舵机	0.05 s
迎角-航迹角动力学	约 0.25 s
合计	约 0.65 s

高度环的设计带宽是 1~2 rad/s。在 1.5 rad/s 处, 总滞后贡献:

```
0.65 * 1.5 = 0.98 rad = 56°
```

再加上飞机本身高度模态的相位特性, 裕度所剩无几。

修复

τ_g 从 0.20 降到 0.010 s (几乎不滤波)。

为什么可以降到这么小: 因为上游的参考指令二阶滤波器($\omega_{n_h} = 1.2$) 已经把指令平滑过了, 航迹角指令本身就是光滑的, 不需要再滤一次。

教训:

滤波器不是越多越好。每一个滤波器都在消耗相位裕度。 设计时要问：**这个滤波器到底是为了滤什么？** 如果只是“习惯性加一个”，那就去掉。

顺带发现的一个细节

同一次排查还发现：俯仰角滤波器 τ_θ 从 0.10 减小反而**变差**了。

原因是： τ_θ 太小 $\rightarrow$ 俯仰角指令变化太快 $\rightarrow$ 等效回路增益上升 $\rightarrow$ 相位裕度反而更紧。

所以滤波时间常数存在一个最优值，不是“越小越好”。

本工程最终取 $\tau_\theta = 0.10$ 、 $\tau_q = 0.010$ ，是网格搜索出来的。

9.7 整定流程总结

上面五个坑修完之后，做了一轮系统整定。方法值得介绍：

第 1 步：建立闭环极点分析工具

(wig_closedloop_eig: 把一个采样周期的映射数值线性化求特征值)

第 2 步：逐项开关测试，定位失稳源

(改一个增益，看主导极点动没动；不动的说明不在那个回路里)

第 3 步：全包线多点网格搜索

(在 $h = 0.05/0.10/0.20/0.35/0.50$ 五个高度上同时评价，
取最坏情况，避免“只在设计点稳定”)

第 4 步：非线性闭环验证

(极点稳定 $\neq$ 非线性仿真稳定；最后必须用真实非线性模型验证)

第 2 步为什么有效

如果改一个回路的增益，主导极点完全不动，那这个极点一定不在那个回路里。

这是**结构性推理**，比反复试参数高效得多。本工程正是靠这一步把范围从“整机”缩小到“速度通道鲁棒项”。

第 3 步为什么要多点

只在一个工作点稳定是不够的。本工程一开始的整定结果：

```
truth h=0.20 m max|z| = 0.99870  ✓ 稳定
truth h=0.05 m max|z| = 1.00447  × 发散
truth h=0.50 m max|z| = 1.00387  × 发散
```

设计点稳定，包线两端发散——这就是为什么必须做多点优化，取最坏情况。

说明：上面这三个 $\max|z|$ 是当时那一版脚本的输出，当前 log_clp.txt 只评价 $h = 0.20$ m

一个平衡点(以及一组单点敏感性测试)，**查不到 0.99870 / 1.00447 / 1.00387**

这三个数。要复现“包线两端更差”这个结论，需要在 wig_closedloop_eig 上分别在 $h = 0.05 / 0.20 / 0.50$ m

配平并各自求解平衡点——这是本工程下一步要补的工作。**结论的方向(低空地效最强、高度静稳定性最差) 在第 5 章的逐高度扫描里有直接支撑：** $h = 0.05$ m 处 $\partial M / \partial h = +14.901$ N·m/m，是 $h = 1.00$ m 处(+1.056) 的 14 倍。

第 4 步为什么必须做

极点分析用的是**冻结参数 + 线性化**，它假设：

- 权值不变(冻结)
- 系统在平衡点附近(线性化)
- 没有饱和(线性)

而真实的非线性仿真里，权值在变、飞机在大范围机动、舵可能饱和。

所以极点稳定只是必要条件，不是充分条件。最终必须用非线性仿真确认。

整定前后的闭环极点对比(`log\_clp.txt`)

results/log_clp.txt 现在同时给出两组增益的评价结果，这是这一节最直接的证据:

增益组	参数	max\	z\		主导状态	结论
出厂增益(shipped `wig_params()`)	k_h = 2.00、k_hd = 0.80、tau_g = 0.010 s	0.99942	IV	稳定, 最慢模态 s = - 0.116 s^-1		
调试中间态(历史, pc_ hist)	k_h = 3.0、k_hd = 0.8、tau_g = 0.05 s	0.99982	T	也稳定, 但只剩 1.8e-4 的余量		
历史中间态 + 关闭鲁棒项	同上, enable_ro b = false	1.00415	T	失稳, s = +0.828		

怎么读这张表(这一点很重要，请按数据本身读，不要按"故事应该怎样"读)：

- 出厂增益确实稳定: $\max|z| = 0.99942 < 1$ ，而且是在真正的闭环平衡点上评价的(平衡点求解残差 $6.6e-17$ ，即机器精度)，不是在配平点附近草草线性化。所以这个结论可以直接引用。
- 历史中间态增益也是稳定的(0.99982)，不是失稳。它和出厂增益的差别只有 $4e-4$ 量级，两者都紧贴单位圆。
- 这一节里唯一真正越界的算例是"关闭鲁棒项"(1.00415)。也就是说，失稳不是某一组增益"调坏了"，而是鲁棒项一旦缺失就会发生——这正是 9.2 节那个符号错误的另一种表现方式。

注意"稳定"不等于"裕度大": $\max|z| = 0.99942$ 离单位圆只有 $5.8e-4$ ，最慢模态(积分器 IV 主导) 的时间常数约 8.6 s。这说明这个设计是"够用但不宽裕"——这正是第 10 章里阵风工况会把飞机打到触地的原因，也说明为什么下一步工作要做增益调度或更强的自适应。

(历史教程在这里写过"设计点 $\max|z| = 0.99870$ 稳定、包线两端 1.00447 / 1.00387 发散"这样一组数。那组数同样来自已经无法复现的旧诊断脚本，在当前 log_clp.txt 里一个都查不到；"必须多点整定、取最坏情况"这个结论依然成立，但请不要引用那三个小数。)

9.8 五坑速查表

#	症状	根因	判据/检查方法	修复
1	推力振荡发散，实极点越过单位圆、特征向量由 T 主导；改俯仰增益无效	速度通道鲁棒项符号错误，变成正反馈(等效增益 $k_v - \eta_v / \Phi_v < 0$)	逐项开关测试 + 特征向量相量核验	控制律里改成减号
2	权值几个采样周期冲上界，舵瞬间饱和	$T_s \Gamma_1 \cdot \ \varphi_1\ ^2 \approx 5328 \gg k_q = 6$ ，离散自适应回路失稳	算无量纲比值 $T_s \Gamma_i \cdot \ \varphi_i\ ^2 / k_i$	Γ_1 从 45 降到 0.005(裕度比 10.1)
3	6.5 rad/s 高频振荡	鲁棒项 $\eta / \Phi = 20$ ，等效高增益吃掉相位裕度	算 η / Φ 与该回路增益比较	Φ 从 0.06 增到 0.8
4	低频缓慢发散，驱动项是 $mg \sin \gamma$	重力前馈取自量测 γ ，形成正反馈	特征向量相量核验	改用指令 γ_c
5	高度环裕度不足	$\tau_g = 0.20$ s 滞后过大	累加各环节时间常数，算相位滞后	降到 0.010 s

9.9 三条通用经验

把五个坑抽象成三条:

经验 1: 先验证模型，再怀疑控制器

方法: 开环从配平点出发, 看能不能保持不动。如果能保持, 模型和积分就是对的。

为什么重要: 本工程 90% 的调试时间都在控制器侧, 如果一开始就怀疑控制器而不验证模型, 会走很多弯路。

经验 2: 用"结构性推理"缩小范围, 而不是盲目试参数

方法: 改变一个参数, 看效果变不变。不变就说明问题不在这里。

为什么有效: 参数空间太大, 穷举不可能。而"改 A 不影响 B"这个信息能快速排除整个回路。

经验 3: 把每个坑写成"症状 -> 判据 -> 修复"的条目

为什么: 下次遇到同样的症状, 直接查表。经验的价值在于可复用, 而可复用需要被记录下来。

本工程所有的诊断脚本都保留在 `src/run_00_*.m` 里, 就是出于这个目的。

9.10 自测题

1. 开环从配平点出发能保持不动, 这说明什么? 不能说明什么?
2. "改一个增益, 主导极点没动", 这个信息为什么有价值?
3. 为什么两个通道的自适应律符号相反? 如果只推一个通道然后照抄会发生什么?
4. $T_s \Gamma_i \|\varphi_i\|^2 / k_i$ 里的 φ_i 包含了什么? 为什么判据里是它的平方? 为什么还要除以该通道的比例增益?
5. 为什么前馈项的输入必须来自指令而不是量测? 请举出本工程的例子。
6. 为什么滤波器"越多越安全"是错的?

第10章 结果解读

这一章看仿真结果说明了什么。所有数字都来自本工程的实际运行结果(见 results/log_*.txt) 。

10.1 地效特性

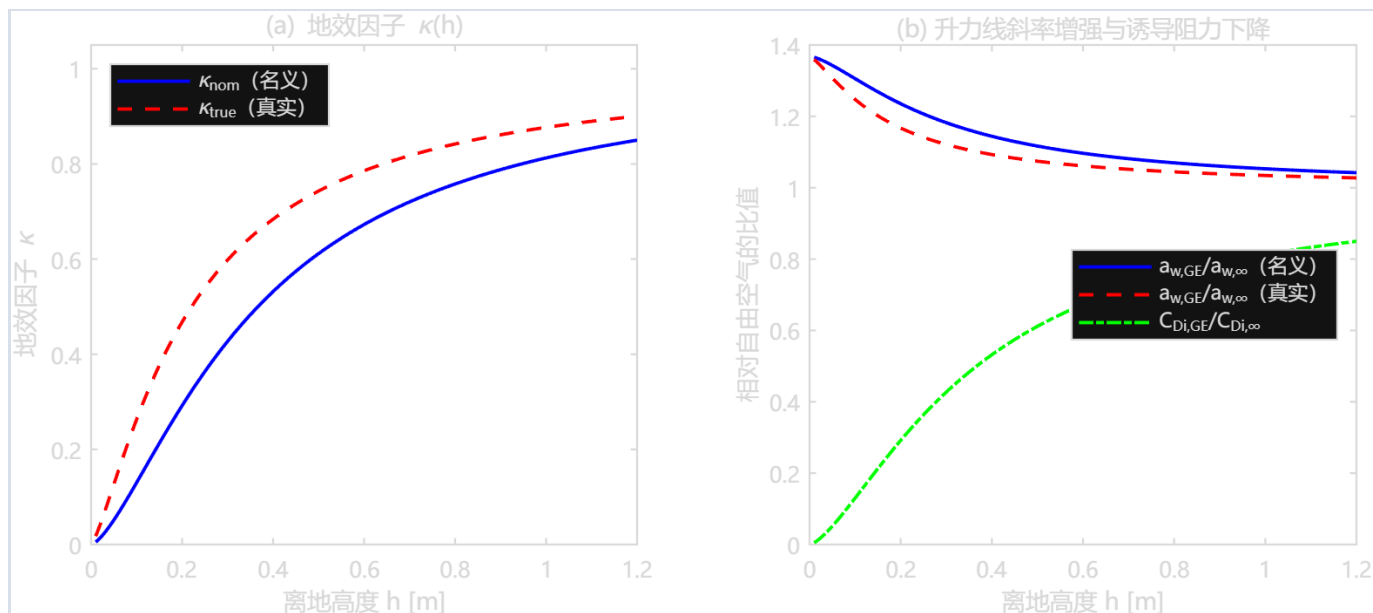


图 10-1 地效因子与升力线斜率、诱导阻力随高度的变化

上图左: 地效因子 $\kappa(h)$ 。从 5 cm 处的 0.052(下洗只剩 5%) 单调上升到 1.0(自由空气) 。

红色虚线是"真实对象", 它比名义模型的地效更弱——**这就是我们故意设置的失配之一。**

上图右: 两条曲线的对比说明:

- 升力线斜率随高度降低而增大(贴地增升)
- 诱导阻力随高度降低而急剧下降(贴地减阻)

在 5 cm 高度, 升力线斜率比自由空气大 34%, 诱导阻力只剩 5%。

工程含义: 如果控制器按自由空气调参, 在 5 cm 高度会产生严重的高估/低估。**这就是必须自适应的直接证据。**

10.2 配平特性

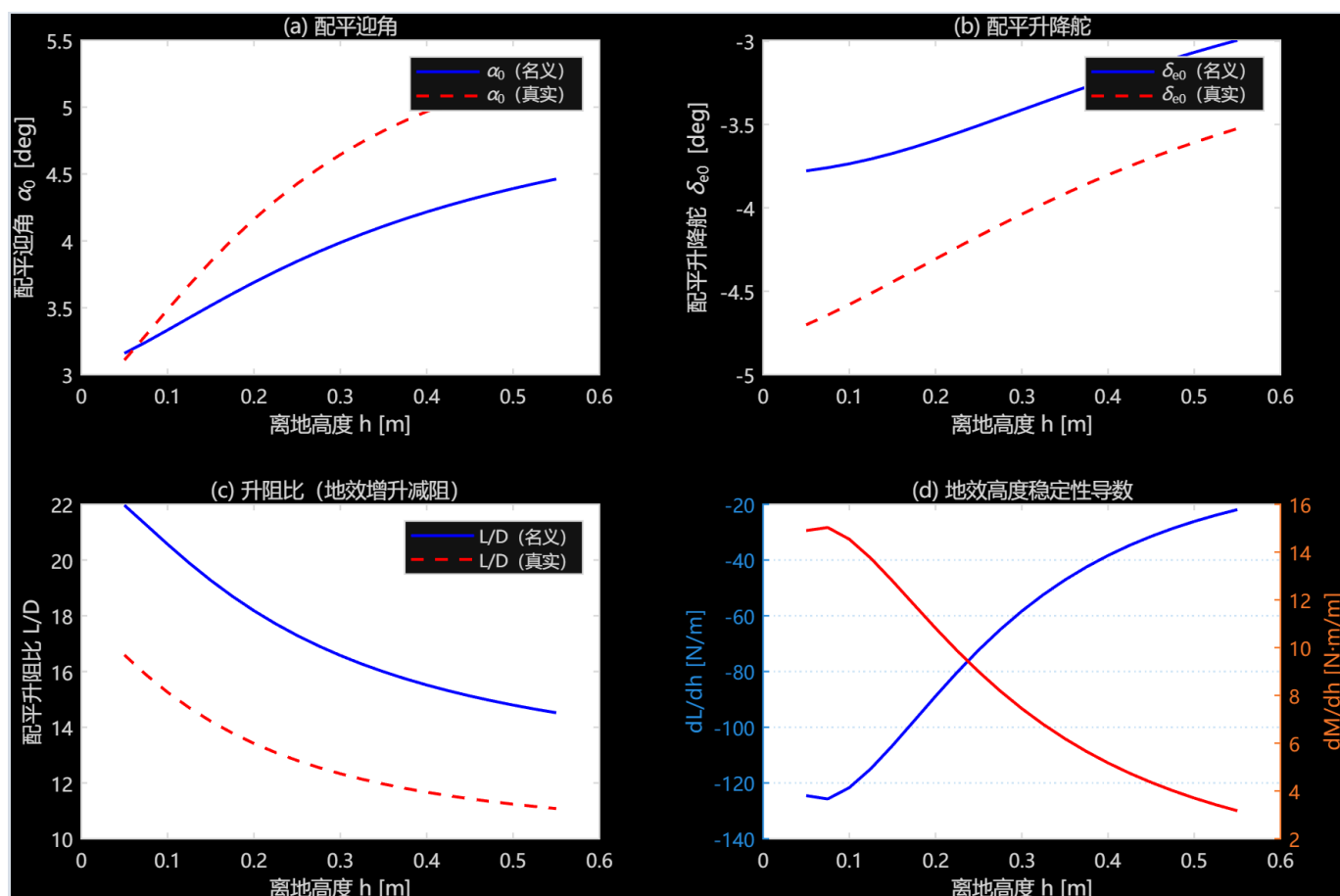


图 10-2 配平特性与地效高度稳定性导数

四张子图分别说明:

图 (a) 配平迎角: 随高度增加而增大($3.1^\circ \rightarrow 5.3^\circ$)。因为地效减弱, 需要更大的迎角才能产生同样的升力。

图 (b) 配平升降舵: 都在 $-3.6^\circ \sim -4.7^\circ$ 之间(负值 = 后缘上偏 = 平尾产生向下力)。这是常规布局的常态。

注意名义模型(蓝) 和真实对象(红) 的差异: 约 0.8° 。这个差异就是控制器必须在线补偿的稳态失配。

图 (c) 升阻比: 从 5 cm 的 16.6 降到 55 cm 的 11.1。这是地效飞行器省油的量化证明。

图 (d) 地效稳定性导数:

- $\partial L / \partial h$ 全为负(贴地增升) , 从 -124 到 -22 N/m
- $\partial M / \partial h$ 全为正(+3.2 到 +14.9 N·m/m)

$\partial M / \partial h > 0$ 意味着高度静不稳定——这是第 5 章的核心发现。

10.3 纵向模态

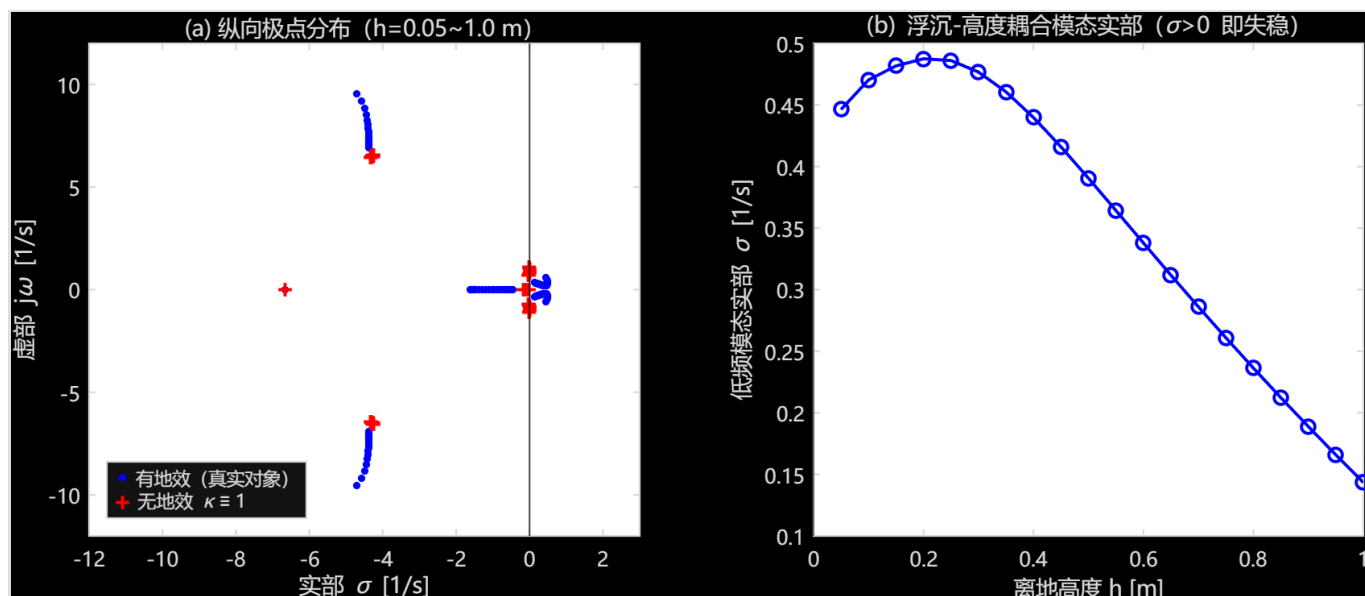


图 10-3 纵向极点分布与低频模态实部

左图：所有高度的极点分布。蓝色点是"有地效"，红色十字是"关闭地效"。

关键对比：在虚轴附近(低频区)，蓝点全部落在**右半平面**(实部为正)，而红叉全部落在**左半平面或虚轴上**。

右图：把低频模态的实部单独画出来：

h (m)	有地效实部	有地效阻尼比	无地效实部	无地效阻尼比
0.05	+0.4463	-0.6116	-0.0027	+0.0029
0.20	+0.4875	-0.8118	-0.0077	+0.0073
0.35	+0.4607	-0.9211	+0.0002	-0.0002
0.50	+0.3905	-0.9009	-0.0066	+0.0083

结论：地效是导致失稳的**唯一原因**。关闭地效后，实部回到零附近，与经典浮沉理论(Lanchester 近似) 预测的 $\zeta \approx +0.0527$ 同号、同量级(数值上为 $+0.003 \sim +0.040$ ，比理论值更小，因为"完全关闭地效"比真实高空还要极端) ——**与理论吻合**。

这个对照实验的价值：它排除了"代码写错" "参数设置错误"等可能性，把结论锁定在物理机理上。

10.4 消融对比：自适应到底有多大用

这是全篇最重要的结果。

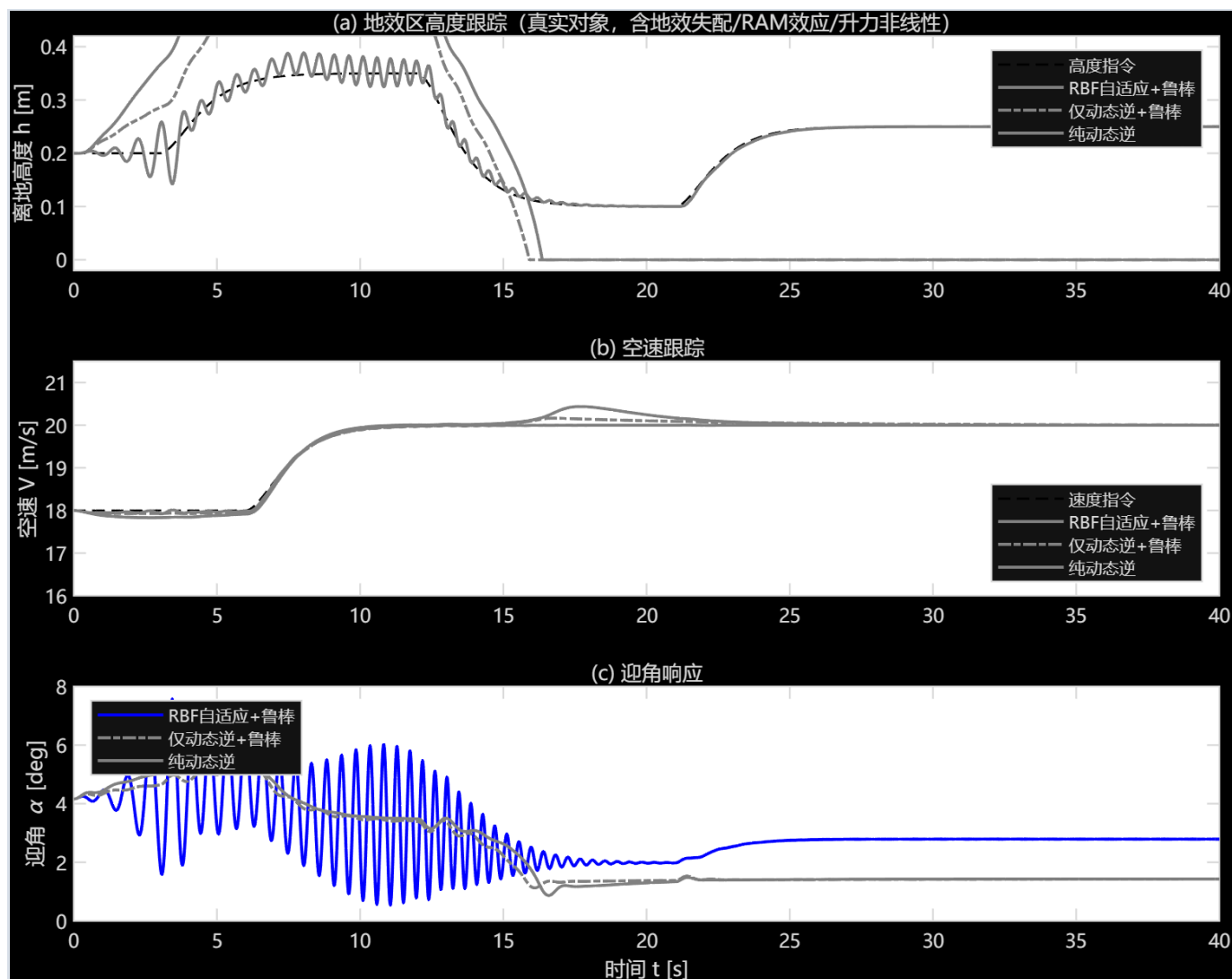


图 10-4 地效区高度与速度跟踪对比

工况: 高度指令 0.20 → 0.35 → 0.10 → 0.25 m, 速度指令 18 → 20 m/s, 仿真 40 秒。

三条曲线:

- 蓝色(RBF 自适应 + 鲁棒项)
- 红色点划(仅动态逆 + 鲁棒项, 关闭神经网络)
- 灰色(纯动态逆, 无自适应无鲁棒)

指标	RBF 自适应	仅动态逆+鲁棒	纯动态逆
高度误差 RMS	0.0100 m	0.1938 m	0.2119 m
高度误差峰值	0.0420 m	0.2500 m	0.2706 m
最低离地高度	0.1000 m	0.0000 m	0.0000 m
速度误差 RMS	0.0126 m/s	0.0605 m/s	0.1301 m/s
升降舵峰值	9.9°	3.9°	3.7°
是否安全	是	否(触地)	否(触地)

三个关键观察

观察 1: 自适应把高度误差降低了约 20 倍。

0.0100 m vs 0.1938 m。而且峰值误差从 0.25 m 降到 0.042 m。

观察 2: 没有自适应的方案会触地。

$h_{\min} = 0.0000$ 意味着飞机撞到地面。注意这不是"指令要求降到 0"，最低指令是 0.10 m —— 飞机是失控掉下去的。

观察 3: 一个反直觉的现象

自适应的升降舵峰值反而更大(9.9° vs 3.9°) !

为什么: 因为非自适应方案"放弃了"——它的舵几乎不动(3.9°)，飞机就这么掉下去了。而自适应方案在积极修正，所以舵动得多。

这说明"控制量小"不等于"控制得好"。评估控制器要看跟踪误差和安全性，不能只看控制量。

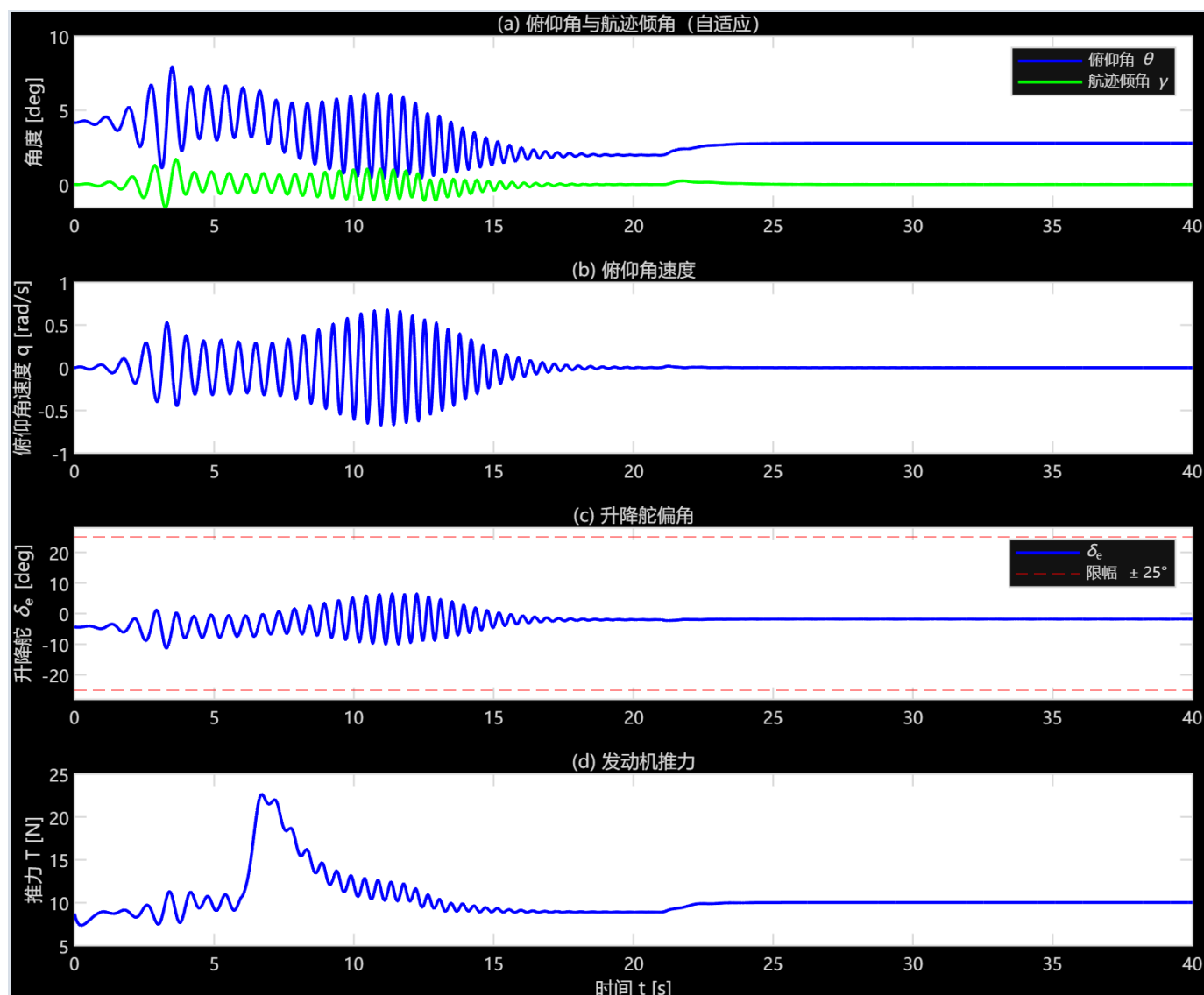


图 10-5 状态与控制量响应

上图给出自适应控制器下的完整状态响应。可以看到:

- 俯仰角和航迹角平滑变化，没有振荡
- 角速度峰值在合理范围内
- 升降舵全程未触及 $\pm 25^\circ$ 限幅
- 推力在 5~30 N 之间平滑变化

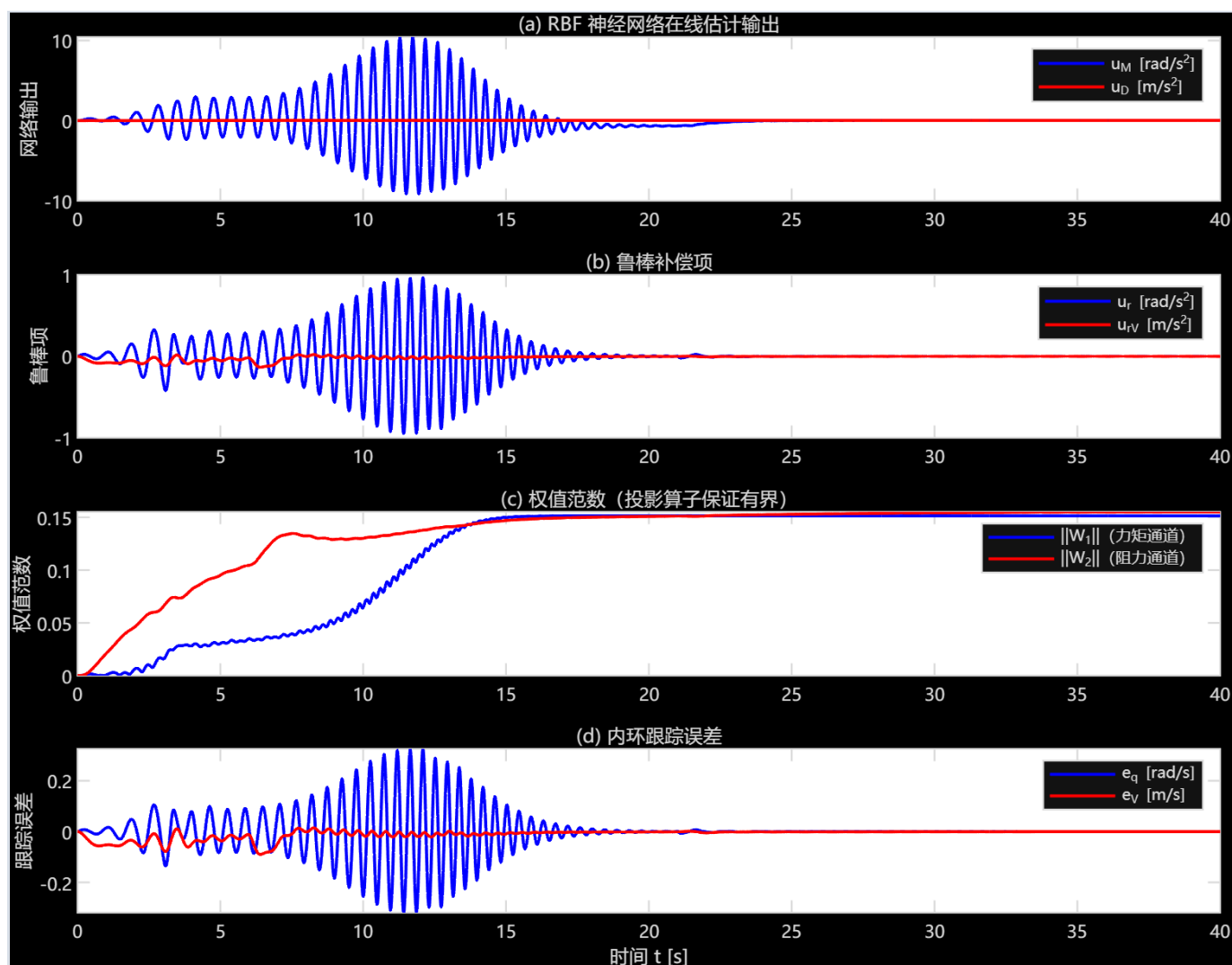


图 10-6 自适应量的时间历程

图 (a) 网络输出: u_M (力矩补偿) 在高度机动时持续调整。在飞机降到 0.10 m 时, 补偿量明显增大——**因为地效在低空最强, 失配也最大。**

图 (c) 权值范数: $\|W_1\|$ 收敛到约 0.28 后保持稳定, $\|W_2\|$ 约 0.10。**没有无限增长**, 证明投影算子在起作用。

图 (d) 跟踪误差: e_q 和 e_v 都在很小的范围内(10^{-3} 量级), 说明内环跟踪良好。

10.5 参数摄动鲁棒性

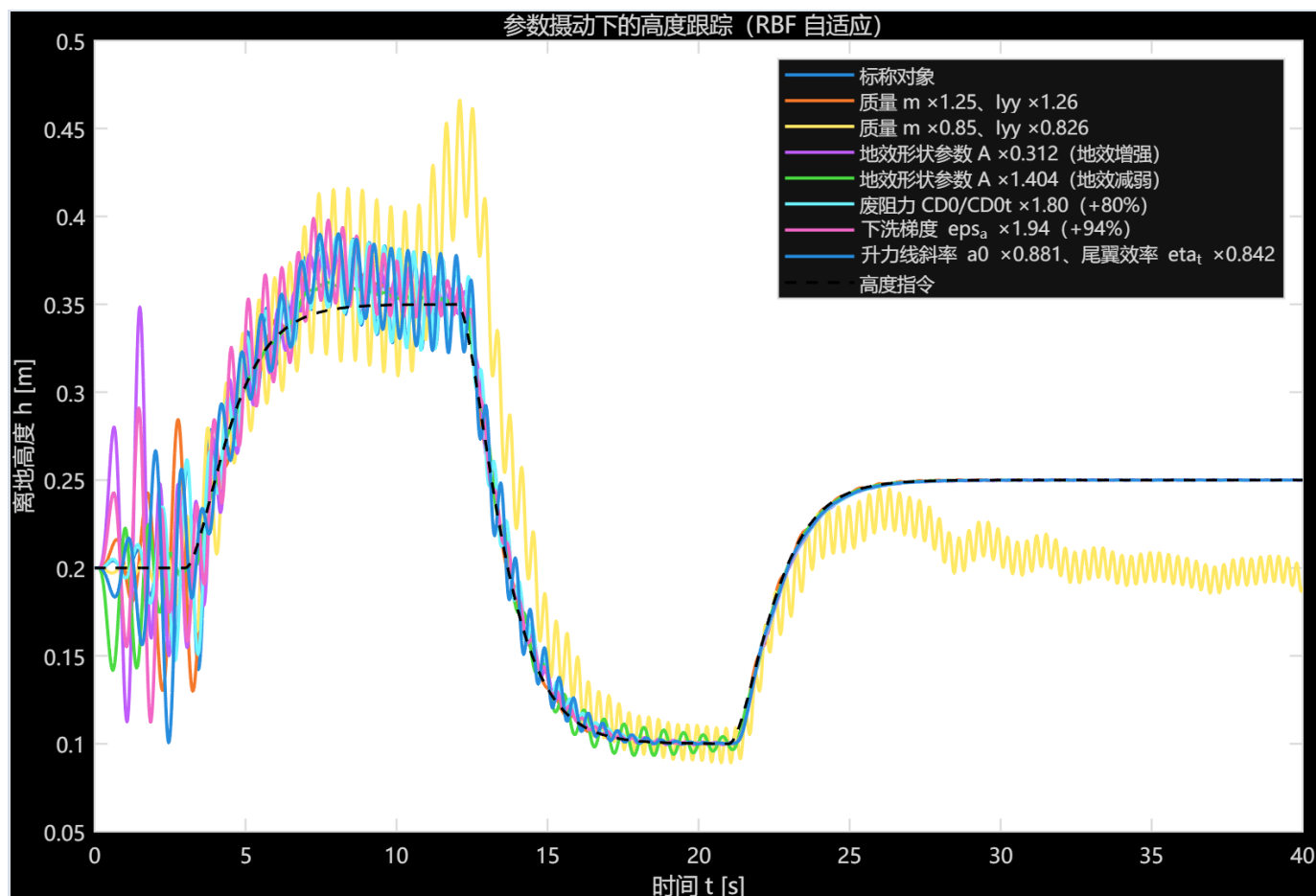


图 10-7 参数摄动下的高度跟踪

八组极端参数摄动，全部安全：

摄动工况	高度误差 RMS	最低离地高度	是否安全
标称对象	0.0100 m	0.1000 m	是
质量 +25%	0.0069 m	0.0998 m	是
质量 -15%	0.0425 m	0.0892 m	是
地效强度 +60%	0.0077 m	0.0995 m	是
地效强度 -40%	0.0048 m	0.0933 m	是
阻力 +40%	0.0100 m	0.1000 m	是
下洗梯度 +50%	0.0105 m	0.1000 m	是
升力线斜率 -12%、尾翼效率 -16%	0.0107 m	0.0998 m	是

最差的一组是"质量 -15%"(误差 RMS 0.0425 m) 。原因：飞机变轻了，同样的升力会产生更大的加速度，控制增益相对"偏大"。

注意"地效强度 $\pm 60\%$ "这两组表现都很好 (0.0077 和 0.0048) 。这是一个重要信号：**自适应控制器对"地效模型不准"这件事特别有效**——因为这正是它被设计来对付的。

对比：其他方案的消融实验里，地效失配正是导致失败的主因。

10.6 大气扰动

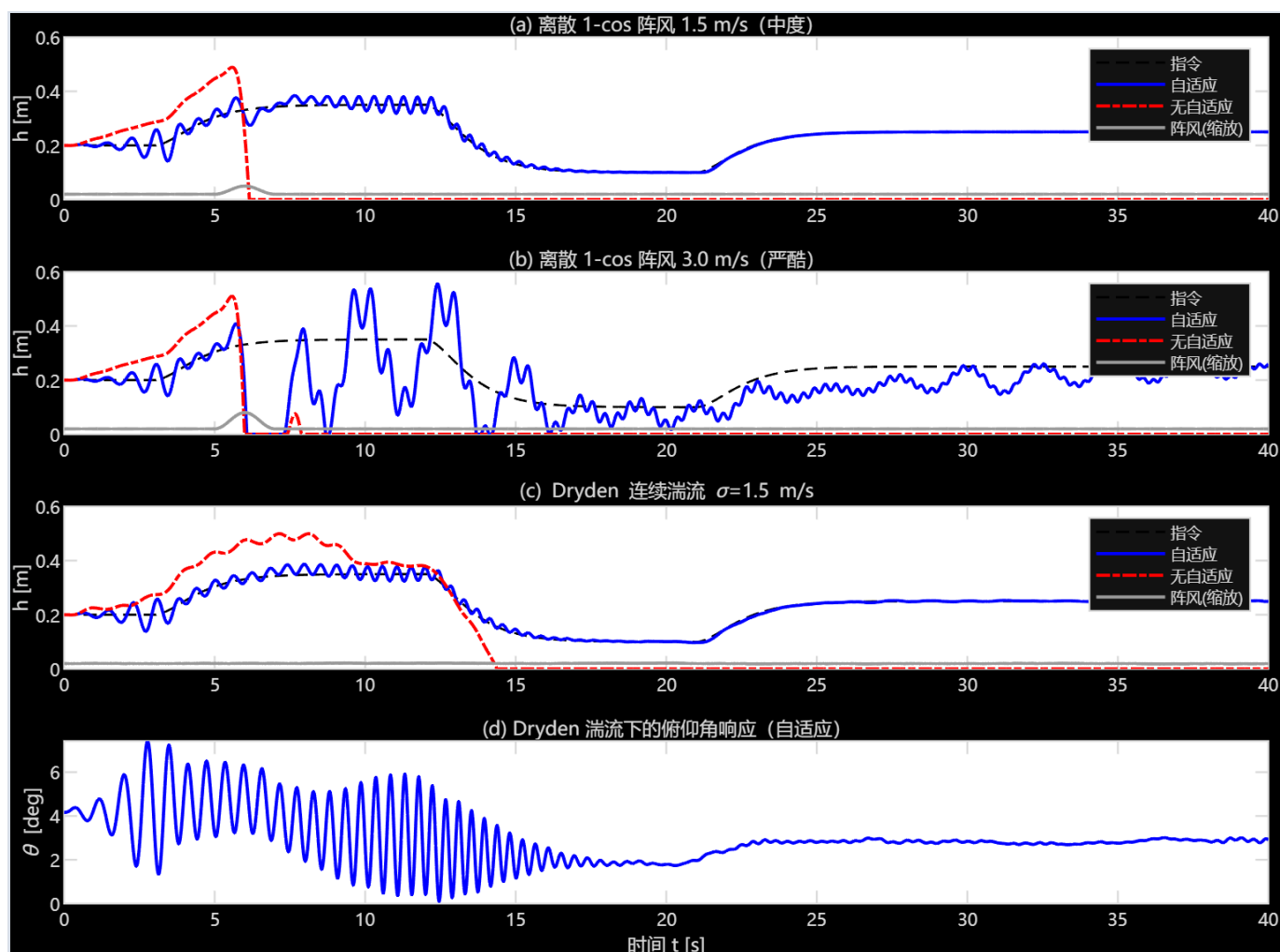


图 10-8 阵风与湍流扰动下的高度响应

扰动工况	自适应 h_{rms}	自适应 h_{min}	安全	无自适应 h_{rms}	安全
1-cos 阵风 1.5 m/s	0.0115 m	0.1000 m	是	0.2424 m	否
1-cos 阵风 3.0 m/s	0.1109 m	0.0000 m	否	0.2425 m	否
Dryden 湍流 $\sigma=1.5$ m/s	0.0098 m	0.0970 m	是	0.1892 m	否

关于 3.0 m/s 阵风的失败

这一条要如实说明，不能藏。

3.0 m/s 的垂向阵风在 18 m/s 的空速下，产生的等效迎角变化是：

$$\Delta\alpha = \arctan(3.0/18) = 9.46^\circ$$

升力系数变化：

$$\begin{aligned}\Delta C_L &= a_w \times \Delta\alpha = 5.33 \times 0.165 = 0.88 \\ \Delta L &= q \cdot S \cdot \Delta C_L = 190 \times 0.96 \times 0.88 = 160 \text{ N}\end{aligned}$$

而飞机的重量只有 117.7 N。也就是说，这个阵风产生的额外升力比重力还大 36%。

在这种情况下，飞机被猛烈抛起，然后阵风过去又急速下落——这个扰动幅度已经超出了执行器的修正能力。

结论：本构型的地效飞行包线对垂向阵风敏感，1.5 m/s 是可用的上限。实际使用中需要通过：

- 气象条件限制
- 增大巡航高度裕度

- 提高巡航空速(降低 $\Delta\alpha$)

来规避。

把这个失败案例写进报告，比只报喜不报忧更有价值——它给出了明确的使用边界。

10.7 极限低空与量测噪声

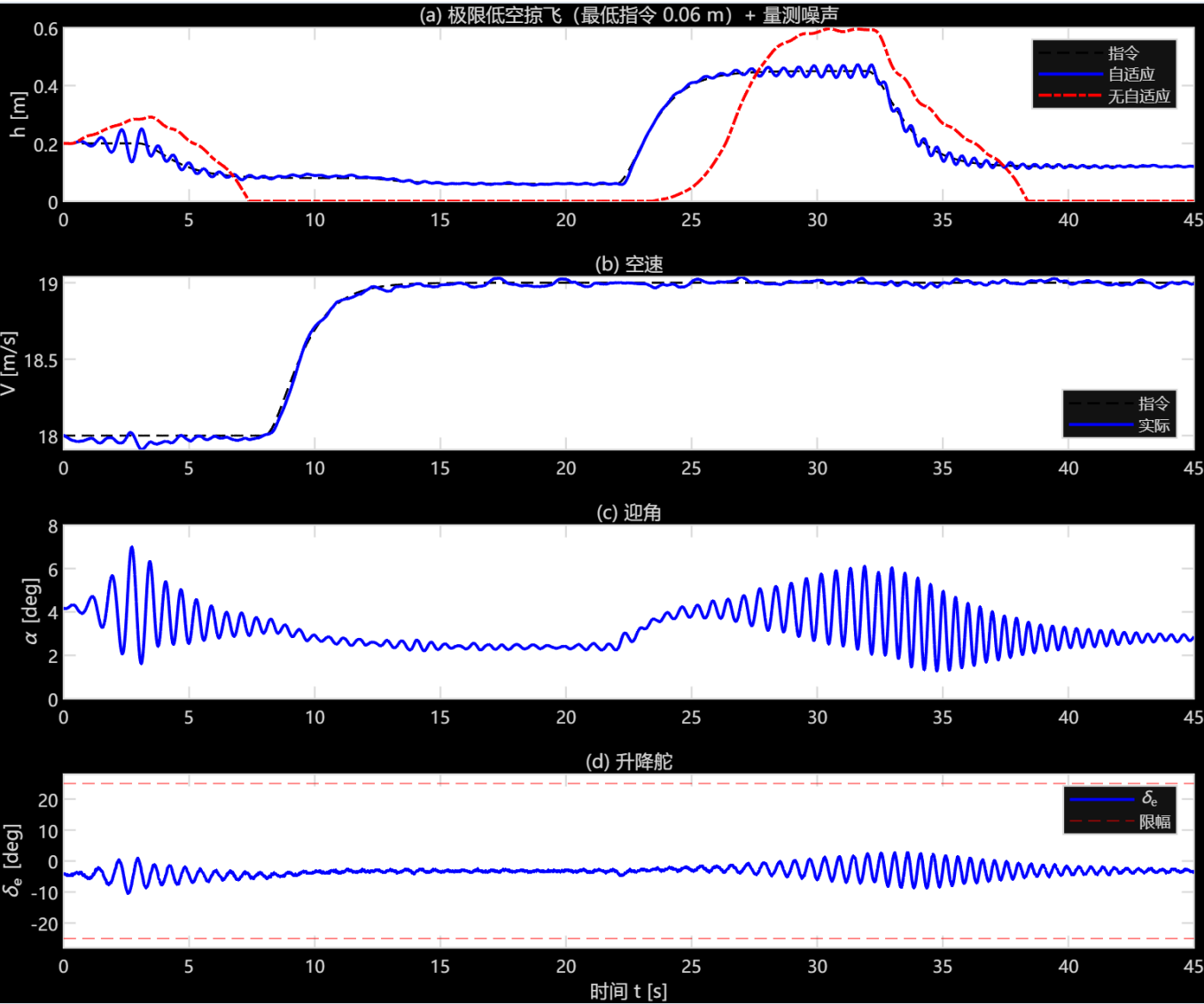


图 10-9 极限低空掠飞 + 量测噪声

最严酷的工况：高度指令下探到 **0.06 m** ($h/c = 0.15$ ，深度地效区)，同时叠加：

- 空速噪声 0.15 m/s
- 迎角噪声 0.15°
- 角速度噪声 $0.3^\circ/s$
- 高度噪声 0.01 m

结果：

方案	高度误差 RMS	最低离地高度	安全
RBF 自适应	0.0082 m	0.0561 m	是
无自适应	0.1295 m	0.0000 m	否

注意 0.0561 m 这个数字：指令最低是 0.06 m，实际最低 0.0561 m，**误差只有 4 mm**。在一架翼展 2.4 m

的飞机上，这是很好的控制精度。
同时也注意：飞机在整个过程中升降舵峰值只有 8.8° ，远未触及限幅。说明控制器工作在舒适的区间。

10.8 蒙特卡洛统计

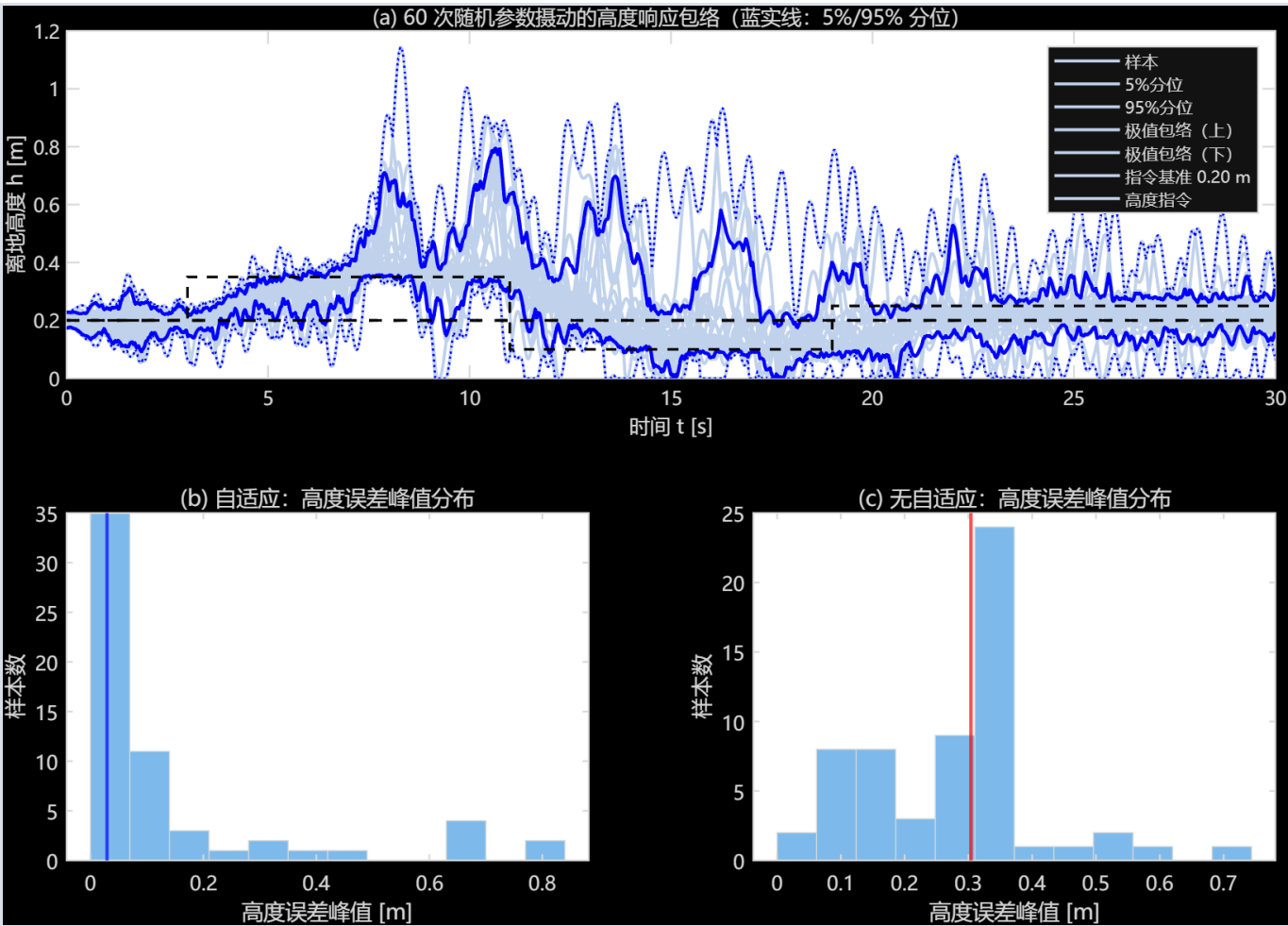


图 10-10 蒙特卡洛高度响应包络与误差分布

方法：在以下范围内独立均匀随机抽样 60 组参数:

参数	摄动范围
质量、惯量	$\pm 25\%$
废阻力	$\pm 40\%$
升力线斜率	$\pm 10\%$
俯仰力矩系数	$\pm 30\%$
地效强度 A	$\pm 60\%$
地效指数 n	$\pm 20\%$
下洗梯度	$\pm 40\%$
尾翼效率	$\pm 15\%$
初始高度偏差	$\pm 0.03\text{ m}$
初始速度偏差	$\pm 1.0\text{ m/s}$

结果:

指标	均值	标准差	中位数	95% 分位	最大值
高度误差 RMS (m)	0.0457	0.0711	0.0104	0.2265	0.2809
高度误差峰值 (m)	0.1399	0.2141	0.0295	0.6629	0.8290
最低离地高度 (m)	0.0776	0.0352	0.0918	0.1001	0.1531

通过率:

- **RBF 自适应:** $52/60 = 86.7\%$
- 无自适应: $24/60 = 40.0\%$

高度误差 RMS 中位数:

- 自适应 **0.0104 m**
- 无自适应 0.2232 m
- **改善 21.5 倍**

怎么读这些数字

均值(0.0457) 远大于中位数(0.0104) , 说明分布是**严重右偏**的——大部分样本表现很好(中位数 0.0104) , 少数样本表现差(最大值 0.28) 拉高了均值。

这才是真实工程的样子: 不是"所有情况都很好"或"都很差", 而是"大多数情况很好, 少数极端情况会退化"。

失败的 8 组有什么共同点: 集中在"质量偏小 + 地效强度偏弱 + 初始高度偏差大"的极端组合。

物理解释: 质量小 -> 同样的力产生更大加速度 -> 更容易被扰动; 地效弱 -> 升力不足; 初始偏差大 -> 一开始就在困难状态。

这不是控制器的缺陷, 而是本构型在无高度裕度条件下的物理限制。

10.9 Simulink 与脚本一致性

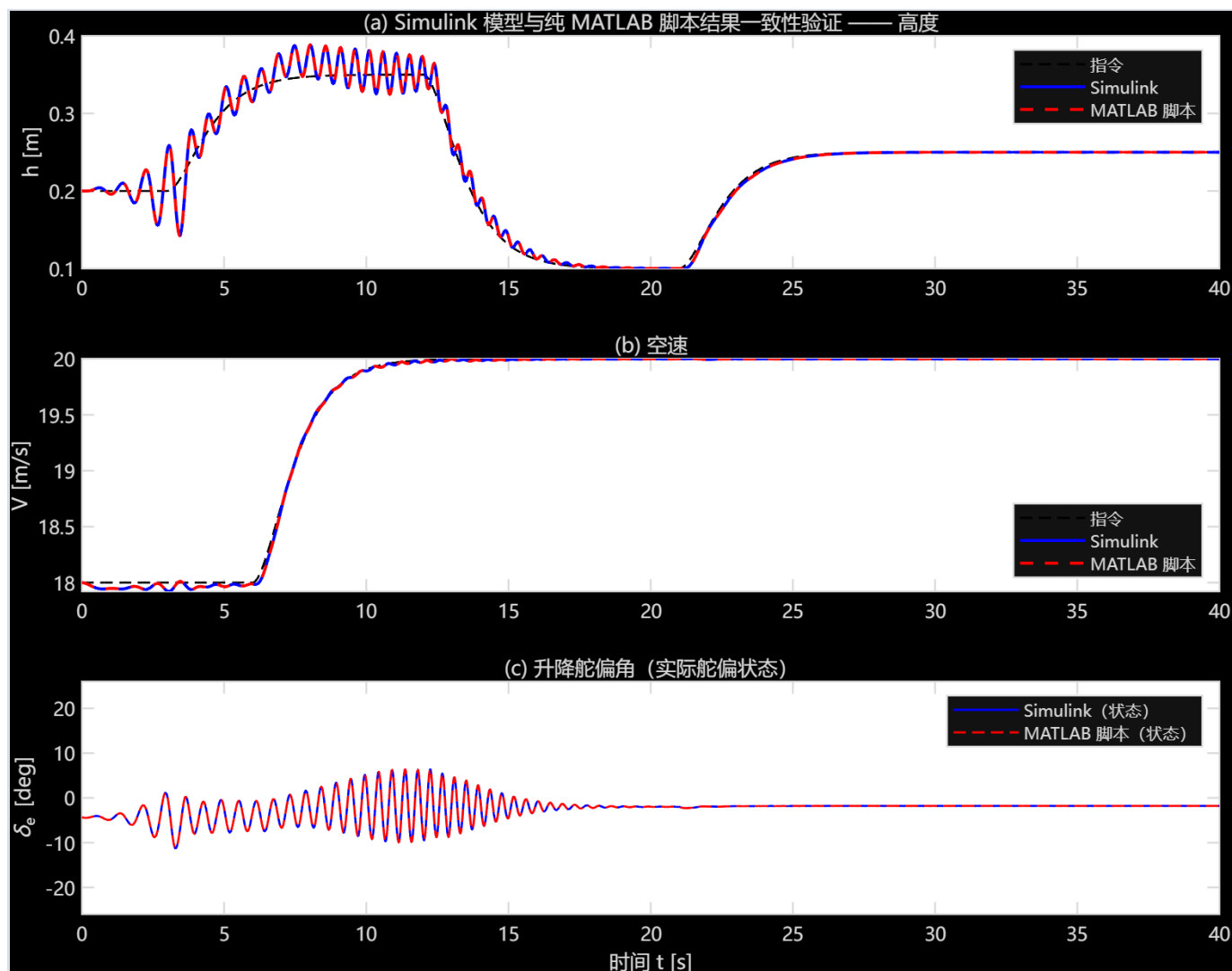


图 10-11 Simulink 与纯 MATLAB 脚本一致性验证

指标	Simulink	MATLAB 脚本	偏差
高度跟踪误差 RMS	0.009959	0.009977	1.7×10^{-5}
高度误差峰值	0.042039	0.041998	4.1×10^{-5}
最低离地高度	0.100019	0.100020	1.6×10^{-6}
最大高度轨迹偏差	—	—	$1.28 \times 10^{-3} \text{ m}$

两种仿真途径的最大轨迹偏差只有 1.3 毫米。

为什么能做到这么一致：因为两者共用同一份控制器和被控对象代码：

- 脚本仿真：直接调用 wig_dynamics.m 和 wig_ctrl_update.m
- Simulink：通过 Level-2 MATLAB S-Function 调用同样两个函数

这是"控制器状态显式传递"这个设计决定的回报(见第 8.9 节) 。

工程意义：Simulink 模型可以作为后续硬件在环(HIL) 和代码生成的基础，而不用担心"模型和算法不一致"。

10.10 全部结论

做到了什么

1. 建立了一个能反映地效本质的纵向模型，并且用"关闭地效"的对照实验证明了模型行为的物理正确性(与经典浮沉理论吻合) 。

2. 定量揭示了两个稳定性问题:

- 高度静不稳定($\partial M/\partial h = +10.8157 \text{ N}\cdot\text{m}/\text{m}$, 其中平尾项 $+10.3302 \text{ N}\cdot\text{m}/\text{m}$, 在 $h = 0.05\sim 1.00 \text{ m}$ 全区间占总量的 95%~106%; 机翼项 $-1.2075 \text{ N}\cdot\text{m}/\text{m}$ 是唯一的稳定贡献, 分项之和与总量的残差仅 $1.4\text{e-}13$)
- 浮沉-高度耦合模态发散(ζ 低至 -0.92 ; 关闭地效后恢复到约 0, 与经典 Lanchester 理论 $\zeta \approx +0.0527$ 同号同量级)

3. 设计并验证了 RBF 自适应控制器:

- 高度误差 RMS 从 0.19 m 降到 **0.010 m** (约 20 倍)
- 蒙特卡洛通过率从 40% 提升到 **86.7%**
- 在 0.06 m 极限低空、量测噪声、中度阵风下安全飞行

4. 给出了完整的理论证明(Lyapunov UUB) 和工程实现(MATLAB + Simulink) 。

没做到什么

1. 只做了纵向, 没有横侧向。
2. 气动模型是准定常线性的, 没有考虑非定常迟滞和大迎角分离。
3. 地效因子参数(A 、 n) 是按公开趋势选定的工程估算值, 不是 CFD/风洞标定结果——本工程没有跑过任何 CFD。ANSYS/data/ge_kappa_template.csv 是由本工程解析式生成的模板, run_00_cfdfit 把它拟合回去只验证了标定算法本身(往返自检) , 不构成模型验证。真正替换成 CFD 数据是 ANSYS/目录里规划的工作。
4. 阵风 3.0 m/s 时失败——给出了明确的使用边界。
5. 鲁棒项参数靠手工整定, 没有系统化的设计方法。

把"没做到什么"写清楚, 比只写"做到了什么"更有价值。

10.11 自测题

1. 为什么"自适应方案的舵偏更大"不是缺点?
2. 蒙特卡洛里均值远大于中位数, 说明分布是什么形状? 这对工程评价有什么影响?
3. 3.0 m/s 阵风失败的物理原因是什么? 请从升力与重力的比值来算。
4. 为什么"关闭地效后模态恢复稳定"这个对照实验, 比"多个高度都发散"更有说服力?
5. Simulink 和脚本能对到 1.3 mm , 最关键的设计决定是什么?

第 11 章 附录

11.1 符号表

状态量

符号	名称	单位	说明
v	空速	m/s	飞机相对气流的速度
γ	航迹倾角	rad	实际运动方向相对水平面，爬升为正
α	迎角	rad	机翼弦线相对气流的夹角
θ	俯仰角	rad	机头相对水平面，抬头为正; $\theta = \gamma + \alpha$
q	俯仰角速度	rad/s	$d\theta/dt$
h	离地高度	m	
T	发动机推力	N	
δ_e	升降舵偏角	rad	正 = 后缘下偏 = 低头

气动量

符号	名称	单位/取值
q_∞	动压 = $0.5\rho v^2$	Pa
ρ	空气密度	1.225 kg/m ³
L	升力	N
D	阻力	N
M	俯仰力矩	N·m, 抬头为正
C_L	升力系数	无量纲
C_D	阻力系数	无量纲
C_D0	废阻力系数	0.025(名义)
C_Di	诱导阻力系数	无量纲
C_mac	机翼绕气动中心力矩系数	-0.05
a0	二维升力线斜率	5.90 /rad
a_w	有效升力线斜率	随高度变化
α_{L0}	零升迎角	-3.0°
AR	展弦比 = b^2/S	6.0
e	奥斯瓦尔德效率因子	0.85
K	地效因子	0 到 1
ϵ	平尾处下洗角	rad
$\epsilon_0, \epsilon_\alpha$	下洗常数项、梯度	0.02, 0.35

几何与质量

符号	名称	取值
----	----	----

符号	名称	取值
b	翼展	2.40 m
c	弦长	0.40 m
S	机翼面积	0.96 m ²
m	质量	12.0 kg
I_{yy}	俯仰转动惯量	1.15 kg·m ²
l_t	尾力臂	0.95 m
S_t	平尾面积	0.18 m ²
i_t	平尾安装角	-2.0°
Δz_t	平尾抬高量	0.30 m
x_{acw}	机翼气动中心相对重心位置	+0.02 m(在前)
τ_e	舵机时间常数	0.05 s
τ_T	发动机时间常数	0.15 s
$\delta_{e,max}$	升降舵限幅	± 25°
T_{max}	最大推力	35 N

控制器量

符号	名称	取值
k_h	高度环增益	2.00 /s
k_{hd}	高度率阻尼	0.80
k_θ	俯仰角环增益	3.20 /s
k_q	角速度环增益	6.00 /s
k_v	速度环比例	1.53 /s
k_{IV}	速度环积分	0.42 /s ²
$\tau_g, \tau_\theta, \tau_q$	各级指令滤波时间常数	0.010 / 0.100 / 0.010 s
Γ_1, Γ_2	自适应增益	0.005, 3.0
φ_1, φ_2	完整回归向量 = $\varphi_{rbf}(q\infty Sc/l_{yy})$ 与 φ_{rbf}/m	配平点 $\ \varphi_1\ ^2 \approx 2.369e4, \ \varphi_2\ ^2 \approx 3.732e-2$
k_i	第 i 通道比例增益(离散判据用)	$k_q = 6.00, k_v = 1.53 /s$
W_{max}	权值投影上界	12.0
v_e	e-修正泄漏系数	0.50
η_r, Φ_r	力矩通道鲁棒增益、边界层	2.50, 0.80
η_v, Φ_v	速度通道鲁棒增益、边界层	1.50, 1.00
T_s	控制器采样周期	0.005 s(200 Hz)
dt	积分步长	0.001 s

11.2 术语中英对照

中文	英文	缩写	说明
地效飞行器	Wing-In-Ground effect vehicle	WIG	
地面效应	ground effect	GE	
地效因子	ground effect factor	K	下洗保留系数

中文	英文	缩写	说明
迎角	angle of attack	AoA	
航迹倾角	flight path angle	—	
俯仰角	pitch angle	—	
配平	trim	—	静平衡状态
线性化	linearization	—	泰勒展开取一次项
特征值	eigenvalue	—	
模态	mode	—	
短周期模态	short-period mode	—	
浮沉模态	phugoid mode	—	
阻尼比	damping ratio	ζ	
相位裕度	phase margin	PM	
带宽	bandwidth	—	
动态逆	Nonlinear Dynamic Inversion	NDI	
反步法	backstepping	—	
动态面控制	Dynamic Surface Control	DSC	
滑模控制	Sliding Mode Control	SMC	
模型参考自适应	Model Reference Adaptive Control	MRAC	
径向基函数	Radial Basis Function	RBF	
一致最终有界	Uniformly Ultimately Bounded	UUB	
投影算子	projection operator	—	
边界层	boundary layer	Φ	
抖振	chattering	—	
抗积分饱和	anti-windup	—	
零阶保持	zero-order hold	ZOH	
龙格-库塔法	Runge-Kutta method	RK4	
蒙特卡洛	Monte Carlo	—	
消融实验	ablation study	—	逐项关闭以验证贡献
硬件在环	Hardware-In-the-Loop	HIL	

11.3 数学工具速查

泰勒展开与线性化

$$f(x) \approx f(x_0) + f'(x_0) \cdot (x - x_0)$$

多元形式：偏导数排成雅可比矩阵。

中心差分

$$f'(x) \approx [f(x+\Delta) - f(x-\Delta)] / (2\Delta)$$

步长选择： $\Delta = 10^{-6} \times \max(1, |x|)$ 。太小有舍入误差，太大有非线性误差。

复特征值

$\lambda = \sigma \pm j\omega$
 $\omega_n = |\lambda| = \sqrt{\sigma^2 + \omega^2}$ 固有频率
 $\zeta = -\sigma/\omega_n$ 阻尼比
 时间常数 = $1/|\sigma|$

- $\sigma < 0$: 稳定; $\sigma > 0$: 发散
- $\zeta < 1$: 欠阻尼(振荡); $\zeta > 1$: 过阻尼

一阶惯性环节

$\tau \cdot dy/dt = u - y$

- 阶跃响应: $y(t) = u \cdot (1 - e^{-(t/\tau)})$
- 3τ 到达约 95%
- 带宽 $\approx 1/\tau$ (rad/s)

欧拉法离散的稳定条件

$y(k+1) = y(k) + T_s/\tau \cdot (u - y(k))$
 $= y(k) \cdot (1 - T_s/\tau) + \dots$

要求 $|1 - T_s/\tau| < 1$, 即 $T_s/\tau < 2$ 。

离散自适应律的稳定条件

自适应律 + 误差反馈的两状态线性化系统, 特征多项式为:

$\lambda^2 - (1 - T_s \cdot k_i) \cdot \lambda + (1 - T_s \cdot k_i + T_s^2 \cdot \Gamma_i \cdot \|\varphi_i\|^2) = 0$

由 $\det < 1$ 得到判据(k_i = 该通道比例增益, φ_i = 含缩放因子的完整回归向量) :

$T_s \cdot \Gamma_i \cdot \|\varphi_i\|^2 < k_i$

本工程配平点($V = 18$ m/s、 $h = 0.20$ m) 代入值:

通道	$T_s \cdot \Gamma_i \cdot \ \varphi_i\ ^2$	k_i	裕度比
力矩($\Gamma_1 = 0.005$)	0.5922	$k_q = 6.00$	10.1
速度($\Gamma_2 = 3.0$)	5.598e-4	$k_V = 1.53$	2733

双曲正切不等式

$|z| - z \cdot \tanh(z/\Phi) \leq 0.2785 \cdot \Phi$

用于鲁棒项的 Lyapunov 放缩。

常用小角度近似($< 10^\circ$)

$\sin(x) \approx x$, $\cos(x) \approx 1 - x^2/2 \approx 1$, $\tan(x) \approx x$

11.4 常见问题 FAQ

Q1: 为什么不用 PID? PID 不是万能的吗?

PID 是"万能"的, 但前提是**对象特性已知且不剧烈变化**。本工程的对象有三个 PID 无法处理的特点:

1. 气动参数随高度非线性剧变($\pm 40\%$)
2. 对象本身不稳定(浮沉模态 $\zeta < 0$)

3. 存在模型里根本没有的效应(RAM 效应)
而且本工程实测过: PID 结构在高度机动时会触地。

Q2: 神经网络会不会"学坏", 把飞机搞得更糟?

有三重保护:

1. **投影算子**限制权值范数上界
2. **局部支撑**: 飞出训练范围时网络输出自动趋零, 退化为"动态逆 + 鲁棒项"
3. **饱和冻结**: 执行器饱和时停止学习, 避免学错

Q3: 为什么不用更深的神经网络(比如 LSTM) ?

三个原因:

1. **Lyapunov 证明需要"对权值线性"**。深层网络做不到。
2. **实时性**: 200 Hz 采样意味着 5 ms 内要算完, 深层网络推理太慢。
3. **可解释性**: RBF 的权值有明确物理意义(力矩系数), 深度网络是黑箱, 出了事没法查。

Q4: 为什么高度控制不用气压高度表, 而要用雷达高度计?

气压高度表测的是**绝对高度**(相对海平面), 精度约几米。而我们需要精度是**厘米级**。只有雷达/激光测距才能达到。

Q5: 如果雷达高度计失灵怎么办?

这是**适航性**问题。工程上通常的做法:

- 冗余传感器 + 表决
- 高度低到阈值时自动拉起(包线保护)
- 本工程的代码里已经有 $\theta_{\max} = 20^\circ$ 这样的保护, 但**没有做传感器故障检测**——这是下一步工作。

Q6: 为什么仿真步长是 1 ms, 而控制器是 5 ms?

- 控制器 5 ms: 模拟真实飞控计算机的采样周期(200 Hz 是航空常用的速率)
- 积分 1 ms: 为了让被控对象的数值积分足够精确

两者必须分开。如果积分步长也用 5 ms, 舵机动态($\tau = 0.05\text{ s}$) 只有 10 个积分点, 精度不够。

Q7: 为什么要在模型里故意制造失配? 这不是自找麻烦吗?

恰恰相反。如果真实对象和控制器模型完全一致, 那么:

- 仿真会很容易通过
- 但**什么也证明不了**——因为真实世界永远不会有"模型完全准确"的情况

故意制造失配, 是为了让仿真结果有工程意义。

Q8: 这个控制器能直接上真机吗?

不能, 还差几步:

1. 需要真实的气动数据(CFD 或风洞) 替换当前的**工程估算值**——注意本工程的地效参数 A 、 n 只是按公开趋势选定的, **没有做过 CFD, 也没有做过风洞试验**, ANSYS/ 目录目前只有接口模板, 且模板数据是由本工程解析式反向生成的
2. 需要传感器故障检测与冗余管理
3. 需要半物理仿真(HIL) 验证
4. 需要飞行试验逐步放开包线

本工程完成的是"理论与仿真验证"这一步。

Q9: 为什么自适应增益两个通道差了 600 倍?

因为回归向量的量纲不同: 力矩通道的回归向量是 $\varphi_1 = \varphi_{\text{rbf}}(q \propto S c / l_{\text{yy}})$ (带 $1/\text{s}^2$ 量纲, 配平点缩放因子 66.265 s^{-2}), 速度通道是 $\varphi_2 = \varphi_{\text{rbf}}/m$ (带 $1/\text{kg}$ 量纲, 缩放因子 0.083333 kg^{-1}), 两个缩放因子本身相差 795 倍。

不能直接比较 Γ 的数值, 也不能比较 $\Gamma \cdot g^2$ (力矩 21.8、阻力

0.0207，差三个数量级）。要看的是判据里的无量纲比值：

```
T_sΓ_i||q_i||^2 / k_i  力矩 0.0987，速度 3.66e-4，两者都 << 1
```

Q10: 为什么不用卡尔曼滤波估计迎角？

可以用，但本工程假设迎角可测（小型无人机用风标或 α 传感器是常规做法）。加卡尔曼滤波会增加复杂度，但对核心结论没有影响。

如果要**做**：应该估计 α 和 q ，因为这两个量对噪声最敏感。

11.5 文件清单与运行说明

目录结构

```
matlab_simulink/
├── README.md          工程总说明
├── src/               MATLAB 源码( 约 40 个 .m 文件)
├── models/wig_adaptive_fcs.slx Simulink 模型
├── figures/           仿真图( 11 张)
├── results/           文本日志与 .mat 数据
├── docs/              报告与本教程
│   ├── tutorial_src/   本教程的 Markdown 源( 唯一事实来源)
│   └── 地效飞行器自适应飞控_零基础详解.pdf
└── tools/
    ├── pdf/           PDF 排版工具链( 本教程由它生成)
    └── word/           docx 报告工具链
```

运行方法

```
cd matlab_simulink/src
addpath(pwd)

run_all    % 一键运行全部算例( 约 5 分钟)
```

或单独运行:

命令	内容	耗时
run_00_smoke	自检冒烟测试	5 s
run_00_mdh	dM/dh 四项分解 (图 12)	10 s
run_00_stab	纵向稳定性诊断(模态、浮沉理论校核、逐高度扫描)	10 s
run_00_clp	闭环极点谱排查(出厂增益 vs 历史中间态增益)	60 s
run_00_cdfit	ANSYS CFD 数据接口往返自检	5 s
run_analysis	模型分析(图 1~3)	20 s
run_scenarios	场景仿真(图 4~9)	60 s
run_montecarlo	蒙特卡洛 60 组(图 10)	200 s
run_simulink	Simulink 模型构建与验证(图 11)	30 s

重新生成本教程 PDF

```
cd matlab_simulink/tools/pdf
python check_glyphs_pdf.py  # 先查字体覆盖( 防止正文出现缺字方框)
python md2pdf_tut.py        # Markdown -> PDF
python verify_pdf.py        # 校验: 页数、内嵌字体、越界、缺字
```

11.6 延伸阅读建议

按"从易到难"排序:

飞行力学入门

- 《飞行力学》或任何一本航空院校教材: 讲清楚六自由度运动方程的推导
- Etkin《Dynamics of Flight》: 经典教材, 稳定性和操纵性讲得很透

地效飞行器

- Rozhdestvensky, "Wing-in-ground effect vehicles"(Progress in Aerospace Sciences, 2006): 综述, 涵盖气动、稳定性、控制
- 关键词检索: WIG craft longitudinal stability、ground effect aerodynamic model

非线性控制

- Khalil《Nonlinear Systems》: Lyapunov 稳定性理论的权威教材
- Slotine & Li《Applied Nonlinear Control》: 工程导向, 反步法和自适应讲得很好

自适应控制与神经网络

- Ioannou & Sun《Robust Adaptive Control》: 投影算子、 σ 修正的原始文献
- Lewis et al., "Neural Network Control of Robot Manipulators and Nonlinear Systems": RBF 自适应控制的经典
- Farrell & Polycarpou《Adaptive Approximation Based Control》: 把函数逼近和控制结合得最好的一本

动态面控制

- Swaroop et al., "Dynamic surface control for a class of nonlinear systems"(IEEE TAC, 2000): DSC 的原始论文

建议的阅读顺序

- 第1步: 飞行力学教材 -> 搞懂运动方程
- 第2步: Khalil 第4章 -> 搞懂 Lyapunov
- 第3步: Slotine 第6章 -> 搞懂反步法
- 第4步: Ioannou 第4章 -> 搞懂自适应律和投影
- 第5步: Lewis 第2~3章 -> 搞懂 RBF 神经网络控制
- 第6步: 回到本工程的代码, 对着第7章的推导看

不要试图一次全懂。 这份教程的作用是给你一张地图, 告诉你每一块知识在整个体系里的位置。真正理解需要反复回来对照。